
Dossier de Certification RNCP 37873

Concepteur Développeur d'Applications — Projet Social Network

Christophe Lecart

Juin 2026

Sommaire

1	00 - Présentation du Candidat	10
1.1	Identité	10
1.2	En une phrase	10
1.3	Développeur full stack en reconversion après plus de 20 ans dans l'industrie, je finalise mon Bac+3 CDA à Zone01 Rouen Normandie (juin 2026) et prépare une alternance pour poursuivre en Bac+5 Chef de Projet, Architecture et Green IT (juin 2026 – juin 2028). .	10
1.4	Le récit : d'où je viens, pourquoi je change	10
1.4.1	20 ans de terrain industriel	10
1.4.2	Le déclic	11
1.4.3	Zone01 Rouen Normandie — depuis 2024	11
1.5	Projets réalisés	11
1.6	Ce que la formation m'a apporté (au-delà des technos)	13
1.6.1	Compétences techniques	13
1.6.2	Compétences transverses gagnées	13
1.6.3	Mon évolution avant / après le projet Social Network	13
1.7	Vision du développeur que je veux être	14
1.7.1	Mes forces	14
1.7.2	Mes faiblesses (et comment je les travaille)	14
1.7.3	Ce que je ne veux pas faire	14
1.7.4	Ce que je veux faire	15
1.8	Recherche d'alternance	15
1.9	Parcours professionnel et formation	15
1.9.1	Parcours	15
1.9.2	Formation	15
1.10	Compétences Techniques	16
1.11	Qualités Professionnelles	16
1.12	Centres d'Intérêt	16
1.13	Langues	16
2	01 - Introduction	17
2.1	Contexte et Objectif	17
2.1.1	Problématique	17
2.1.2	Solution Proposée	17
2.2	Informations Projet	17
2.3	Équipe et Contexte	17
2.3.1	Contexte du Code Initial	18
2.4	Sommaire du Dossier	18
2.5	Critères de Réussite	19
3	02 - Cahier des Charges	20
3.1	Objectifs Principaux	20
3.1.1	Organisation GitHub du projet	20

3.1.2	Fonctionnalités Essentielles	20
3.2	Contraintes Techniques	21
3.2.1	Sécurité	21
3.2.2	Scalabilité	21
3.2.3	Compatibilité	21
3.2.4	Performance	21
3.3	User Stories	22
3.4	Objectifs de Qualité	22
3.5	Timeline	22
4	03 - Conception	24
4.1	Résumé	24
4.2	Documents du Projet	24
4.2.1	Pilotage du travail	24
4.2.2	1. Pages et Wireframes	24
4.2.3	2. Dictionnaire de Données	24
4.2.4	3. User Stories	25
4.2.5	4. Diagrammes UML Complets	25
4.3	Fonctionnalités Clés	25
4.3.1	Réseautage Social	25
4.3.2	Communication	25
4.3.3	Groupes & Événements	26
4.3.4	Discovery	26
4.3.5	Personalization	26
4.4	Architecture de Données	26
4.4.1	Modèles Principaux	26
4.4.2	Statistiques	27
4.5	Pages Implémentées	27
4.6	Design System	27
4.6.1	Palette Couleurs (Thème Dark)	27
4.6.2	Typography	28
4.6.3	Breakpoints	28
4.6.4	Components	28
4.7	Data Flow	29
4.8	Diagrammes	29
4.8.1	MCD - Modèle Conceptuel de Données	29
4.8.2	MLD - Modèle Logique de Données	29
4.9	Wireframes & Maquettes	30
4.9.1	Wireframes basse-fidélité	30
4.9.2	Maquettes haute-fidélité	33
4.10	Checklist Conception	36
4.11	Fichiers Liés	36

5	User Stories & Cas d'Usage	38
5.1	Vue d'Ensemble	38
5.2	Preuves & Mapping GitHub	38
5.3	Utilisateur Classique (Regular User)	38
5.3.1	Authentification & Profil	38
5.3.2	Posts & Contenu	40
5.3.3	Stories	42
5.3.4	Amitié & Suivi	43
5.3.5	Messagerie	44
5.3.6	Recherche	45
5.3.7	Notifications	45
5.3.8	Groupes (Communities)	46
5.3.9	Événements (Events)	47
5.3.10	Reels (Vidéos)	48
5.4	Priorités & Estimation	49
5.4.1	Phase 1: MVP (MUST HAVE) - 4 semaines	49
5.4.2	Phase 2: Features (SHOULD HAVE) - 2 semaines	50
5.4.3	Phase 3: Polish (NICE TO HAVE) - 1 semaine	50
5.5	Acceptance Criteria Summary	50
5.6	Prochaines Étapes	50
6	Modélisation de Données	51
6.1	Dictionnaire de Données	51
6.2	Model: User	51
6.2.1	Attributs	51
6.2.2	Relations	52
6.3	Model: Post	52
6.3.1	Attributs	52
6.3.2	Enum	53
6.3.3	Relations	53
6.4	Model: Comment	53
6.4.1	Attributs	53
6.4.2	Relations	54
6.5	Model: Reaction	54
6.5.1	Attributs	54
6.5.2	Contraintes d'unicité	54
6.5.3	Relations	54
6.6	Model: Message	55
6.6.1	Attributs	55
6.6.2	Relations	55
6.7	Model: Friendship	55
6.7.1	Attributs	56
6.7.2	Contrainte d'unicité	56

6.7.3	Relations	56
6.8	Model: Story	56
6.8.1	Attributs	56
6.8.2	Relations	57
6.9	Model: Conversation	57
6.9.1	Attributs	57
6.9.2	Relations	57
6.10	Model: ConversationMember	58
6.10.1	Attributs	58
6.10.2	Contrainte d'unicité	58
6.11	Model: GroupMessage	58
6.11.1	Attributs	58
6.12	Model: GroupInvitation	59
6.12.1	Attributs	59
6.12.2	Contrainte d'unicité	59
6.13	Model: GroupJoinRequest	59
6.13.1	Attributs	59
6.13.2	Contrainte d'unicité	60
6.14	Model: GroupMember	60
6.14.1	Attributs	60
6.14.2	Contrainte d'unicité	60
6.15	Model: Event	60
6.15.1	Attributs	61
6.15.2	Relations	61
6.16	Model: Rsvp	61
6.16.1	Attributs	61
6.16.2	Contrainte d'unicité	62
6.17	Model: UserSettings	62
6.17.1	Attributs	62
6.18	Model: Notification	62
6.18.1	Attributs	62
6.18.2	Index	63
6.19	Model: Account	63
6.19.1	Attributs	63
6.19.2	Contrainte d'unicité	63
6.20	MLD — Modèle Logique de Données	63
6.21	Cardinalités	64
6.22	Checklist Modélisation	65
6.23	Diagrammes Complets	65
7	Diagrammes UML - Social Network (Conforme Prisma)	66
7.1	Objectif	66
7.2	1. MCD (Modèle Conceptuel de Données)	66

7.3	2. MLD (Modèle Logique de Données)	70
7.4	3. MPD (Modèle Physique de Données - PostgreSQL)	75
7.5	Utilisation	79
8	04 - Développement	80
8.1	Objectif	80
8.1.1	Preuves & Mapping GitHub	80
8.2	Diagrammes d'Architecture (Mermaid)	80
8.2.1	Architecture Système	80
8.2.2	Auth Flow (Login)	81
8.2.3	Real-time Messaging Sequence (SSE + Redis Polling)	81
8.2.4	Schéma ER (Simplifié)	82
8.3	Authentification & Middleware	82
8.4	Temps réel (Upstash Redis + SSE Polling)	82
8.5	Endpoints API	83
8.5.1	Résumé Rapide	83
8.6	Implémentation - Code Samples	84
8.6.1	Authentification (Middleware)	84
8.6.2	API Route Sample	84
8.6.3	Real-time avec Redis + SSE Polling	85
8.7	Tests & CI/CD	86
8.7.1	Tests	86
8.7.2	GitHub Actions (CI/CD)	86
8.8	Structure de Code	87
8.8.1	Directories Clés	87
8.8.2	Validation & Error Handling	88
8.9	Checklist Implémentation	88
9	Sécurité et conformité RGPD	89
9.1	Objectif	89
9.2	1. Authentification et sessions	89
9.2.1	1.1 JWT signé via <code>jose</code>	89
9.2.2	1.2 Hashage des mots de passe avec <code>bcrypt</code>	89
9.2.3	1.3 Cookies HTTP-only	90
9.2.4	1.4 Middleware d'autorisation par défaut	90
9.2.5	1.5 OAuth Google : protection CSRF par <code>state</code>	91
9.3	2. Protection contre les attaques classiques	91
9.3.1	2.1 Injection SQL — Prisma	91
9.3.2	2.2 Cross-Site Scripting (XSS) — React + <code>audit dangerouslySetInnerHTML</code>	91
9.3.3	2.3 Cross-Site Request Forgery (CSRF)	91
9.3.4	2.4 Validation des entrées — Zod	92
9.3.5	2.5 Headers HTTP de sécurité	92
9.3.6	2.6 Rate limiting	93
9.4	3. Upload de fichiers (avatars, posts, stories)	93

9.5	4. Conformité RGPD	93
9.5.1	4.1 Données personnelles collectées	93
9.5.2	4.2 Principe de minimisation — respecté	94
9.5.3	4.3 Droits RGPD de la personne concernée	94
9.5.4	4.4 Consentement et âge	95
9.5.5	4.5 Mentions légales et politique de confidentialité	95
9.5.6	4.6 Sécurité des données en transit et au repos	95
9.5.7	4.7 Logs et durée de conservation	95
9.6	5. Roadmap de durcissement	96
9.6.1	Critique — avant mise en production	96
9.6.2	Important — sous 1 mois post-prod	96
9.6.3	Bonus — confort exploitation	97
9.7	6. Synthèse pour le jury RNCP	98
9.8	7. Références	99
10	Stratégie de tests	100
10.1	Objectif	100
10.2	1. Pyramide de tests visée	100
10.3	2. État actuel	100
10.3.1	2.1 Infrastructure de test en place	100
10.3.2	2.2 Test existant — extrait	101
10.3.3	2.3 Lancer les tests	102
10.4	3. Plan d'extension (par priorité)	102
10.4.1	Critique — à implémenter rapidement	102
10.4.2	Important — sous 1 mois	103
10.4.3	Bonus	104
10.5	4. Jeux d'essai documentés	104
10.5.1	Authentification	104
10.5.2	Posts et interactions	105
10.5.3	Messagerie temps réel	106
10.5.4	Groupes et événements	106
10.5.5	Sécurité	107
10.6	5. Outils et commandes	108
10.7	6. Position pour la soutenance RNCP	108
10.8	7. Références	108
11	Veille technique et recherche personnelle	109
11.1	Objectif	109
11.2	1. Domaines techniques explorés	109
11.2.1	1.1 Server-Sent Events vs WebSockets vs polling	109
11.2.2	1.2 JWT stateless vs sessions Redis	110
11.2.3	1.3 Prisma : pourquoi et limites	110
11.2.4	1.4 Bun en runtime production	111
11.2.5	1.5 Sécurité applicative — au-delà du cours	111

11.2.6	1.6 Architecture serverless vs serveur traditionnel	112
11.3	2. Sources d'apprentissage	112
11.3.1	Documentation officielle (source primaire, toujours)	113
11.3.2	Sécurité et bonnes pratiques	113
11.3.3	Articles techniques et retours d'expérience	113
11.3.4	Communautés	113
11.4	3. Méthode personnelle de veille	113
11.4.1	3.1 Curation hebdomadaire	113
11.4.2	3.2 Veille appliquée	114
11.4.3	3.3 Outils utilisés pour la veille	114
11.5	4. Points où je me suis surpris	114
11.6	5. Application concrète au dossier	115
11.7	6. Plan de veille post-certification	115
12	05 - Déploiement	116
12.1	Objectif	116
12.2	Preuves & Mapping GitHub	116
12.3	Docker et Containerisation	116
12.3.1	Services principaux	116
12.3.2	docker-compose.yml	116
12.3.3	Dockerfile	117
12.4	Bases de Données	118
12.4.1	PostgreSQL sur Neon	118
12.4.2	Redis	118
12.5	Mise en Production	118
12.5.1	Vercel	118
12.5.2	Variables d'environnement	118
12.5.3	Contrôles post-déploiement	119
12.6	CI/CD	119
12.6.1	GitHub Actions	119
12.7	Monitoring et Exploitation	120
12.7.1	Observabilité	120
12.7.2	Réversibilité	120
12.8	Checklist de Déploiement	120
12.9	Résultat Attendu	120
13	06 - Bilan	121
13.1	Objectif	121
13.2	Retour d'Expérience	121
13.2.1	Défis Techniques Rencontrés	121
13.2.2	Défis d'Organisation et de Documentation	121
13.2.3	Apprentissages Clés	122
13.3	Accomplissements Mesurables	122
13.3.1	Livrables du Projet	122

13.3.2	Résultats Obtenus	122
13.3.3	Points Forts du Projet	123
13.3.4	Points à Améliorer (Phase Suivante)	124
13.4	Compétences Validées (RNCP 37873)	124
13.4.1	Bloc 1 : Développer une application sécurisée	124
13.4.2	Bloc 2 : Concevoir une application organisée en couches	124
13.4.3	Bloc 3 : Préparer le déploiement sécurisé	124
13.4.4	Compétences Techniques Consolidées	124
13.4.5	Compétences Transversales	125
13.5	Améliorations Futures	125
13.5.1	Court Terme	125
13.5.2	Moyen Terme	125
13.5.3	Long Terme	125
13.6	Métriques du Projet	126
13.7	Conclusion	126
14	Conformité RNCP 37873 – Social Network	127
14.1	Structure du Référentiel RNCP 37873	127
14.2	Bloc 1 – Développer une application sécurisée (BC01)	127
14.2.1	C1 – Installer et configurer son environnement de travail	127
14.2.2	C2 – Développer des interfaces utilisateur	128
14.2.3	C3 – Développer des composants métier	128
14.2.4	C4 – Contribuer à la gestion de projet	129
14.3	Bloc 2 – Concevoir et développer une application sécurisée organisée en couches (BC02)	129
14.3.1	C5 – Analyser les besoins et maquetter une application	129
14.3.2	C6 – Définir l’architecture logicielle d’une application	130
14.3.3	C7 – Concevoir une base de données relationnelle	131
14.3.4	C8 – Développer des composants d’accès aux données	132
14.4	Bloc 3 – Préparer le déploiement d’une application sécurisée (BC03)	132
14.4.1	C9 – Préparer et exécuter les plans de tests d’une application	132
14.4.2	C10 – Préparer et documenter le déploiement d’une application	133
14.4.3	C11 – Contribuer à la mise en production dans une démarche DevOps	134
14.5	Tableau de Synthèse – Couverture RNCP 37873	134
14.6	Correspondance Dossier → Blocs RNCP	135
15	Mapping RNCP 37873 – 11 compétences → preuves projet	136
15.1	Vue d’ensemble	136
15.2	BLOC 1 – Développer une application sécurisée	137
15.2.1	Compétence 1 – Installer et configurer son environnement de travail en fonction du projet	137
15.2.2	Compétence 2 – Développer des interfaces utilisateur	138
15.2.3	Compétence 3 – Développer des composants métier	139
15.2.4	Compétence 4 – Contribuer à la gestion d’un projet informatique	140

15.3	BLOC 2 — Concevoir et développer une application sécurisée organisée en couches . . .	140
15.3.1	Compétence 5 — Analyser les besoins et maquetter une application	140
15.3.2	Compétence 6 — Définir l’architecture logicielle d’une application	141
15.3.3	Compétence 7 — Concevoir et mettre en place une base de données relationnelle	142
15.3.4	Compétence 8 — Développer des composants d’accès aux données SQL et NoSQL	143
15.4	BLOC 3 — Préparer le déploiement d’une application sécurisée	143
15.4.1	Compétence 9 — Préparer et exécuter les plans de tests d’une application . . .	143
15.4.2	Compétence 10 — Préparer et documenter le déploiement d’une application . .	144
15.4.3	Compétence 11 — Contribuer à la mise en production dans une démarche DevOps	145
15.5	Compétences transverses	146
15.5.1	Sécurité de l’application	146
15.6	Vue d’ensemble — couverture estimée	146
15.7	Recommandation de lecture pour le jury	147
16	Contribution personnelle de Christophe Lecart	148
16.1	Pourquoi ce document	148
16.2	Vue d’ensemble	148
16.3	Contribution par domaine	148
16.3.1	1. Système de réactions universel (posts, stories, commentaires)	148
16.3.2	2. Design system et thème (palette, dark mode, layout)	150
16.3.3	3. Création de posts (dialog responsive, médias multiples)	150
16.3.4	4. Stories (timer, background dominant, likes)	151
16.3.5	5. Chat (bulles, emoji picker, layout)	151
16.3.6	6. Recherche stylée avec filtres	152
16.3.7	7. Commentaires (rework UX)	152
16.3.8	8. Profil et navigation	153
16.3.9	9. Sécurité et authentification (fix cookie JWT)	153
16.3.10	DevOps et stabilisation (Prisma, dépendances)	153
16.3.11	1.1. Dossier de certification (intégralité)	154
16.4	Synthèse — couverture personnelle des 11 compétences	154
16.5	Phrase d’ancrage pour la soutenance	156
16.6	Liens directs vérifiables	156

1 00 - Présentation du Candidat

1.1 Identité

- **Nom** : Lecart
 - **Prénom** : Christophe
 - **Date de naissance** : 15 décembre 1981
 - **Localisation** : 76510 Notre-Dame d'Aliermont
 - **Contact** : djlike@hotmail.fr | 06.70.02.80.47
 - **Permis** : B / A
 - **Portfolio** : <https://clecart.fr>
-

1.2 En une phrase

1.3 Développeur full stack en reconversion après plus de 20 ans dans l'industrie, je finalise mon Bac+3 CDA à Zone01 Rouen Normandie (juin 2026) et prépare une alternance pour poursuivre en Bac+5 Chef de Projet, Architecture et Green IT (juin 2026 – juin 2028).

1.4 Le récit : d'où je viens, pourquoi je change

1.4.1 20 ans de terrain industriel

J'ai commencé en alternance en 2000 chez GPH comme technicien de maintenance, puis en 2002 chez Lubrizol en bureau d'études. Sorti de formation BTS MAI en 2003, j'ai enchaîné en missions d'intérim (manutentionnaire, technicien, régisseur technique), puis chez KONE Ascenseurs (2004-2010) comme technicien hautement qualifié, régleur et mise en service, avant de rejoindre SOCOTEC Équipements pendant 14 ans (2010-2024) : expert en vérifications générales périodiques et mise en service (ascenseurs, levage, machines, EPI), collaboration avec les ingénieurs sur l'analyse technique et les rapports de conformité, responsable matériels, étalonnage et métrologie, tutorat et accompagnement d'équipes sur outils et process.

Ces deux décennies m'ont appris :

- **Analyser des systèmes complexes** : un ascenseur, une ligne de production, un équipement de levage — comprendre comment ça marche pour diagnostiquer et corriger.
- **Travailler en autonomie sur le terrain**, avec une obligation de résultat et de sécurité.
- **Formaliser des processus** : rédiger des comptes-rendus de contrôle, des procédures de vérification, des analyses de risque.
- **Collaborer avec des ingénieurs et des opérateurs** de tous horizons, traduire les besoins métiers en spécifications techniques.
- **Travailler la rigueur** : dans le contrôle d'équipements industriels, une erreur peut coûter cher — humainement et financièrement.

1.4.2 Le déclic

Le déclic n'a pas été soudain. Il s'est construit progressivement, sur plusieurs années :

- L'envie d'**outils numériques** pour automatiser les comptes-rendus, structurer les données de contrôle, faciliter le suivi des clients.
- La curiosité grandissante pour le **code** : j'ai commencé à scripter en VBA, puis Python, pour gagner du temps sur les tâches répétitives du quotidien.
- Le constat que mon **métier de technicien** atteignait un plafond de progression et que mon envie d'apprendre dépassait ce que mon poste permettait.

En 2024, j'ai pris la décision : **je passe à 100 % côté concepteur d'outils plutôt que d'utilisateur d'outils**. Pas par rejet du terrain, mais parce que je veux créer ce que j'ai utilisé pendant 20 ans.

1.4.3 Zone01 Rouen Normandie — depuis 2024

J'ai rejoint **Zone01 Rouen Normandie** en 2024 pour suivre la formation de Concepteur Développeur d'Applications. La pédagogie m'a séduit :

- **Pas de cours magistraux** : on apprend par projet, en autonomie — la pédagogie repose sur l'apprentissage pair-à-pair (peer-to-peer) entre apprenants : chacun avance à son rythme, s'entraide, et s'évalue mutuellement via des audits de code.
- **Stack moderne** : Go, JavaScript/TypeScript, React, Next.js, PostgreSQL, Docker — les technos qui se trouvent réellement sur le marché en 2026.
- **Travail en équipe** : tous les projets significatifs sont en groupe, ce qui rejoue les conditions d'une vraie équipe dev.
- **Échec encouragé** : on a le droit de se planter, c'est même attendu — l'important est de comprendre pourquoi.

Cette approche colle parfaitement à mon profil : je sais apprendre seul (20 ans à diagnostiquer des pannes m'ont entraîné), je travaille bien en équipe (j'ai toujours géré des binômes ou des juniors), et j'aime quand on me laisse trouver mon chemin plutôt qu'on me déroule un cours.

1.5 Projets réalisés

Projet	Technologies	Description
Social Network	Next.js, React/TS, Prisma, PostgreSQL, Redis, Docker	Réseau social complet : authentification JWT, posts/stories/réactions, chat direct et groupe en temps réel, groupes, événements, notifications. Projet principal de certification.
PartiQ	Next.js, React/TS, Prisma, PostgreSQL, Docker	Plateforme B2B : catalogue produits, ticketing avec workflows de statut, messagerie temps réel, tableaux de bord, base de connaissances alimentée par IA.
AbrisCraft	Next.js, Three.js/R3F, React/TS, Prisma, PostgreSQL	Configurateur 3D interactif d'abris de jardin : sélection matériaux, estimation dynamique des coûts, chatbot intégré pour support contextuel.
Pochet du Courval	React/TS, Three.js/R3F, Framer Motion, Vite, Vitest	Site vitrine luxe pour un artisan verrier : bouteille 3D générée procéduralement, animations Framer Motion, détection WebGL avec fallback.
Groupe-Tracker	Go, HTML/CSS/JS, Gorilla Mux, API REST	Application web affichant des données d'artistes musicaux depuis une API REST : carte interactive des concerts, recherche et filtres par artiste, lieu ou année.
BombermanDX	JavaScript, HTML5, CSS3	Jeu navigateur inspiré de Bomberman : IA ennemie, power-ups, niveaux multiples, détection de collision et gestion DOM optimisées.

1.6 Ce que la formation m'a apporté (au-delà des technos)

1.6.1 Compétences techniques

- **Langages** : TypeScript, JavaScript, Java, Go
- **Frameworks** : Next.js 15, React 19, Node.js, Angular
- **BDD** : PostgreSQL, SQL, Prisma 6
- **DevOps** : Git, CI/CD, Docker, pnpm
- **API & intégration** : REST, GraphQL, chatbots IA, Server-Sent Events
- **Sécurité applicative** : JWT, bcrypt, OWASP Top 10, RGPD

1.6.2 Compétences transverses gagnées

- **Estimation et priorisation** : découper un gros projet (Social Network) en user stories puis en sprints
- **Documentation pour pairs** : passer d'un compte-rendu industriel à une doc technique pour développeurs
- **Code review** : lire le code des coéquipiers, formuler du feedback constructif
- **Veille technique** : sélectionner les sources qui valent le temps qu'on y passe (voir [04-developpement/veille-technique.md](#))

1.6.3 Mon évolution avant / après le projet Social Network

Domaine	Avant le projet	Après le projet
Architecture	Une appli = un dossier de fichiers	Architecture en couches, séparation domaine / infrastructure, choix de plateforme
BDD	Une table = une feuille Excel	Modélisation Merise, normalisation, contraintes d'intégrité, indexation
Auth	Un mot de passe en clair, ça suffit	JWT signé, bcrypt cost 12, cookies httpOnly+secure+sameSite, OAuth state cookie
Tests	Tests = on clique et on regarde	Pyramide de tests, tests d'intégration API, mock Prisma, jeux d'essai documentés

Domaine	Avant le projet	Après le projet
Sécurité	« C'est l'hébergeur qui s'en occupe »	OWASP Top 10, audit du code, identification des XSS résiduelles, roadmap de durcissement
Déploiement	Je l'envoie sur un serveur, ça doit marcher	Docker multi-stage, CI/CD, healthcheck, rollback, observabilité

1.7 Vision du développeur que je veux être

1.7.1 Mes forces

- **Rigueur** : 20 ans de contrôle d'équipements m'ont formé à vérifier deux fois avant de valider une fois.
- **Pragmatisme** : je préfère la solution qui marche et qui se maintient à la solution élégante mais fragile.
- **Capacité d'apprentissage** : je suis autodidacte depuis toujours, le numérique n'y change rien.
- **Communication métier ↔ technique** : j'ai été des deux côtés du pont, je sais traduire.
- **Endurance** : un projet long ne me fait pas peur. J'ai géré des chantiers de 6 mois sur le terrain.

1.7.2 Mes faiblesses (et comment je les travaille)

- **L'algorithmique pure** n'est pas mon point fort initial. Je compense par la pratique (LeetCode, exercices Zone01) et par la connaissance du contexte (un algo s'optimise dans une utilisation réelle, pas dans le vide).
- **Le réflexe « beau code »** : je dois encore me retenir de sur-architecturer. Je m'oblige à livrer en MVP puis itérer.
- **Pas de bagage informatique académique** : pas de cursus universitaire en CS. Je le compense par la curiosité (RFC, specs WHATWG, OWASP) et par les retours d'expérience pratiques.

1.7.3 Ce que je ne veux pas faire

- Rester cantonné à une seule couche technique sans compréhension globale du produit.
- Développer sans tests ni validation.
- Construire des solutions difficiles à maintenir, à sécuriser, ou à passer à un collègue.
- Subir le code legacy sans le refactorer progressivement.

1.7.4 Ce que je veux faire

- **Du full-stack avec une appétence backend** : conception d'API, modélisation BDD, sécurité.
 - Travailler dans une **équipe où la qualité compte** : revue de code, tests, documentation.
 - Contribuer à un produit qui a un **vrai usage métier** (B2B SaaS, outils internes, e-commerce, applications professionnelles).
 - Progresser sur la **dimension architecture** : DDD, hexagonal, séparation domaine / infrastructure.
-

1.8 Recherche d'alternance

La formation CDA (Bac+3) à Zone01 Rouen Normandie est conçue sur 3 ans : 1 an d'école puis 2 ans en alternance. N'ayant pas trouvé de contrat d'apprentissage, je me suis replié sur le cursus 2 ans à temps plein (juin 2024 – juin 2026). À l'issue de cette certification, je cherche un contrat d'apprentissage pour poursuivre en **Bac+5 Chef de Projet, Architecture et Green IT** à Zone01 (juin 2026 – juin 2028).

1.9 Parcours professionnel et formation

1.9.1 Parcours

- **2024 → 2026** : Zone01 Rouen Normandie — Bac+3 Concepteur Développeur d'Applications (RNCP 37873) — formation 2 ans à temps plein
- **2010 → 2024** : SOCOTEC Équipements — Expert en vérifications générales périodiques et mise en service (ascenseurs, levage, machines, EPI) · analyse technique · rapports de conformité · étalonnage et métrologie · responsable matériels · tutorat et accompagnement d'équipes sur outils et process
- **2004 → 2010** : KONE Ascenseurs et Portes Automatiques — Technicien hautement qualifié, régleur et mise en service
- **2003 → 2004** : Missions d'intérim — Manutentionnaire, technicien, régisseur technique
- **2002 → 2003** : Lubrizol — Technicien en bureau d'études (alternance)
- **2000 → 2001** : GPH — Technicien de maintenance (alternance)

1.9.2 Formation

- **Bac+3 CDA RNCP 37873** : Zone01 Rouen Normandie, juin 2024 – juin 2026 (2 ans, temps plein)
- **BTS MAI** (Mécanique et Automatismes Industriels) : 2001 – 2003
- **BAC PRO MSMA** (Maintenance des Systèmes Mécaniques Automatisés) : 2001

1.10 Compétences Techniques

- TypeScript, JavaScript, Java, Go
- Next.js, React, Node.js, Angular
- PostgreSQL, SQL
- Git, CI/CD, Docker, pnpm
- REST, GraphQL, chatbots IA

1.11 Qualités Professionnelles

- Ouvert
- Curieux
- Pragmatique
- Sociable
- Proactif
- Autodidacte
- Créatif
- Rigoureux

1.12 Centres d'Intérêt

- Musique (production et pratique de divers instruments)
- Jeux vidéo
- Sport (Handball / Trail)
- Sociologie et culture
- Astronomie

1.13 Langues

- Français
- Anglais

2 01 - Introduction

2.1 Contexte et Objectif

2.1.1 Problématique

Le projet Social Network a pour objectif de concevoir une application web sociale moderne permettant à des utilisateurs de publier du contenu, interagir avec leur réseau, échanger en temps réel et organiser leurs échanges autour de groupes, d'événements et de notifications.

Dans le cadre de la formation Zone01 Rouen Normandie, ce projet sert de support principal pour démontrer la maîtrise d'un développement full stack structuré, sécurisé et maintenable, en environnement collaboratif.

2.1.2 Solution Proposée

Plateforme de réseau social permettant:

- publication de posts, stories et médias,
 - réactions et commentaires,
 - messagerie privée et notifications en temps réel,
 - gestion de profils, d'amis et de groupes,
 - recherche et découverte de contenu.
-

2.2 Informations Projet

Titre: Social Network

Type: Application web full-stack **Repository:** [arocchet/social-network](https://github.com/arocchet/social-network) **Stack Technologique:**

- Frontend: Next.js 15, React 19, TypeScript, Tailwind CSS
 - Backend: Next.js API Routes, TypeScript
 - Database: PostgreSQL (Neon), Prisma ORM
 - Real-time: Upstash Redis, SSE/Polling
 - Media: Cloudinary
 - Auth: JWT (jose), bcrypt
 - Deployment: Docker, Docker Compose
-

2.3 Équipe et Contexte

Le projet Social Network est un projet de groupe Zone01. La particularité de notre équipe : j'étais le seul membre **sans alternance**, les autres apprenants étant en entreprise à des rythmes différents (fréquences

et jours variables selon les contrats). Travailler ensemble en synchrone était donc structurellement impossible.

Cette contrainte nous a conduits à **utiliser GitHub de manière professionnelle et systématique** comme seul outil de coordination :

- **Issues** pour tout tracker : chaque fonctionnalité, bug ou tâche DevOps fait l'objet d'une issue dédiée
- **Pull Requests** pour toute contribution : aucun push direct sur `main`, tout passe par une PR avec revue de code
- **Branch protection rules** : la branche `main` est protégée — merge impossible sans que tous les checks soient au vert
- **CI/CD (GitHub Actions)** : lint ESLint, tests Jest, build Next.js déclenchés automatiquement à chaque PR
- **SonarQube** : analyse statique du code intégrée au pipeline — détection de code smells, vulnérabilités et duplications avant tout merge
- **Vercel preview deployments** : chaque PR génère automatiquement un environnement de prévisualisation déployé, permettant de tester le rendu réel avant merge en production
- **Branches feature** : chacun travaille sur sa branche, merge après validation de l'ensemble des checks
- **Labels et milestones** pour prioriser et organiser les lots de travail

Chacun avançait dans son coin selon ses disponibilités, mais le code convergait proprement grâce à ces pratiques. Ce contexte m'a forcé à acquérir très tôt les réflexes d'un développeur travaillant en équipe distribuée.

2.3.1 Contexte du Code Initial

Le projet a démarré avec une base fonctionnelle incomplète. Ma contribution a été d' :

- Stabiliser l'architecture (middleware, auth, API)
- Implémenter des fonctionnalités manquantes (notifications, temps réel, déploiement)
- Sécuriser l'authentification (JWT avec jose, hachage bcrypt)
- Documenter l'ensemble du système pour la certification

2.4 Sommaire du Dossier

1. **Introduction** - Contexte et objectifs
2. **Cahier des Charges** - Spécifications fonctionnelles
3. **Conception** - Wireframes, maquettes, modélisation
4. **Développement** - Architecture et implémentation
5. **Déploiement** - Infrastructure et mise en production
6. **Bilan** - Retour d'expérience

7. Annexes - Code source et documentation technique

2.5 Critères de Réussite

- ☒ Authentification sécurisée (JWT + Google OAuth, cookies HTTP-only)
- ☒ Gestion des utilisateurs (profil, avatar, bannière, visibilité)
- ☒ Création et partage de contenu (posts, stories, reels, images via Cloudinary)
- ☒ Système de réactions et de commentaires (7 types de réactions, commentaires imbriqués)
- ☒ Système de notifications en temps réel (SSE + Upstash Redis)
- ☒ Responsive design (mobile/tablet/desktop, Tailwind CSS)
- ☒ Performance optimisée (React Query, SWR, Next.js Image, code splitting)
- ☒ Tests automatisés (Jest, tests d'intégration authentification)
- ☒ Documentation complète (dossier 60+ pages, diagrammes, API spec)
- ☒ Déploiement fonctionnel (Docker multi-stage, docker-compose, Vercel)

3 02 - Cahier des Charges

3.1 Objectifs Principaux

Le projet Social Network est réalisé en équipe dans le cadre de la formation Zone01 Rouen Normandie. Le cahier des charges ci-dessous synthétise les fonctionnalités réellement suivies dans le dépôt GitHub du projet, sous forme d'issues et de PR, puis les contraintes de mise en oeuvre qui structurent le développement.

3.1.1 Organisation GitHub du projet

Le travail a été découpé en tickets GitHub pour suivre le développement en équipe. Les sujets visibles dans le dépôt couvrent notamment:

- [Follow System](#)
- [Groups & Events](#)
- [Group feed — Posts & comments inside group](#)
- [Chat System](#)
- [Notifications](#)
- [DevOps: Docker, CI/CD](#)
- [CI — Lint, test, build, push image](#)
- [Seed script — Demo data](#)
- [PATCH /notifications/:id/read — Mark as read](#)
- [OAuth \(Google\) authentication](#)
- [Internationalization](#)
- [Settings](#)

La stabilisation du build et du déploiement a ensuite été consolidée par la PR: [PR #118](#)

3.1.2 Fonctionnalités Essentielles

3.1.2.1 Gestion des Utilisateurs

- Inscription et authentification (email/password + Google OAuth)
- Gestion de profil (avatar, bannière, bio, visibilité PUBLIC/PRIVATE)
- Système de suivi (followers/following + demandes d'amitié)

3.1.2.2 Publication et Interaction

- Création de posts/articles
- Commentaires et réponses
- Système de likes/réactions sur posts, stories et commentaires
- Partage de contenu
- Gestion de la visibilité des publications

3.1.2.3 Communication

- Système de messagerie privée
- Notifications en temps réel
- Groupes/Communautés

3.1.2.4 Découverte

- Recherche et filtrage (users, posts, groupes)
 - Historique de recherche
 - Exploration du contenu public
-

3.2 Contraintes Techniques

3.2.1 Sécurité

- Authentification JWT/OAuth
- Validation des données
- Chiffrement des mots de passe (bcrypt)
- Contraintes de réactions: un utilisateur ne peut pas multiplier une même réaction sur un même contenu

3.2.2 Scalabilité

- Cache Redis
- Base de données PostgreSQL
- CDN pour assets

3.2.3 Compatibilité

- Support navigateurs modernes (Chrome, Firefox, Safari, Edge)
- Responsive design (mobile/tablet/desktop)

3.2.4 Performance

- Optimisation images (Next.js Image)
 - Code splitting automatique
 - Lazy loading
 - Chargement fluide du feed, des profils et des conversations
-

3.3 User Stories

Utilisateur	Action	Priorité	Critères d'acceptation
Visiteur	S'inscrire		Email validé, compte créé
Utilisateur	Se connecter		Session JWT créée
Utilisateur	Créer un post		Post visible immédiatement
Utilisateur	Commenter		Commentaire visible
Utilisateur	Liker/Réagir		Compteur mis à jour
Utilisateur	Suivre un user		Feed personnalisé

3.4 Objectifs de Qualité

Indicateur	Cible	Approche retenue
Disponibilité	> 99 %	Docker + Vercel + Neon serverless
Temps de chargement	< 2 s	SSR Next.js, React Query, lazy loading
TTFB	< 600 ms	Edge CDN Vercel, cache Redis
Stabilité visuelle (CLS)	< 0.1	Next.js Image, layout réservé
Couverture de tests	> 80 %	Jest, tests d'intégration API

3.5 Timeline

Phase	Durée	Statut
Conception	25 jours	Planifié (MVP scope)
Développement	14 jours	Phase 2 (features)
Tests	7 jours	Phase 3 (polish & intégration)
Déploiement	3 jours	Préparation & déploiement initial

Notes:

- Phase 1 (MVP - 25 jours): implémentation des fonctionnalités critiques (auth, posts, feed, basic chat, notifications).

- Phase 2 (14 jours): fonctionnalités avancées (groups, events, reels, amélioration UX, i18n).
- Phase 3 (7 jours): tests d'intégration, corrections, optimisation performances et préparation de la soutenance.

4 03 - Conception

4.1 Résumé

Cette section documente la conception complète du réseau social `social-network`:

- **Pages & Wireframes:** 12+ pages principales documentées
 - **Modélisation Données:** 18 modèles Prisma avec diagrammes MCD/MLD
 - **User Stories:** 43 user stories avec priorités et estimations
 - **Design System:** Palette couleurs, typographie, composants
-

4.2 Documents du Projet

4.2.1 Pilotage du travail

La conception s'appuie sur les issues GitHub du projet Social Network. Les lots fonctionnels ont été organisés autour des sujets suivis dans le dépôt: authentication, follow, chat, notifications, groupes, événements, settings, i18n et DevOps.

4.2.2 1. Pages et Wireframes

- Accueil (Home/Feed)
- Authentification (Login/Register/Onboarding)
- Profil utilisateur
- Messages/Chat (Direct & Groupe)
- Reels/Vidéos
- Recherche
- Événements
- Groupes/Communautés
- Paramètres
- Invitations
- Design System (couleurs, typo, composants)
- Architecture page et data flow

4.2.3 2. Dictionnaire de Données

- **18 Modèles documentés:**
 - User, Post, Comment, Reaction, Story
 - Message, Friendship, Conversation
 - GroupMessage, GroupInvitation, GroupJoinRequest, GroupMember
 - Event, Rsvp, UserSettings, Notification, Account

- **Attributs complets:** Types, constraints, descriptions
- **Relations:** Cardinalités et foreign keys
- **Enums:** Réaction types, statuts invitation, etc.
- **MCD/MLD Diagrammes:** Modèles conceptuels et logiques

4.2.4 3. User Stories

- **38 User Stories** organisées par rôle:
 - Utilisateur classique (38 stories)
- **Priorités & Estimations:** HIGH/MEDIUM/LOW
- **Phases de développement:**
 - Phase 1 MVP (25 jours)
 - Phase 2 Features (2 semaines)
 - Phase 3 Polish (1 semaine)
- **Acceptance Criteria:** Front/Back/DevOps

4.2.5 4. Diagrammes UML Complets

- MCD en DBML (import dbdiagram.io)
 - MLD détaillé en DBML
 - MPD PostgreSQL complet (DDL, contraintes, index, triggers)
 - Support de présentation technique pour la soutenance
-

4.3 Fonctionnalités Clés

4.3.1 Réseautage Social

- Posts (texte + images + vidéos)
- Followers/Following + Amitié
- Likes/Comments/Reactions
- Stories (24h ephemeral)
- Reels (vertical short videos)

4.3.2 Communication

- Direct Messages (1-to-1)
- Group Chat
- Typing Indicator + Presence
- Message Status (SENT/DELIVERED/READ)

4.3.3 Groupes & Événements

- Create/Join Groups
- Group Invitations
- Events Calendar
- RSVP Management

4.3.4 Discovery

- User Search
- Post Search
- Search History
- Recent Activity

4.3.5 Personalization

- Theme Toggle (Light/Dark)
 - Language (EN/FR)
 - Notification Settings
 - Privacy Controls
-

4.4 Architecture de Données

4.4.1 Modèles Principaux

User ↔ Post ↔ Comment ↔ Reaction

- Friendship (2-way)
- Conversation
 - ConversationMember
 - GroupMessage
 - Event
 - Rsvp
 - GroupMessage
 - GroupMember
 - GroupInvitation
 - GroupJoinRequest
- Story ↔ Reaction
- Message
- Notification

- UserSettings
- Account (OAuth)

4.4.2 Statistiques

- 18 modèles Prisma
- 40+ relations
- 35+ attributs User
- 7 enums (Visibilité, Réaction types, statuts)

4.5 Pages Implémentées

Page	Route	Rôle	Statut
Feed	/	Affichage posts	
Login	/login	Authentification	
Register	/register	Inscription	
Onboarding	/onboarding	Complétude profil	
Profil	/profile/[userId]	Voir profil utilisateur	
Chat (List)	/chat	Liste conversations	
Chat (Direct)	/chat/[id]	Message 1-to-1	
Reels	/reels	Short videos	
Search	/search	Rechercher contenu	
Groupes	/groups	Lister groupes	
Groupe Detail	/groups/[id]	Détail groupe + chat	
Événements	/events	Calendar événements	
Paramètres	/settings	Préférences user	
Invitations	/invitations	Demandes d'amitié/groupe	

4.6 Design System

4.6.1 Palette Couleurs (Thème Dark)

Primary Colors:

```
--blue40: #ff6d00 (Primary action) --blue60: #ff7900 (Hover/Active);
```

Backgrounds:

```
--bgLevel1: #1a191c (Darkest) --bgLevel2: #222024 --bgLevel3: #272529  
--bgLevel4: #333036 --bgLevel5: #534e57 (Lightest);
```

Text:

```
--textNeutral: #ffffff (Primary text) --textNeutralAlt: #e8e1f0 (Secondary)  
--textMinimal: #bdbdbd (Tertiary);
```

Accents:

```
--red50: #a71e34 (Danger) --teal50: #56ab91 (Success) --purple60: #8b2fc9  
(Accent) --green50: #25a244 (Complete);
```

4.6.2 Typography

- Font Family: Geist (via Next.js)
- H1: 28-32px
- H2: 20-24px
- Body: 14-16px
- Small: 12px

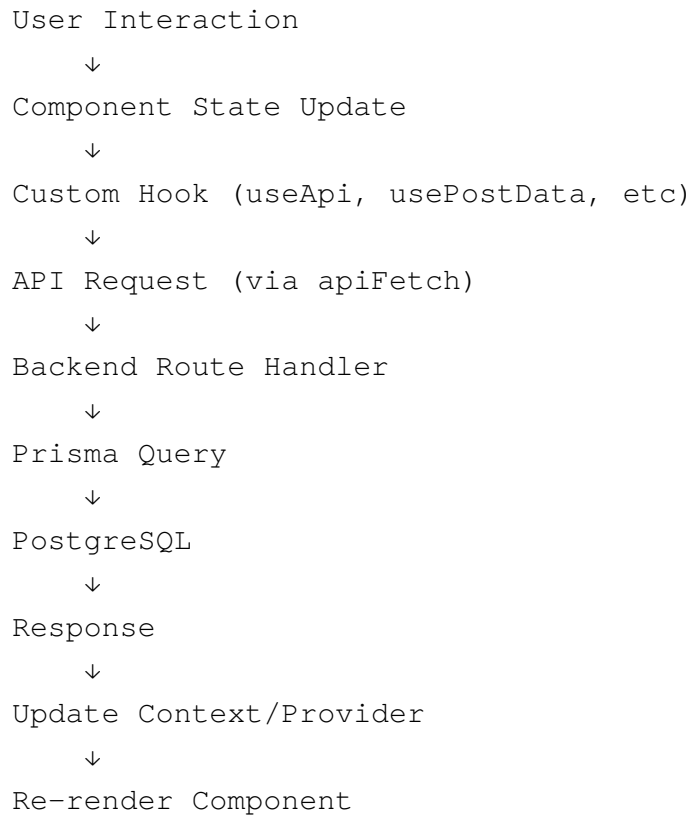
4.6.3 Breakpoints

- Mobile: < 640px
- Tablet: 640px - 1024px
- Desktop: > 1024px

4.6.4 Components

- Button, Avatar, Card, Badge, Dialog
- Input, Tabs, Switch, Notification Badge

4.7 Data Flow



4.8 Diagrammes

4.8.1 MCD - Modèle Conceptuel de Données

Voir [donnees.md - MCD Section](#)

Représente toutes les entités et leurs relations logiques.

4.8.2 MLD - Modèle Logique de Données

Voir [donnees.md - MLD Section](#)

Montre le schéma de base de données normalisé.

4.9 Wireframes & Maquettes

4.9.1 Wireframes basse-fidélité

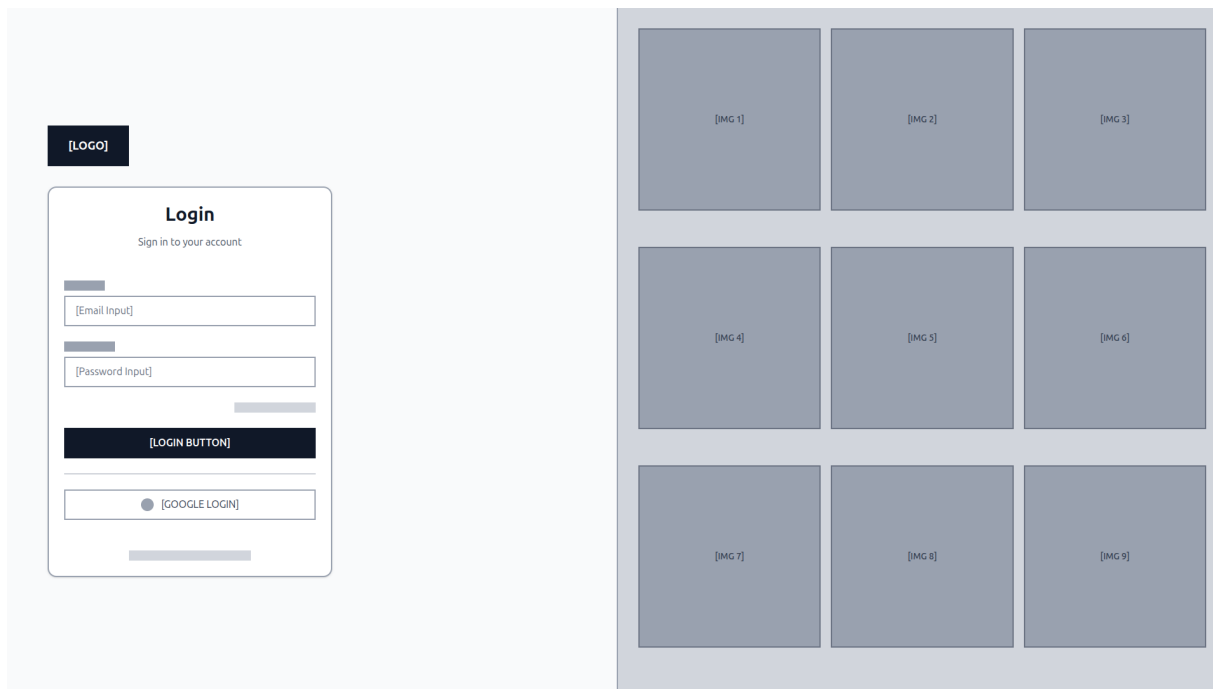


Figure 1: Connexion — wireframe

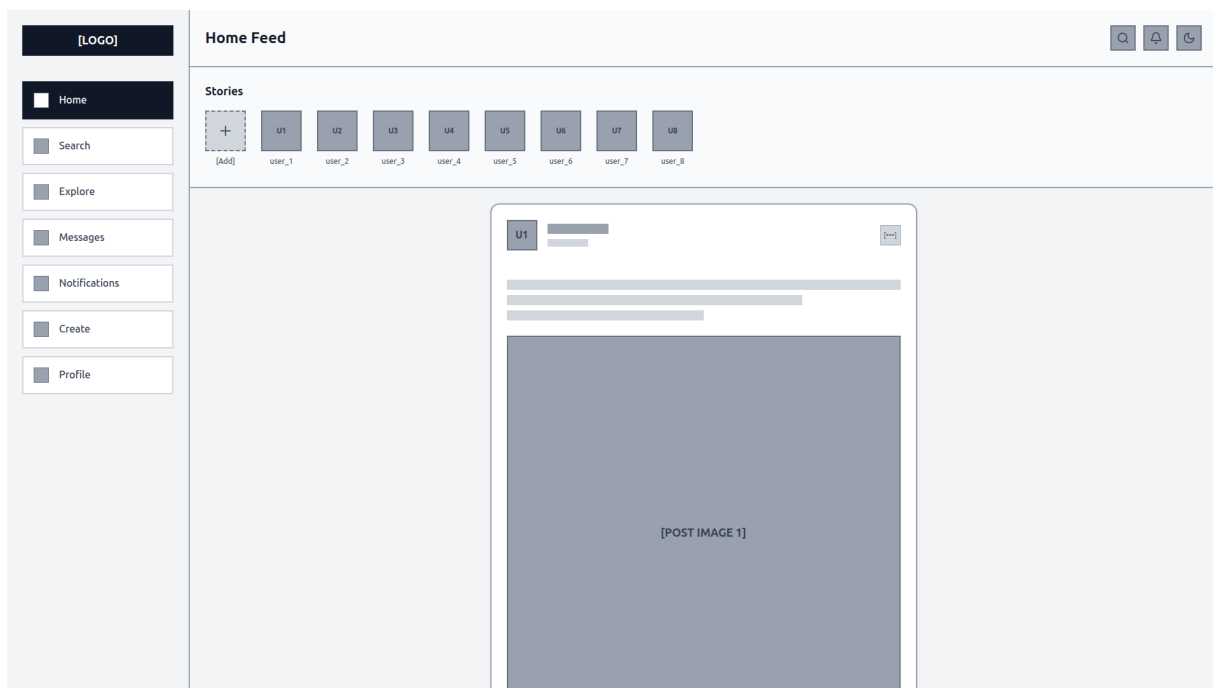


Figure 2: Feed principal — wireframe

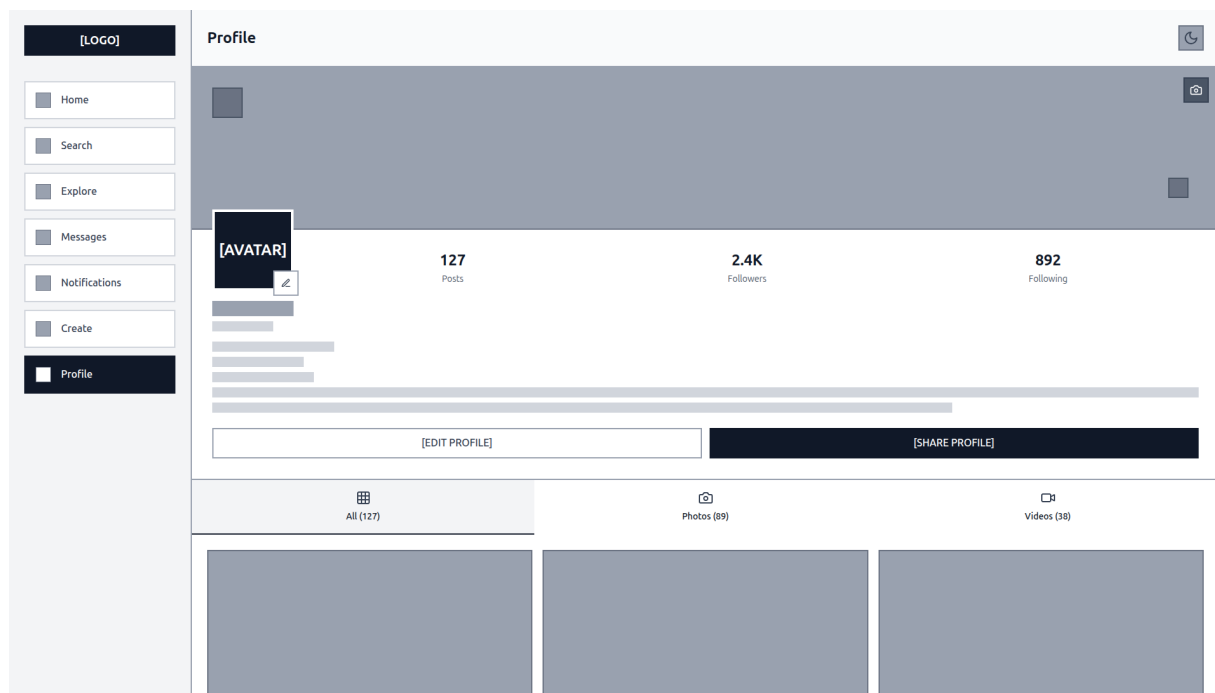


Figure 3: Profil utilisateur — wireframe

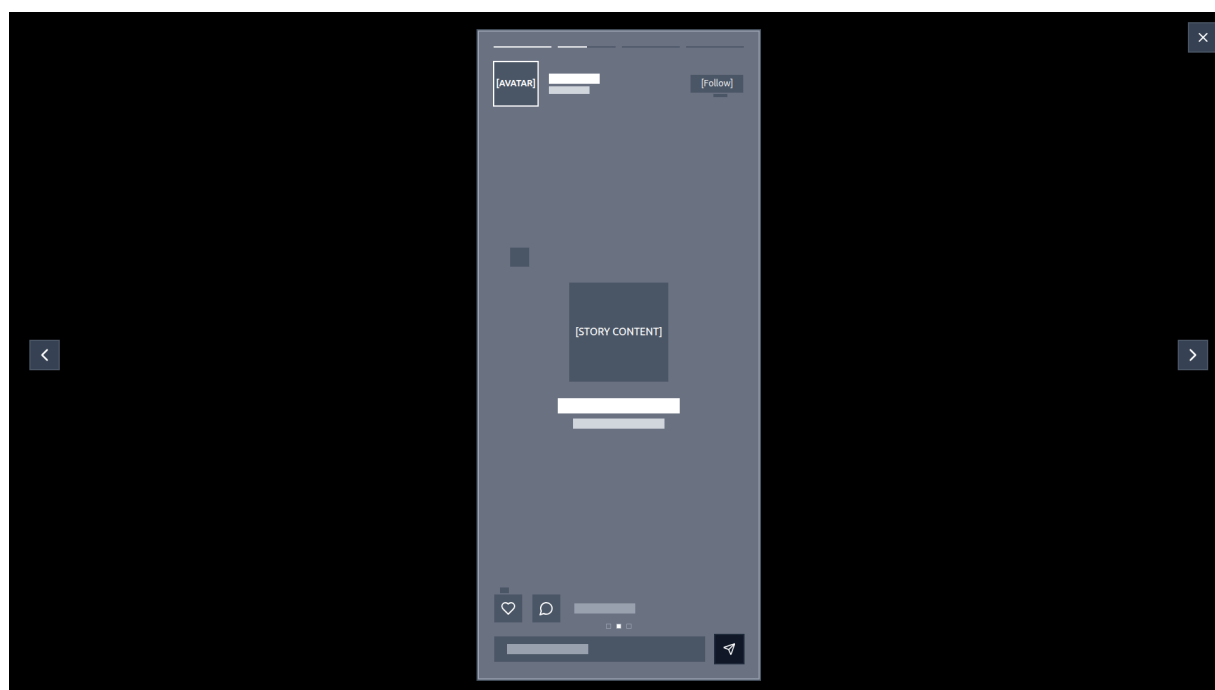


Figure 4: Visionneuse de stories — wireframe

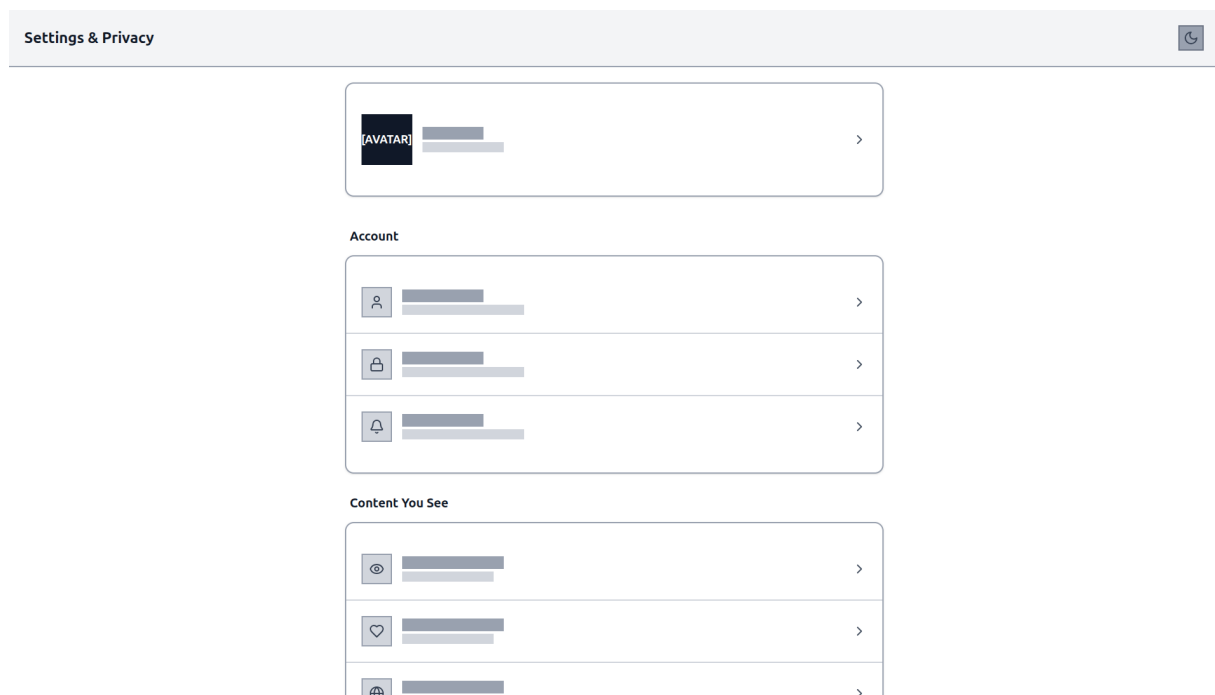


Figure 5: Paramètres — wireframe

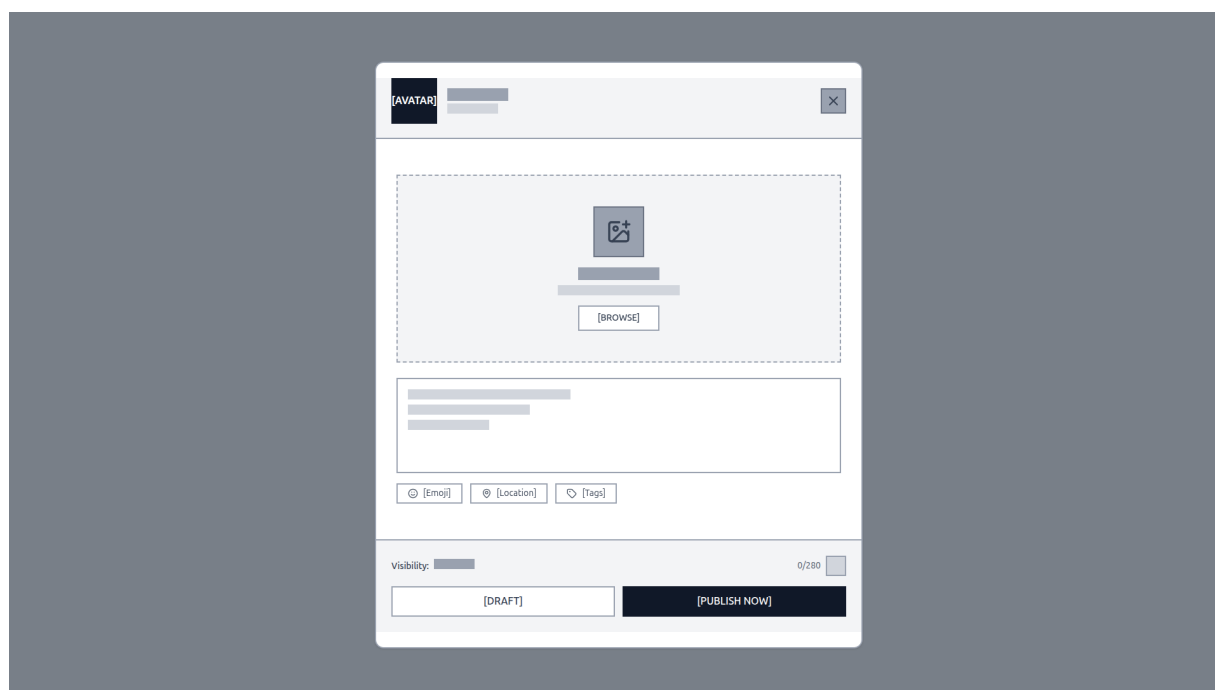
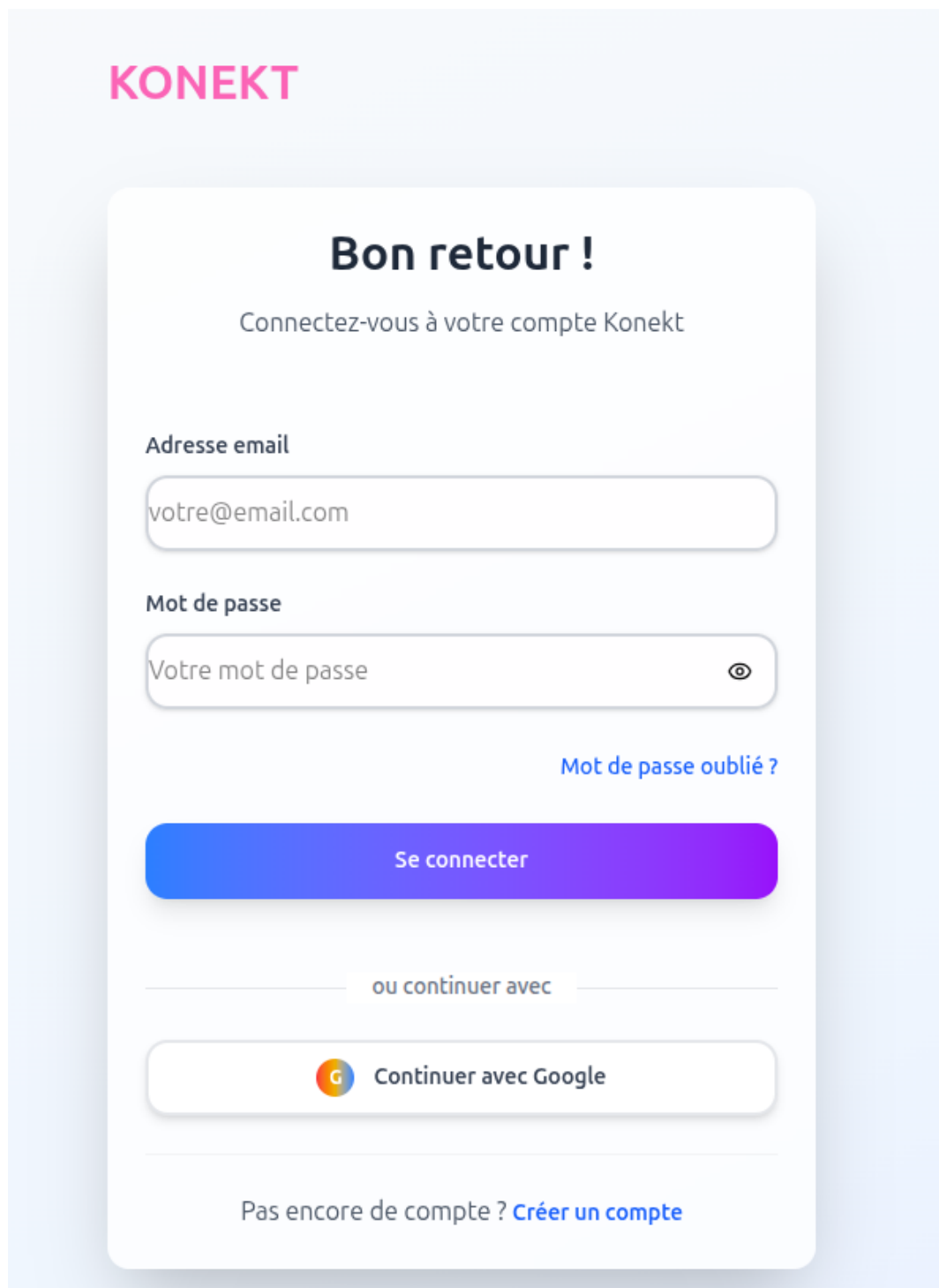


Figure 6: Création de post — wireframe

4.9.2 Maquettes haute-fidélité



KONEKT

Bon retour !

Connectez-vous à votre compte Konekt

Adresse email

Mot de passe

[Mot de passe oublié ?](#)

Se connecter

ou continuer avec

 Continuer avec Google

Pas encore de compte ? [Créer un compte](#)

Figure 7: Connexion — maquette

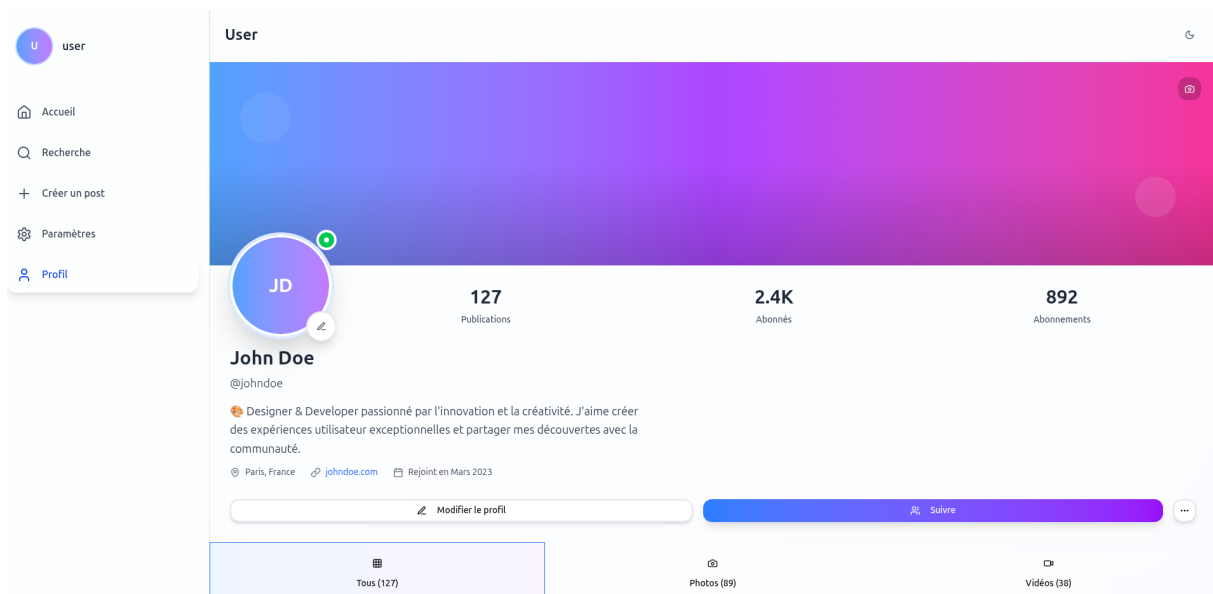


Figure 8: Feed principal — maquette

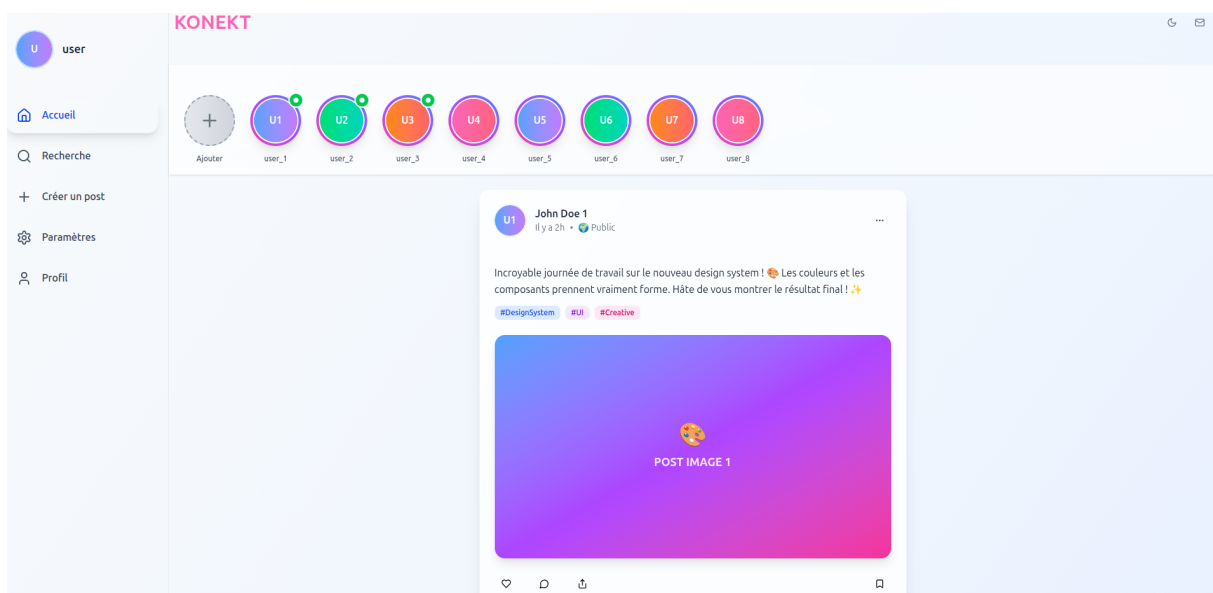


Figure 9: Profil utilisateur — maquette

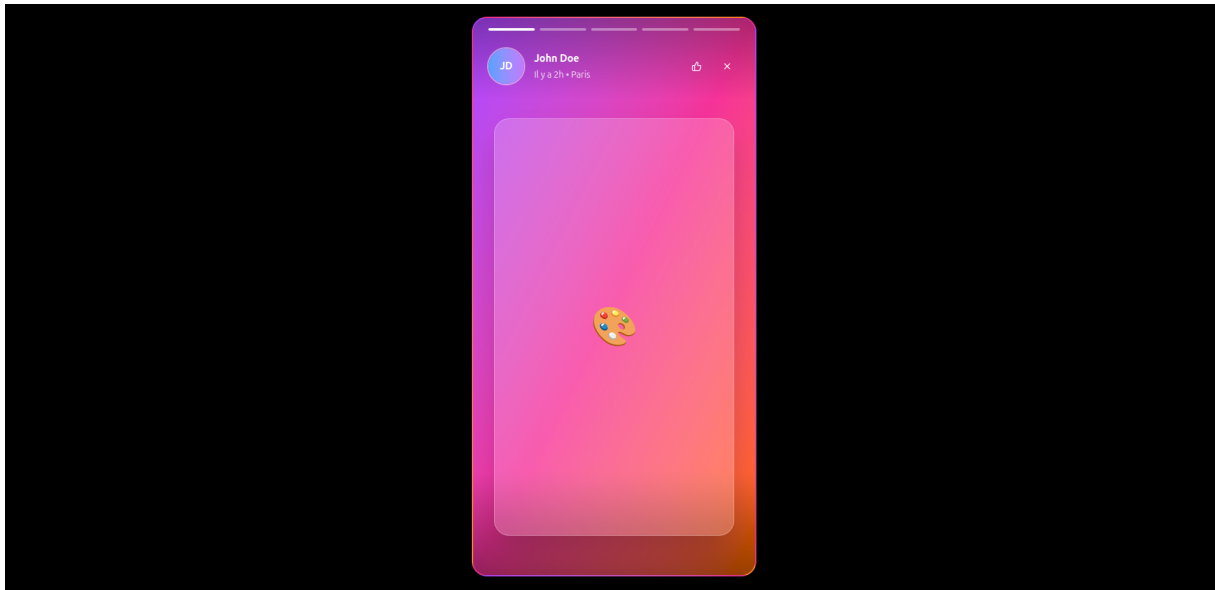


Figure 10: Visionneuse de stories — maquette

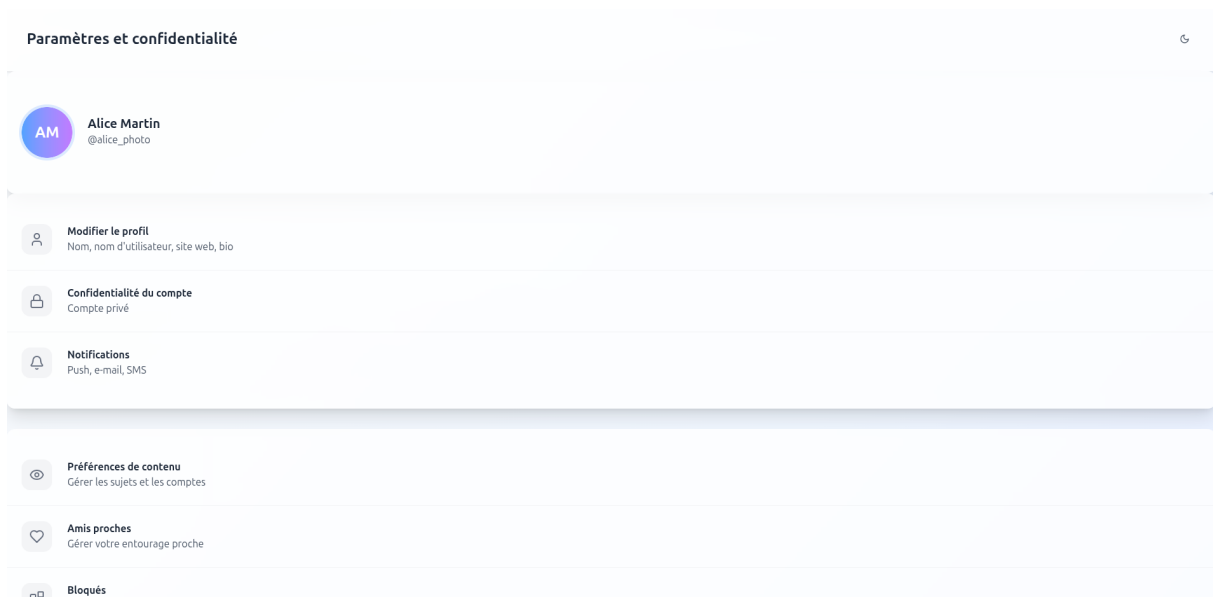


Figure 11: Paramètres — maquette

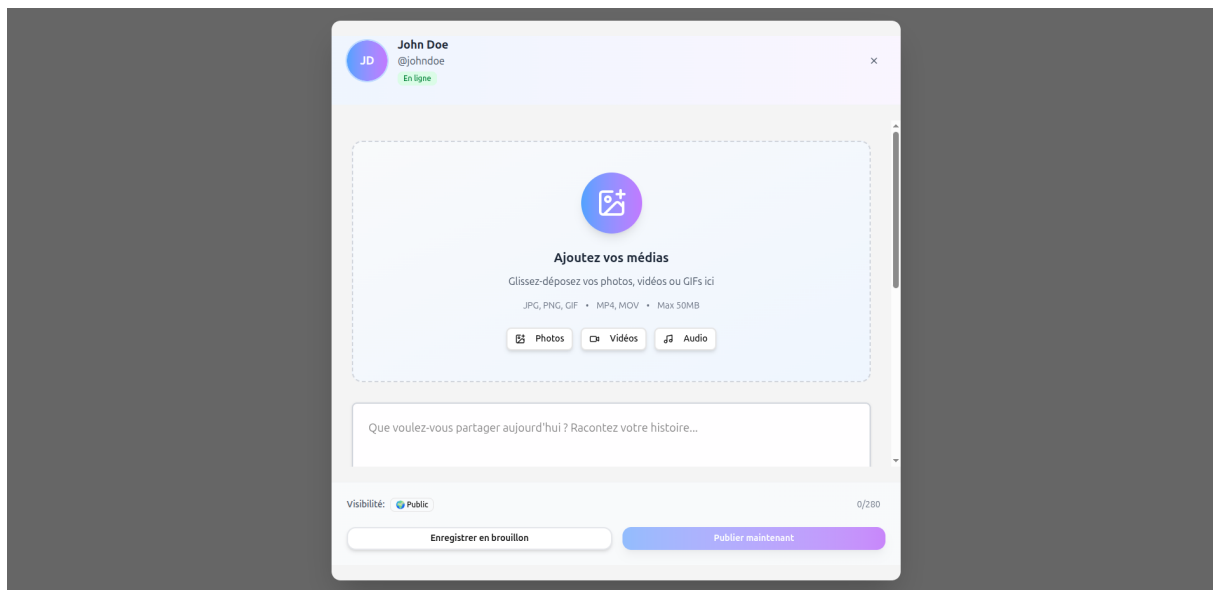


Figure 12: Création de post — maquette

4.10 Checklist Conception

- ☑ Pages principales documentées (14 routes)
- ☑ Dictionnaire de données complet (18 modèles Prisma)
- ☑ Diagrammes MCD/MLD/MPD
- ☑ User stories (38+ stories avec critères d'acceptation)
- ☑ Design system défini (palette, typographie, breakpoints, composants)
- ☑ Architecture data flow documentée
- ☑ Wireframes basse-fidélité (07-annexes)
- ☑ Maquettes haute-fidélité (07-annexes)
- ☑ User flows décrits par fonctionnalité

4.11 Fichiers Liés

- **Introduction:** [01-introduction/README.md](#)
- **Cahier des charges:** [02-cahier-des-charges/README.md](#)
- **Développement:** [04-developpement/README.md](#)
- **Code source:** `/src`
- **Prisma Schema:** `/prisma/schema.prisma`

Last Updated: 2026-05-19

Version: 2.0

5 User Stories & Cas d'Usage

5.1 Vue d'Ensemble

Nous décrivons ci-dessous les user stories que nous avons rédigées et priorisées pour le projet. Le texte est rédigé à la première personne du pluriel pour refléter notre travail d'équipe et permettre d'indexer facilement chaque story sur les issues/PRs correspondantes du dépôt GitHub.

5.2 Preuves & Mapping GitHub

Nous avons lié les fonctionnalités principales aux tickets du dépôt `arocchet/social-network` :

- Follow / friendship
 - Groups & Events
 - Group feed (posts in groups)
 - Chat system
 - Notifications
 - DevOps / Docker / CI
 - CI pipeline (lint/test/build)
 - Seed script / demo data
 - Mark notification as read
 - OAuth (Google)
 - Internationalization (i18n)
 - Settings
 - PR stabilisation (Docker/Neon/Prisma/Redis)
-

5.3 Utilisateur Classique (Regular User)

5.3.1 Authentification & Profil

5.3.1.1 US001: Créer un compte

- As a nouvel utilisateur
- I want to créer un compte avec email/password
- So that je peux accéder au réseau social
- Acceptance Criteria:
 - ☐ Formulaire d'inscription avec email/password
 - ☐ Validation email (pas de doublons)
 - ☐ Password strength checker

- ☐ Confirmation email (optionnel)
- ☐ Auto-login après inscription
- ☐ Redirection vers onboarding

5.3.1.2 US002: Compléter le profil

- **As a** nouvel utilisateur
- **I want to** compléter mon profil avec avatar, bio, date de naissance
- **So that** ma présence est visible aux autres

5.3.1.3 US003: Se connecter

- **As a** utilisateur inscrit
- **I want to** me connecter avec email/username et password
- **So that** j'accède à mon compte
- **Acceptance Criteria:**
 - ☐ Login avec email OU username
 - ☐ JWT token en cookie HTTP-only
 - ☐ Session persistent
 - ☐ Remember me (7 jours)

5.3.1.4 US004: Consulter mon profil

- **As a** utilisateur connecté
- **I want to** voir mon profil public
- **So that** je vérifie mes informations visibles
- **Acceptance Criteria:**
 - ☐ Avatar, banner, bio
 - ☐ Statistiques (followers, following, posts)
 - ☐ Bouton edit profil

5.3.1.5 US005: Éditer mon profil

- **As a** utilisateur
- **I want to** modifier mes données (nom, bio, avatar, banner)
- **So that** mon profil est à jour
- **Acceptance Criteria:**
 - ☐ Modify firstname, lastname, username, bio
 - ☐ Upload avatar/banner
 - ☐ Prévisualisation
 - ☐ Confirmation de modification

5.3.1.6 US006: Gérer les paramètres

- **As a** utilisateur
 - **I want to** accéder aux paramètres (notifications, sécurité, langue)
 - **So that** je contrôle mon expérience
 - **Acceptance Criteria:**
 - ☐ Theme toggle (light/dark)
 - ☐ Langue (EN/FR)
 - ☐ Notifications (push, email)
 - ☐ Confidentialité (public/private)
 - ☐ Mots de passe à proximité
 - ☐ 2FA optionnel
-

5.3.2 Posts & Contenu

5.3.2.1 US007: Créer un post

- **As a** utilisateur connecté
- **I want to** publier un post avec texte ±images
- **So that** je partage mon contenu
- **Acceptance Criteria:**
 - ☐ Texte uniquement
 - ☐ Texte + image
 - ☐ Vidéo YouTube embed
 - ☐ Sélectionneur de visibilité (PUBLIC/PRIVATE/FRIENDS)
 - ☐ Draft auto-save
 - ☐ Vérification avant publication

5.3.2.2 US008: Voir le feed

- **As a** utilisateur connecté
- **I want to** voir les posts des utilisateurs que je follow
- **So that** je reste au courant
- **Acceptance Criteria:**
 - ☐ Infinite scroll pagination
 - ☐ Posts ordonnés par datetime DESC
 - ☐ Avatar + nom auteur
 - ☐ Affichage images/vidéos
 - ☐ Like count, comment count
 - ☐ Filtrer par friend only (optionnel)

5.3.2.3 US009: Liker un post

- **As a** utilisateur
- **I want to** liker un post en cliquant sur l'icône
- **So that** j'exprime mon appréciation
- **Acceptance Criteria:**
 - ☐ Toggle like/unlike
 - ☐ Update like count en temps réel
 - ☐ Icône change de couleur
 - ☐ Only 1 like par user/post

5.3.2.4 US010: Réagir à un post

- **As a** utilisateur
- **I want to** utiliser des reactions (Love, Wow, Laugh, Sad, Angry)
- **So that** j'exprime différentes émotions
- **Acceptance Criteria:**
 - ☐ Menu de 7 reactions
 - ☐ Only 1 reaction par user/post
 - ☐ Replace like avec reaction
 - ☐ Affichage des reactions populaires

5.3.2.5 US011: Commenter un post

- **As a** utilisateur
- **I want to** ajouter un commentaire sur un post
- **So that** je participe à la discussion
- **Acceptance Criteria:**
 - ☐ Texte + optionnel images
 - ☐ Timestamp
 - ☐ Author info (avatar, nom)
 - ☐ Like sur commentaires
 - ☐ Réponses à commentaires (thread)

5.3.2.6 US012: Supprimer mon post

- **As a** propriétaire d'un post
- **I want to** supprimer mon post
- **So that** je contrôle mon contenu
- **Acceptance Criteria:**
 - ☐ Only le propriétaire peut supprimer
 - ☐ Confirmation avant suppression
 - ☐ Supprime aussi commentaires/reactions
 - ☐ Update feed immédiat

5.3.2.7 US013: Partager un post

- **As a** utilisateur
 - **I want to** partager un post sur ma timeline
 - **So that** mes amis voient ce contenu
 - **Acceptance Criteria:**
 - ☐ Share button
 - ☐ Crée un nouveau post avec citation du post original
 - ☐ Garde lien vers original
-

5.3.3 Stories

5.3.3.1 US014: Créer une story

- **As a** utilisateur
- **I want to** partager une story (24h duration)
- **So that** je partage des moments éphémères
- **Acceptance Criteria:**
 - ☐ Photo/vidéo upload
 - ☐ Filtre texte optionnel
 - ☐ Visibilité (PUBLIC/FRIENDS)
 - ☐ Auto-delete après 24h
 - ☐ Privacy: only followers

5.3.3.2 US015: Voir les stories

- **As a** utilisateur
- **I want to** voir les stories des utilisateurs que je follow
- **So that** je suis actif dans le réseau
- **Acceptance Criteria:**
 - ☐ Carousel en haut du feed
 - ☐ Stories chronologiques (oldest → newest)
 - ☐ Affiche seen/unseen count
 - ☐ Avatar avec badge vert (unseen)

5.3.3.3 US016: Interagir avec une story

- **As a** utilisateur
- **I want to** liker ou commenter une story
- **So that** j'interagis avec le contenu
- **Acceptance Criteria:**

- ☐ Like button
 - ☐ React avec emojis
 - ☐ Voir likes/comments count
-

5.3.4 Amitié & Suivi

5.3.4.1 US017: Suivre un utilisateur

- **As a** utilisateur
- **I want to** suivre (follow) un autre utilisateur
- **So that** je vois ses posts dans mon feed
- **Acceptance Criteria:**
 - ☐ Follow button sur profil
 - ☐ Toggle follow/unfollow
 - ☐ Counter updated
 - ☐ Follow = voir tous posts PUBLIC/FRIENDS

5.3.4.2 US018: Demander en amitié

- **As a** utilisateur
- **I want to** envoyer une demande d'amitié
- **So that** on devienne amis
- **Acceptance Criteria:**
 - ☐ Add friend button
 - ☐ Status PENDING → ACCEPTED
 - ☐ Notification reçue
 - ☐ Only amis = voir PRIVATE posts

5.3.4.3 US019: Accepter demande d'amitié

- **As a** utilisateur
- **I want to** accepter une demande d'amitié
- **So that** je confirme la relation
- **Acceptance Criteria:**
 - ☐ Voir liste demandes
 - ☐ Accept/Decline buttons
 - ☐ Notification à l'autre user
 - ☐ Add to friends list

5.3.4.4 US020: Consulter mes amis

- **As a** utilisateur
 - **I want to** voir ma liste d'amis
 - **So that** je gère mes relations
 - **Acceptance Criteria:**
 - ☐ Affiche tous les amis
 - ☐ Avatar + nom
 - ☐ Last seen
 - ☐ Remove friend option
-

5.3.5 Messagerie

5.3.5.1 US021: Envoyer un message direct

- **As a** utilisateur
- **I want to** envoyer un message privé à un autre utilisateur
- **So that** on communique 1-to-1
- **Acceptance Criteria:**
 - ☐ Accès depuis profil/friends
 - ☐ Message ±image
 - ☐ Status: SENT → DELIVERED → READ
 - ☐ SSE / polling real-time
 - ☐ Typing indicator

5.3.5.2 US022: Voir liste des conversations

- **As a** utilisateur
- **I want to** voir toutes mes conversations
- **So that** je gère mes DMs
- **Acceptance Criteria:**
 - ☐ List toutes conversations
 - ☐ Last message preview
 - ☐ Unread badge
 - ☐ Search conversations
 - ☐ Sort par recent
 - ☐ SSE notifications

5.3.5.3 US023: Archiver une conversation

- **As a** utilisateur
- **I want to** archiver une conversation

- **So that** ma liste reste organisée
 - **Acceptance Criteria:**
 - ☐ Archive action
 - ☐ Hidden from main list
 - ☐ Restaurable depuis archive
-

5.3.6 Recherche

5.3.6.1 US024: Rechercher des utilisateurs

- **As a** utilisateur
- **I want to** chercher d'autres utilisateurs par nom/username
- **So that** je trouve des gens à follow
- **Acceptance Criteria:**
 - ☐ Search bar en haut
 - ☐ Debounced search
 - ☐ Results: avatar, nom, bio, follow button
 - ☐ Recent searches saved

5.3.6.2 US025: Rechercher des posts

- **As a** utilisateur
 - **I want to** chercher des posts par mots-clés
 - **So that** je trouve du contenu pertinent
 - **Acceptance Criteria:**
 - ☐ Full-text search
 - ☐ Results: author, date, like count
 - ☐ Filter par date, author
-

5.3.7 Notifications

5.3.7.1 US026: Recevoir notifications

- **As a** utilisateur
- **I want to** être notifié des actions (like, comment, follow, message)
- **So that** je reste engagé
- **Acceptance Criteria:**
 - ☐ Notifications temps réel via SSE/Redis
 - ☐ Badge sur icône notifs

5.3.7.2 US027: Consulter notifications

- **As a** utilisateur
 - **I want to** voir ma liste de notifications
 - **So that** je vérifie les actions
 - **Acceptance Criteria:**
 - ☐ List notifications
 - ☐ Mark as read/unread
 - ☐ Filter par type
 - ☐ Timestamp
-

5.3.8 Groupes (Communities)

5.3.8.1 US028: Créer un groupe

- **As a** utilisateur
- **I want to** créer un groupe privé/public
- **So that** je crée une communauté
- **Acceptance Criteria:**
 - ☐ Nom + description
 - ☐ Avatar groupe
 - ☐ Visibilité (PUBLIC/PRIVATE)
 - ☐ Creator = owner
 - ☐ Invite members

5.3.8.2 US029: Rejoindre un groupe

- **As a** utilisateur
- **I want to** rejoindre un groupe public
- **So that** j'accède au contenu
- **Acceptance Criteria:**
 - ☐ Auto-join si PUBLIC
 - ☐ Request if PRIVATE
 - ☐ Owner approval
 - ☐ Add to my groups

5.3.8.3 US030: Voir groupe détails

- **As a** utilisateur
- **I want to** voir les infos d'un groupe
- **So that** je sais ce que c'est
- **Acceptance Criteria:**

- ☐ Nom + description
- ☐ Membres list
- ☐ Events tab
- ☐ Chat tab
- ☐ Settings (owner only)

5.3.8.4 US031: Inviter dans groupe

- **As a** group owner
- **I want to** inviter des utilisateurs
- **So that** elles rejoignent le groupe
- **Acceptance Criteria:**
 - ☐ Select users
 - ☐ Send invite
 - ☐ Notification reçue
 - ☐ Accept/decline

5.3.8.5 US032: Communiquer dans groupe

- **As a** membre groupe
 - **I want to** envoyer des messages dans le groupe
 - **So that** on collabore
 - **Acceptance Criteria:**
 - ☐ Group chat (comme DM)
 - ☐ All members voient
 - ☐ Typing indicator
 - ☐ Attachments (images)
-

5.3.9 Événements (Events)

5.3.9.1 US033: Créer un événement

- **As a** group owner
- **I want to** créer un événement dans le groupe
- **So that** on organise une activité
- **Acceptance Criteria:**
 - ☐ Titre + description
 - ☐ Date/heure
 - ☐ Location (text)
 - ☐ Creator = attendee YES

5.3.9.2 US034: Consulter événements

- **As a** utilisateur
- **I want to** voir les événements de mes groupes
- **So that** je vois le calendrier
- **Acceptance Criteria:**
 - ☐ List view + calendar
 - ☐ Filter (upcoming, past, my events)
 - ☐ Event details: title, date, members attending
 - ☐ RSVP count

5.3.9.3 US035: RSVP événement

- **As a** utilisateur
 - **I want to** répondre à un événement (YES/NO/MAYBE)
 - **So that** l'organisateur sait combien attendus
 - **Acceptance Criteria:**
 - ☐ 3 options: YES, NO, MAYBE
 - ☐ Change response
 - ☐ Affiche attendees
 - ☐ Push notif 24h avant
-

5.3.10 Reels (Vidéos)

5.3.10.1 US036: Partager une vidéo

- **As a** utilisateur
- **I want to** partager une vidéo courte
- **So that** je crée du contenu engageant
- **Acceptance Criteria:**
 - ☐ Video upload
 - ☐ Auto-trim si > 60s
 - ☐ Thumbnail generation
 - ☐ Published dans /reels

5.3.10.2 US037: Regarder reels

- **As a** utilisateur
- **I want to** scroller verticalement des vidéos
- **So that** je découvre du contenu (TikTok style)
- **Acceptance Criteria:**

- ☐ Fullscreen vertical video
- ☐ Infinite scroll
- ☐ Auto-play next
- ☐ Pause on scroll out

5.3.10.3 US038: Interagir avec reel

- **As a** utilisateur
 - **I want to** liker/commenter une vidéo
 - **So that** j'engage avec le créateur
 - **Acceptance Criteria:**
 - ☐ Like button floating
 - ☐ Comment button
 - ☐ Share button
 - ☐ Creator info (link to profile)
-

5.4 Priorités & Estimation

5.4.1 Phase 1: MVP (MUST HAVE) - 4 semaines

User Story	Priorité	Estimation
US001	HIGH	2 jours
US002	HIGH	1 jour
US003	HIGH	2 jours
US004	HIGH	1 jour
US005	HIGH	2 jours
US007	HIGH	3 jours
US008	HIGH	2 jours
US009	HIGH	1 jour
US011	HIGH	2 jours
US021	HIGH	3 jours
US024	MEDIUM	2 jours
US026	MEDIUM	2 jours
Total		25 jours

5.4.2 Phase 2: Features (SHOULD HAVE) - 2 semaines

- US010, US014, US017, US018, US028, US033, US036

5.4.3 Phase 3: Polish (NICE TO HAVE) - 1 semaine

- US013, US016, US023
-

5.5 Acceptance Criteria Summary

Front-End:

- Responsive design (mobile, tablet, desktop)
- Theme switcher (light/dark)
- Loading states + error handling
- Infinite scroll pagination
- Real-time updates (SSE / Redis)
- Image optimization (Cloudinary)
- SEO optimization (meta tags)
- Accessibility (WCAG 2.1 AA)

Back-End:

- JWT authentication
- Input validation + sanitization
- Error logging
- Database indexes

DevOps:

- Docker containerization
 - CI/CD pipeline (GitHub Actions)
-

5.6 Prochaines Étapes

1. User stories écrites
 - ☐ Wireframes créés pour le projet
 - ☐ Prototype Figma du projet
 - ☐ Validation avec l'équipe
 - ☐ Sprint planning
 - ☐ Implémentation Phase 1

6 Modélisation de Données

6.1 Dictionnaire de Données

Source : `prisma/schema.prisma` — traduit en types SQL PostgreSQL.

Conventions de types :

Type SQL	Description
CHAR(25)	Identifiant CUID (chaîne fixe 25 caractères)
VARCHAR(255)	Chaîne courte (nom, URL, token)
TEXT	Chaîne longue (message, biographie)
TIMESTAMP	Date et heure
BOOLEAN	Vrai / Faux
BIGINT	Entier 64 bits (timestamp OAuth)
ENUM(...)	Valeur parmi un ensemble fini

6.2 Model: User

Description : Entité centrale représentant un utilisateur du réseau social.

6.2.1 Attributs

Attribut	Type SQL	Contraintes	Description
id	CHAR(25)	PRIMARY KEY, NOT NULL	Identifiant CUID
firstName	VARCHAR(255)	NULL	Prénom
lastName	VARCHAR(255)	NULL	Nom de famille
password	VARCHAR(255)	NULL	Hash bcrypt du mot de passe
email	VARCHAR(255)	UNIQUE, NOT NULL	Adresse email
birthDate	TIMESTAMP	NULL	Date de naissance
username	VARCHAR(50)	UNIQUE, NULL	Pseudonyme
biography	TEXT	NULL	Biographie
avatar	TEXT	NULL	URL avatar (Cloudinary)

Attribut	Type SQL	Contraintes	Description
avatarId	VARCHAR(255)	NULL	ID asset Cloudinary
banner	TEXT	NULL	URL bannière (Cloudinary)
bannerId	VARCHAR(255)	NULL	ID asset Cloudinary
visibility	ENUM('PUBLIC','PRIVATE')	DEFAULT 'PUBLIC', NOT NULL	Visibilité du profil

6.2.2 Relations

- `posts : Post[]` – Posts créés
- `comments : Comment[]` – Commentaires
- `friendships : Friendship[]` – Amitiés initiées
- `friendsWithMe : Friendship[]` – Amitiés reçues
- `conversations : Conversation[]` – Conversations créées
- `conversationMembers : ConversationMember[]` – Participations aux conversations
- `messages : Message[]` (SentMessages) – Messages envoyés
- `receivedMessages : Message[]` – Messages reçus
- `notifications : Notification[]` – Notifications reçues
- `reactions : Reaction[]` – Réactions publiées
- `stories : Story[]` – Stories publiées
- `rsvps : Rsvp[]` – Réponses aux événements
- `groupInvitations : GroupInvitation[]` (invitant + invité)
- `groupMembers : GroupMember[]` – Adhésions aux groupes
- `groupMessages : GroupMessage[]` – Messages de groupe
- `eventsCreated : Event[]` – Événements créés
- `userSettings : UserSettings` – Préférences
- `accounts : Account[]` – Comptes OAuth (Google)

6.3 Model: Post

Description : Publication sur le fil d'actualité (texte, image, vidéo).

6.3.1 Attributs

Attribut	Type SQL	Contraintes	Description
id	CHAR(25)	PRIMARY KEY, NOT NULL	Identifiant CUID

Attribut	Type SQL	Contraintes	Description
userId	CHAR(25)	FOREIGN KEY (User), NOT NULL	Auteur du post
message	TEXT	NOT NULL	Contenu textuel
datetime	TIMESTAMP	DEFAULT NOW(), NOT NULL	Date de publication
image	TEXT	NULL	URL image (Cloudinary)
mediaId	VARCHAR(255)	NULL	ID asset Cloudinary
visibility	ENUM('PUBLIC','PRIVATE','FRIENDS')	DEFAULT 'PUBLIC', NOT NULL	Visibilité du post

6.3.2 Enum

- `Visibility` : PUBLIC, PRIVATE, FRIENDS

6.3.3 Relations

- `user` : User — Auteur
- `comments` : Comment[] — Commentaires
- `reactions` : Reaction[] — Réactions

6.4 Model: Comment

Description : Commentaire posté sur un Post.

6.4.1 Attributs

Attribut	Type SQL	Contraintes	Description
id	CHAR(25)	PRIMARY KEY, NOT NULL	Identifiant CUID
postId	CHAR(25)	FOREIGN KEY (Post), NOT NULL	Post parent
userId	CHAR(25)	FOREIGN KEY (User), NOT NULL	Auteur
message	TEXT	NOT NULL	Contenu
datetime	TIMESTAMP	DEFAULT NOW(), NOT NULL	Date de création

6.4.2 Relations

- `post` : `Post`
- `user` : `User`
- `reactions` : `Reaction[]`

6.5 Model: Reaction

Description : Réaction d'un utilisateur sur un Post, une Story ou un Comment.

6.5.1 Attributs

Attribut	Type SQL	Contraintes	Description
id	CHAR(25)	PRIMARY KEY, NOT NULL	Identifiant CUID
type	ENUM('LIKE','DISLIKE','LOVE','LAUGH','SAD','ANGRY','WOW')	NOT NULL	Type de réaction
userId	CHAR(25)	FOREIGN KEY (User), NOT NULL	Auteur
postId	CHAR(25)	FOREIGN KEY (Post), NULL	Post ciblé
storyId	CHAR(25)	FOREIGN KEY (Story), NULL	Story ciblée
commentId	CHAR(25)	FOREIGN KEY (Comment), NULL	Commentaire ciblé
datetime	TIMESTAMP	DEFAULT NOW(), NOT NULL	Date

6.5.2 Contraintes d'unicité

- (`userId`, `postId`) — une réaction par utilisateur par post
- (`userId`, `storyId`) — une réaction par utilisateur par story
- (`userId`, `commentId`) — une réaction par utilisateur par commentaire

6.5.3 Relations

- `user` : `User`
- `post` : `Post` (nullable)

- `story` : Story (nullable)
- `comment` : Comment (nullable)

6.6 Model: Message

Description : Message privé direct entre deux utilisateurs (1-to-1).

6.6.1 Attributs

Attribut	Type SQL	Contraintes	Description
<code>id</code>	CHAR(25)	PRIMARY KEY, NOT NULL	Identifiant CUID
<code>senderId</code>	CHAR(25)	FOREIGN KEY (User), NOT NULL	Expéditeur
<code>receiverId</code>	CHAR(25)	FOREIGN KEY (User), NOT NULL	Destinataire
<code>message</code>	TEXT	NOT NULL	Contenu
<code>image</code>	TEXT	NULL	URL image attachée
<code>datetime</code>	TIMESTAMP	DEFAULT NOW(), NOT NULL	Date d'envoi
<code>deliveredAt</code>	TIMESTAMP	NULL	Date de livraison
<code>readAt</code>	TIMESTAMP	NULL	Date de lecture
<code>status</code>	ENUM('SENT','DELIVERED','READ')	DEFAULT 'SENT', NOT NULL	Statut du message

6.6.2 Relations

- `sender` : User
- `receiver` : User

6.7 Model: Friendship

Description : Relation d'amitié entre deux utilisateurs.

6.7.1 Attributs

Attribut	Type SQL	Contraintes	Description
id	CHAR(25)	PRIMARY KEY, NOT NULL	Identifiant CUID
userId	CHAR(25)	FOREIGN KEY (User), NOT NULL	Demandeur
friendId	CHAR(25)	FOREIGN KEY (User), NOT NULL	Destinataire
status	ENUM('PENDING','ACCEPTED')	NOT NULL	Statut de la demande
createdAt	TIMESTAMP	DEFAULT NOW(), NOT NULL	Date de création

6.7.2 Contrainte d'unicité

- (userId, friendId) — pas de doublon de relation

6.7.3 Relations

- user : User (demandeur)
- friend : User (ami)

6.8 Model: Story

Description : Contenu éphémère publié par un utilisateur.

6.8.1 Attributs

Attribut	Type SQL	Contraintes	Description
id	CHAR(25)	PRIMARY KEY, NOT NULL	Identifiant CUID
userId	CHAR(25)	FOREIGN KEY (User), NOT NULL	Créateur
datetime	TIMESTAMP	DEFAULT NOW(), NOT NULL	Date de publication
media	TEXT	NULL	URL média (Cloudinary)

Attribut	Type SQL	Contraintes	Description
mediaId	VARCHAR(255)	NULL	ID asset Cloudinary
visibility	ENUM('PUBLIC','PRIVATE','FRIENDS')	DEFAULT 'PUBLIC', NOT NULL	Visibilité

6.8.2 Relations

- user : User
- reactions : Reaction[]

6.9 Model: Conversation

Description : Conversation de groupe ou wrapper DM.

6.9.1 Attributs

Attribut	Type SQL	Contraintes	Description
id	CHAR(25)	PRIMARY KEY, NOT NULL	Identifiant CUID
title	VARCHAR(255)	NULL	Nom du groupe
isGroup	BOOLEAN	DEFAULT FALSE, NOT NULL	Est un groupe
createdAt	TIMESTAMP	DEFAULT NOW(), NOT NULL	Date de création
ownerId	CHAR(25)	FOREIGN KEY (User), NOT NULL	Propriétaire du groupe

6.9.2 Relations

- owner : User (nullable)
- members : ConversationMember[]
- messages : GroupMessage[]
- events : Event[]
- groupInvitations : GroupInvitation[]
- groupMembers : GroupMember[]
- groupJoinRequests : GroupJoinRequest[]

6.10 Model: ConversationMember

Description : Participant à une conversation.

6.10.1 Attributs

Attribut	Type SQL	Contraintes	Description
id	CHAR(25)	PRIMARY KEY, NOT NULL	Identifiant CUID
userId	CHAR(25)	FOREIGN KEY (User), NOT NULL	Utilisateur
conversationId	CHAR(25)	FOREIGN KEY (Conversation), NOT NULL	Conversation
joinedAt	TIMESTAMP	DEFAULT NOW(), NOT NULL	Date d'adhésion
lastSeenAt	TIMESTAMP	NULL	Dernier accès
lastSeenMessageId	CHAR(25)	NULL	Dernier message vu

6.10.2 Contrainte d'unicité

- (userId, conversationId) — pas de doublon

6.11 Model: GroupMessage

Description : Message dans une conversation de groupe.

6.11.1 Attributs

Attribut	Type SQL	Contraintes	Description
id	CHAR(25)	PRIMARY KEY, NOT NULL	Identifiant CUID
conversationId	CHAR(25)	FOREIGN KEY (Conversation), NOT NULL	Conversation parente
senderId	CHAR(25)	FOREIGN KEY (User), NOT NULL	Expéditeur
message	TEXT	NOT NULL	Contenu
image	TEXT	NULL	URL image
sentAt	TIMESTAMP	DEFAULT NOW(), NOT NULL	Date d'envoi

Attribut	Type SQL	Contraintes	Description
eventId	CHAR(25)	FOREIGN KEY (Event), NULL	Événement associé
deliveredAt	TIMESTAMP	NULL	Date de livraison
readAt	TIMESTAMP	NULL	Date de lecture
status	ENUM('SENT','DELIVERED','READ')	DEFAULT 'SENT', NOT NULL	Statut

6.12 Model: GroupInvitation

Description : Invitation d'un utilisateur à rejoindre un groupe.

6.12.1 Attributs

Attribut	Type SQL	Contraintes	Description
id	CHAR(25)	PRIMARY KEY, NOT NULL	Identifiant CUID
groupId	CHAR(25)	FOREIGN KEY (Conversation), NOT NULL	Groupe cible
inviterId	CHAR(25)	FOREIGN KEY (User), NOT NULL	Invitant
invitedId	CHAR(25)	FOREIGN KEY (User), NOT NULL	Invité
status	ENUM('PENDING','ACCEPTED')	DEFAULT 'PENDING', NOT NULL	Statut
createdAt	TIMESTAMP	DEFAULT NOW(), NOT NULL	Date de création

6.12.2 Contrainte d'unicité

- (groupId, invitedId) — pas de doublon

6.13 Model: GroupJoinRequest

Description : Demande spontanée d'adhésion à un groupe.

6.13.1 Attributs

Attribut	Type SQL	Contraintes	Description
id	CHAR(25)	PRIMARY KEY, NOT NULL	Identifiant CUID
groupId	CHAR(25)	FOREIGN KEY (Conversation), NOT NULL	Groupe cible
seeker	CHAR(25)	FOREIGN KEY (User), NOT NULL	Demandeur
status	ENUM('PENDING','ACCEPTED')	DEFAULT 'PENDING', NOT NULL	Statut
createdAt	TIMESTAMP	DEFAULT NOW(), NOT NULL	Date de création

6.13.2 Contrainte d'unicité

- (groupId, seeker) — pas de doublon

6.14 Model: GroupMember

Description : Adhésion confirmée d'un utilisateur à un groupe.

6.14.1 Attributs

Attribut	Type SQL	Contraintes	Description
id	CHAR(25)	PRIMARY KEY, NOT NULL	Identifiant CUID
groupId	CHAR(25)	FOREIGN KEY (Conversation), NOT NULL	Groupe
userId	CHAR(25)	FOREIGN KEY (User), NOT NULL	Membre
joinedAt	TIMESTAMP	DEFAULT NOW(), NOT NULL	Date d'adhésion

6.14.2 Contrainte d'unicité

- (groupId, userId) — pas de doublon

6.15 Model: Event

Description : Événement créé dans un groupe, avec RSVP.

6.15.1 Attributs

Attribut	Type SQL	Contraintes	Description
id	CHAR(25)	PRIMARY KEY, NOT NULL	Identifiant CUID
title	VARCHAR(255)	NOT NULL	Titre
description	TEXT	NOT NULL	Description
datetime	TIMESTAMP	NOT NULL	Date/heure
groupId	CHAR(25)	FOREIGN KEY (Conversation), NOT NULL	Groupe hôte
ownerId	CHAR(25)	FOREIGN KEY (User), NOT NULL	Créateur
createdAt	TIMESTAMP	DEFAULT NOW(), NOT NULL	Date de création

6.15.2 Relations

- group : Conversation
- owner : User
- messages : GroupMessage[]
- rsvps : Rsvp[]

6.16 Model: Rsvp

Description : Réponse d'un utilisateur à un événement.

6.16.1 Attributs

Attribut	Type SQL	Contraintes	Description
id	CHAR(25)	PRIMARY KEY, NOT NULL	Identifiant CUID
userId	CHAR(25)	FOREIGN KEY (User), NOT NULL	Utilisateur
eventId	CHAR(25)	FOREIGN KEY (Event), NOT NULL	Événement
status	ENUM('YES','NO','MAYBE')	NOT NULL	Réponse
createdAt	TIMESTAMP	DEFAULT NOW(), NOT NULL	Date de réponse

6.16.2 Contrainte d'unicité

- (userId, eventId) — un RSVP par utilisateur par événement

6.17 Model: UserSettings

Description : Préférences de l'utilisateur (thème, langue, notifications).

6.17.1 Attributs

Attribut	Type SQL	Contraintes	Description
id	CHAR(25)	PRIMARY KEY, NOT NULL	Identifiant CUID
userId	CHAR(25)	UNIQUE, FOREIGN KEY (User), NOT NULL	Utilisateur (1-to-1)
theme	VARCHAR(10)	DEFAULT 'light', NOT NULL	Thème (light / dark)
language	VARCHAR(10)	DEFAULT 'en', NOT NULL	Code langue (en, fr)
notificationsEnabled	BOOLEAN	DEFAULT TRUE, NOT NULL	Notifications actives
createdAt	TIMESTAMP	DEFAULT NOW(), NOT NULL	Date de création

6.18 Model: Notification

Description : Notification système envoyée à un utilisateur.

6.18.1 Attributs

Attribut	Type SQL	Contraintes	Description
id	CHAR(25)	PRIMARY KEY, NOT NULL	Identifiant CUID
userId	CHAR(25)	FOREIGN KEY (User), NOT NULL	Destinataire
type	VARCHAR(50)	NOT NULL	Type d'événement
message	TEXT	NOT NULL	Contenu
isRead	BOOLEAN	DEFAULT FALSE, NOT NULL	Lue ou non
createdAt	TIMESTAMP	DEFAULT NOW(), NOT NULL	Date de création

6.18.2 Index

- `userId` – accès rapide aux notifications par utilisateur

6.19 Model: Account

Description : Compte OAuth lié à un utilisateur (Google).

6.19.1 Attributs

Attribut	Type SQL	Contraintes	Description
<code>id</code>	<code>CHAR(25)</code>	PRIMARY KEY, NOT NULL	Identifiant CUID
<code>userId</code>	<code>CHAR(25)</code>	FOREIGN KEY (User), NOT NULL	Utilisateur propriétaire
<code>provider</code>	<code>VARCHAR(50)</code>	NOT NULL	Fournisseur (google...)
<code>providerAccountId</code>	<code>VARCHAR(255)</code>	NOT NULL	ID chez le fournisseur
<code>refresh_token</code>	<code>TEXT</code>	NULL	Token de rafraîchissement
<code>access_token</code>	<code>TEXT</code>	NULL	Token d'accès
<code>expires_at</code>	<code>BIGINT</code>	NULL	Expiration (epoch ms)
<code>token_type</code>	<code>VARCHAR(50)</code>	NULL	Type de token
<code>scope</code>	<code>TEXT</code>	NULL	Permissions accordées
<code>id_token</code>	<code>TEXT</code>	NULL	ID token JWT
<code>session_state</code>	<code>VARCHAR(255)</code>	NULL	État de session

6.19.2 Contrainte d'unicité

- (`provider`, `providerAccountId`) – un compte par fournisseur

6.20 MLD – Modèle Logique de Données

User (id, firstName, lastName, password, email, birthDate, username, biography)
Account (id, userId#, provider, providerAccountId, refresh_token, access_token)
Post (id, userId#, message, datetime, image, mediaId, visibility)
Comment (id, postId#, userId#, message, datetime)


```

Reaction      (id, type, userId#, postId#, storyId#, commentId#, datetime)
Story         (id, userId#, datetime, media, mediaId, visibility)
Message       (id, senderId#, receiverId#, message, image, datetime, deliveredAt, r
Friendship    (id, userId#, friendId#, status, createdAt)
Conversation   (id, title, isGroup, createdAt, ownerId#)
ConversationMember (id, userId#, conversationId#, joinedAt, lastSeenAt, lastSeen
GroupMessage  (id, conversationId#, senderId#, message, image, sentAt, eventId#,
GroupInvitation (id, groupId#, inviterId#, invitedId#, status, createdAt)
GroupJoinRequest (id, groupId#, seeker#, status, createdAt)
GroupMember   (id, groupId#, userId#, joinedAt)
Event         (id, title, description, datetime, groupId#, ownerId#, createdAt)
Rsvp          (id, userId#, eventId#, status, createdAt)
UserSettings  (id, userId#, theme, language, notificationsEnabled, createdAt)
Notification   (id, userId#, type, message, isRead, createdAt)

```

désigne une clé étrangère (FOREIGN KEY).

6.21 Cardinalités

Relation	Type	Description
User → Post	1,N	Un utilisateur publie plusieurs posts
User → Comment	1,N	Un utilisateur écrit plusieurs commentaires
User → Reaction	1,N	Un utilisateur crée plusieurs réactions
Post → Comment	1,N	Un post reçoit plusieurs commentaires
Post / Story / Comment → Reaction	1,N	Chaque entité reçoit plusieurs réactions
User → Friendship	1,N	Un utilisateur a plusieurs relations d'amitié
Conversation → GroupMessage	1,N	Une conversation contient plusieurs messages
Conversation → ConversationMember	1,N	Une conversation a plusieurs participants
User → Event	1,N	Un utilisateur crée plusieurs événements
Event → Rsvp	1,N	Un événement reçoit plusieurs réponses
User → UserSettings	1,1	Un utilisateur a exactement un paramétrage
User → Account	1,N	Un utilisateur peut avoir plusieurs comptes OAuth

6.22 Checklist Modélisation

- ☒ Dictionnaire de données — types SQL PostgreSQL
 - ☒ Relations et clés étrangères documentées
 - ☒ Enums PostgreSQL définis
 - ☒ Contraintes d'unicité listées
 - ☒ MLD avec notation clés étrangères
 - ☒ Cardinalités définies
 - ☒ Tests de cohérence (contraintes vérifiées via `prisma validate`)
-

6.23 Diagrammes Complets

Les diagrammes MCD, MLD et MPD en format DBML et SQL sont disponibles dans [diagrammes-uml.md](#).

7 Diagrammes UML - Social Network (Conforme Prisma)

7.1 Objectif

Ce document fournit des versions exploitables du modèle de données à partir de la source de vérité du projet: `prisma/schema.prisma`.

- **MCD**: vision conceptuelle (entités et relations)
 - **MLD**: vision logique (tables, colonnes, contraintes)
 - **MPD**: vision physique PostgreSQL (DDL)
-

7.2 1. MCD (Modèle Conceptuel de Données)

Copie le code suivant dans dbdiagram.io:

```
// MCD - aligné avec prisma/schema.prisma
```

```
Table User {
  id string [pk]
  firstName string
  lastName string
  email string [unique, not null]
  username string [unique]
  password string
  birthDate datetime
  biography text
  avatar string
  avatarId string
  banner string
  bannerId string
  visibility enum [default: 'PUBLIC']
}
```

```
Table Account {
  id string [pk]
  userId string [not null]
  provider string [not null]
  providerAccountId string [not null]
  refresh_token string
  access_token string
  expires_at bigint
  token_type string
  scope string
  id_token string
  session_state string
}
```

```
Table Post {
```

```

    id string [pk]
    userId string [not null]
    message text [not null]
    datetime datetime
    image string
    mediaId string
    visibility enum [default: 'PUBLIC']
}

```

```

Table Story {
    id string [pk]
    userId string [not null]
    datetime datetime
    media string
    mediaId string
    visibility enum [default: 'PUBLIC']
}

```

```

Table Comment {
    id string [pk]
    postId string [not null]
    userId string [not null]
    message text [not null]
    datetime datetime
}

```

```

Table Reaction {
    id string [pk]
    type enum [not null]
    userId string [not null]
    postId string
    storyId string
    commentId string
    datetime datetime
}

```

```

Table Message {
    id string [pk]
    senderId string [not null]
    receiverId string [not null]
    message text [not null]
    image string
    datetime datetime
    deliveredAt datetime
    readAt datetime
    status enum [default: 'SENT']
}

```

```

Table Friendship {
    id string [pk]
    userId string [not null]
    friendId string [not null]
}

```

```

    status enum [not null]
    createdAt datetime
}

Table Conversation {
    id string [pk]
    title string
    isGroup boolean [default: false]
    createdAt datetime
    ownerId string
}

Table ConversationMember {
    id string [pk]
    userId string [not null]
    conversationId string [not null]
    joinedAt datetime
    lastSeenAt datetime
    lastSeenMessageId string
}

Table GroupMessage {
    id string [pk]
    conversationId string [not null]
    senderId string [not null]
    message text [not null]
    image string
    sentAt datetime
    eventId string
    deliveredAt datetime
    readAt datetime
    status enum [default: 'SENT']
}

Table Event {
    id string [pk]
    title string [not null]
    description text [not null]
    datetime datetime [not null]
    groupId string [not null]
    ownerId string [not null]
    createdAt datetime
}

Table Rsvp {
    id string [pk]
    userId string [not null]
    eventId string [not null]
    status enum [not null]
    createdAt datetime
}

```

```

Table GroupInvitation {
  id string [pk]
  groupId string [not null]
  inviterId string [not null]
  invitedId string [not null]
  status enum [default: 'PENDING']
  createdAt datetime
}

Table GroupJoinRequest {
  id string [pk]
  groupId string [not null]
  seeker string [not null]
  status enum [default: 'PENDING']
  createdAt datetime
}

Table GroupMember {
  id string [pk]
  groupId string [not null]
  userId string [not null]
  joinedAt datetime
}

Table Notification {
  id string [pk]
  userId string [not null]
  type string [not null]
  message text [not null]
  isRead boolean [default: false]
  createdAt datetime
}

Table UserSettings {
  id string [pk]
  userId string [not null, unique]
  theme string [default: 'light']
  language string [default: 'en']
  notificationsEnabled boolean [default: true]
  createdAt datetime
}

// Relations
Ref: Account.userId > User.id
Ref: Post.userId > User.id
Ref: Story.userId > User.id
Ref: Comment.postId > Post.id
Ref: Comment.userId > User.id
Ref: Reaction.userId > User.id
Ref: Reaction.postId > Post.id
Ref: Reaction.storyId > Story.id
Ref: Reaction.commentId > Comment.id

```

```

Ref: Message.senderId > User.id
Ref: Message.receiverId > User.id
Ref: Friendship.userId > User.id
Ref: Friendship.friendId > User.id
Ref: Conversation.ownerId > User.id
Ref: ConversationMember.userId > User.id
Ref: ConversationMember.conversationId > Conversation.id
Ref: GroupMessage.conversationId > Conversation.id
Ref: GroupMessage.senderId > User.id
Ref: GroupMessage.eventId > Event.id
Ref: Event.groupId > Conversation.id
Ref: Event.ownerId > User.id
Ref: Rsvp.userId > User.id
Ref: Rsvp.eventId > Event.id
Ref: GroupInvitation.groupId > Conversation.id
Ref: GroupInvitation.inviterId > User.id
Ref: GroupInvitation.invitedId > User.id
Ref: GroupJoinRequest.groupId > Conversation.id
Ref: GroupJoinRequest.seeker > User.id
Ref: GroupMember.groupId > Conversation.id
Ref: GroupMember.userId > User.id
Ref: Notification.userId > User.id
Ref: UserSettings.userId > User.id

```

7.3 2. MLD (Modèle Logique de Données)

Version logique en DBML (naming SQL-like) compatible dbdiagram.io.

```

Table users {
  id text [pk]
  first_name varchar(100)
  last_name varchar(100)
  email varchar(255) [unique, not null]
  username varchar(100) [unique]
  password_hash varchar(255)
  birth_date timestampz
  biography text
  avatar varchar(500)
  avatar_id varchar(255)
  banner varchar(500)
  banner_id varchar(255)
  visibility varchar(20) [default: 'PUBLIC']
}

Table accounts {
  id text [pk]
  user_id text [not null]
  provider varchar(100) [not null]
}

```

```

provider_account_id varchar(255) [not null]
refresh_token text
access_token text
expires_at bigint
token_type varchar(50)
scope text
id_token text
session_state text
}

Table posts {
  id text [pk]
  user_id text [not null]
  message text [not null]
  datetime timestampz [default: `now()`]
  image varchar(500)
  media_id varchar(255)
  visibility varchar(20) [default: 'PUBLIC']
}

Table stories {
  id text [pk]
  user_id text [not null]
  datetime timestampz [default: `now()`]
  media varchar(500)
  media_id varchar(255)
  visibility varchar(20) [default: 'PUBLIC']
}

Table comments {
  id text [pk]
  post_id text [not null]
  user_id text [not null]
  message text [not null]
  datetime timestampz [default: `now()`]
}

Table reactions {
  id text [pk]
  type varchar(20) [not null]
  user_id text [not null]
  post_id text
  story_id text
  comment_id text
  datetime timestampz [default: `now()`]

  indexes {
    (user_id, post_id) [unique]
    (user_id, story_id) [unique]
    (user_id, comment_id) [unique]
  }
}

```



```

Table messages {
  id text [pk]
  sender_id text [not null]
  receiver_id text [not null]
  message text [not null]
  image varchar(500)
  datetime timestamptz [default: `now()`]
  delivered_at timestamptz
  read_at timestamptz
  status varchar(20) [default: 'SENT']
}

Table friendships {
  id text [pk]
  user_id text [not null]
  friend_id text [not null]
  status varchar(20) [not null]
  created_at timestamptz [default: `now()`]

  indexes {
    (user_id, friend_id) [unique]
  }
}

Table conversations {
  id text [pk]
  title varchar(255)
  is_group boolean [default: false]
  created_at timestamptz [default: `now()`]
  owner_id text
}

Table conversation_members {
  id text [pk]
  user_id text [not null]
  conversation_id text [not null]
  joined_at timestamptz [default: `now()`]
  last_seen_at timestamptz
  last_seen_message_id text

  indexes {
    (user_id, conversation_id) [unique]
  }
}

Table group_messages {
  id text [pk]
  conversation_id text [not null]
  sender_id text [not null]
  message text [not null]
  image varchar(500)

```

```

    sent_at timestampz [default: `now()`]
    event_id text
    delivered_at timestampz
    read_at timestampz
    status varchar(20) [default: 'SENT']
}

Table events {
    id text [pk]
    title varchar(255) [not null]
    description text [not null]
    datetime timestampz [not null]
    group_id text [not null]
    owner_id text [not null]
    created_at timestampz [default: `now()`]
}

Table rsvps {
    id text [pk]
    user_id text [not null]
    event_id text [not null]
    status varchar(20) [not null]
    created_at timestampz [default: `now()`]

    indexes {
        (user_id, event_id) [unique]
    }
}

Table group_invitations {
    id text [pk]
    group_id text [not null]
    inviter_id text [not null]
    invited_id text [not null]
    status varchar(20) [default: 'PENDING']
    created_at timestampz [default: `now()`]

    indexes {
        (group_id, invited_id) [unique]
    }
}

Table group_join_requests {
    id text [pk]
    group_id text [not null]
    seeker text [not null]
    status varchar(20) [default: 'PENDING']
    created_at timestampz [default: `now()`]

    indexes {
        (group_id, seeker) [unique]
    }
}

```

```

}

Table group_members {
  id text [pk]
  group_id text [not null]
  user_id text [not null]
  joined_at timestamptz [default: `now()`]

  indexes {
    (group_id, user_id) [unique]
  }
}

Table notifications {
  id text [pk]
  user_id text [not null]
  type varchar(100) [not null]
  message text [not null]
  is_read boolean [default: false]
  created_at timestamptz [default: `now()`]

  indexes {
    (user_id)
  }
}

Table user_settings {
  id text [pk]
  user_id text [not null, unique]
  theme varchar(20) [default: 'light']
  language varchar(10) [default: 'en']
  notifications_enabled boolean [default: true]
  created_at timestamptz [default: `now()`]
}

Ref: accounts.user_id > users.id
Ref: posts.user_id > users.id
Ref: stories.user_id > users.id
Ref: comments.post_id > posts.id
Ref: comments.user_id > users.id
Ref: reactions.user_id > users.id
Ref: reactions.post_id > posts.id
Ref: reactions.story_id > stories.id
Ref: reactions.comment_id > comments.id
Ref: messages.sender_id > users.id
Ref: messages.receiver_id > users.id
Ref: friendships.user_id > users.id
Ref: friendships.friend_id > users.id
Ref: conversations.owner_id > users.id
Ref: conversation_members.user_id > users.id
Ref: conversation_members.conversation_id > conversations.id
Ref: group_messages.conversation_id > conversations.id

```

```

Ref: group_messages.sender_id > users.id
Ref: group_messages.event_id > events.id
Ref: events.group_id > conversations.id
Ref: events.owner_id > users.id
Ref: rsvps.user_id > users.id
Ref: rsvps.event_id > events.id
Ref: group_invitations.group_id > conversations.id
Ref: group_invitations.inviter_id > users.id
Ref: group_invitations.invited_id > users.id
Ref: group_join_requests.group_id > conversations.id
Ref: group_join_requests.seeker > users.id
Ref: group_members.group_id > conversations.id
Ref: group_members.user_id > users.id
Ref: notifications.user_id > users.id
Ref: user_settings.user_id > users.id

```

7.4 3. MPD (Modèle Physique de Données - PostgreSQL)

Remarque: Prisma génère des identifiants `cuid()` côté application, donc les PK sont stockées en TEXT.

```

-- MPD PostgreSQL aligné sur prisma/schema.prisma

CREATE TABLE users (
  id TEXT PRIMARY KEY,
  first_name VARCHAR(100),
  last_name VARCHAR(100),
  email VARCHAR(255) UNIQUE NOT NULL,
  username VARCHAR(100) UNIQUE,
  password_hash VARCHAR(255),
  birth_date TIMESTAMPTZ,
  biography TEXT,
  avatar VARCHAR(500),
  avatar_id VARCHAR(255),
  banner VARCHAR(500),
  banner_id VARCHAR(255),
  visibility VARCHAR(20) NOT NULL DEFAULT 'PUBLIC'
);

CREATE TABLE accounts (
  id TEXT PRIMARY KEY,
  user_id TEXT NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  provider VARCHAR(100) NOT NULL,
  provider_account_id VARCHAR(255) NOT NULL,
  refresh_token TEXT,
  access_token TEXT,
  expires_at BIGINT,
  token_type VARCHAR(50),

```

```

scope TEXT,
id_token TEXT,
session_state TEXT,
CONSTRAINT accounts_provider_account_unique UNIQUE (provider,
    ↪ provider_account_id)
);

CREATE TABLE posts (
    id TEXT PRIMARY KEY,
    user_id TEXT NOT NULL REFERENCES users(id),
    message TEXT NOT NULL,
    datetime TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    image VARCHAR(500),
    media_id VARCHAR(255),
    visibility VARCHAR(20) NOT NULL DEFAULT 'PUBLIC'
);

CREATE TABLE stories (
    id TEXT PRIMARY KEY,
    user_id TEXT NOT NULL REFERENCES users(id),
    datetime TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    media VARCHAR(500),
    media_id VARCHAR(255),
    visibility VARCHAR(20) NOT NULL DEFAULT 'PUBLIC'
);

CREATE TABLE comments (
    id TEXT PRIMARY KEY,
    post_id TEXT NOT NULL REFERENCES posts(id),
    user_id TEXT NOT NULL REFERENCES users(id),
    message TEXT NOT NULL,
    datetime TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE TABLE reactions (
    id TEXT PRIMARY KEY,
    type VARCHAR(20) NOT NULL,
    user_id TEXT NOT NULL REFERENCES users(id),
    post_id TEXT REFERENCES posts(id),
    story_id TEXT REFERENCES stories(id),
    comment_id TEXT REFERENCES comments(id),
    datetime TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    CONSTRAINT reactions_user_post_unique UNIQUE (user_id, post_id),
    CONSTRAINT reactions_user_story_unique UNIQUE (user_id, story_id),
    CONSTRAINT reactions_user_comment_unique UNIQUE (user_id, comment_id),
    CONSTRAINT reactions_single_target CHECK (
        (post_id IS NOT NULL AND story_id IS NULL AND comment_id IS NULL) OR
        (post_id IS NULL AND story_id IS NOT NULL AND comment_id IS NULL) OR
        (post_id IS NULL AND story_id IS NULL AND comment_id IS NOT NULL)
    )
);

```

```

CREATE TABLE messages (
  id TEXT PRIMARY KEY,
  sender_id TEXT NOT NULL REFERENCES users(id),
  receiver_id TEXT NOT NULL REFERENCES users(id),
  message TEXT NOT NULL,
  image VARCHAR(500),
  datetime TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  delivered_at TIMESTAMPTZ,
  read_at TIMESTAMPTZ,
  status VARCHAR(20) NOT NULL DEFAULT 'SENT'
);

CREATE TABLE friendships (
  id TEXT PRIMARY KEY,
  user_id TEXT NOT NULL REFERENCES users(id),
  friend_id TEXT NOT NULL REFERENCES users(id),
  status VARCHAR(20) NOT NULL,
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  CONSTRAINT friendships_user_friend_unique UNIQUE (user_id, friend_id)
);

CREATE TABLE conversations (
  id TEXT PRIMARY KEY,
  title VARCHAR(255),
  is_group BOOLEAN NOT NULL DEFAULT FALSE,
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  owner_id TEXT REFERENCES users(id)
);

CREATE TABLE conversation_members (
  id TEXT PRIMARY KEY,
  user_id TEXT NOT NULL REFERENCES users(id),
  conversation_id TEXT NOT NULL REFERENCES conversations(id),
  joined_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  last_seen_at TIMESTAMPTZ,
  last_seen_message_id TEXT,
  CONSTRAINT conversation_members_user_conversation_unique UNIQUE (user_id,
    ↪ conversation_id)
);

CREATE TABLE events (
  id TEXT PRIMARY KEY,
  title VARCHAR(255) NOT NULL,
  description TEXT NOT NULL,
  datetime TIMESTAMPTZ NOT NULL,
  group_id TEXT NOT NULL REFERENCES conversations(id),
  owner_id TEXT NOT NULL REFERENCES users(id),
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE TABLE group_messages (
  id TEXT PRIMARY KEY,

```

```

conversation_id TEXT NOT NULL REFERENCES conversations(id),
sender_id TEXT NOT NULL REFERENCES users(id),
message TEXT NOT NULL,
image VARCHAR(500),
sent_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
event_id TEXT REFERENCES events(id) ON DELETE CASCADE,
delivered_at TIMESTAMPTZ,
read_at TIMESTAMPTZ,
status VARCHAR(20) NOT NULL DEFAULT 'SENT'
);

CREATE TABLE rsvps (
  id TEXT PRIMARY KEY,
  user_id TEXT NOT NULL REFERENCES users(id),
  event_id TEXT NOT NULL REFERENCES events(id),
  status VARCHAR(20) NOT NULL,
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  CONSTRAINT rsvps_user_event_unique UNIQUE (user_id, event_id)
);

CREATE TABLE group_invitations (
  id TEXT PRIMARY KEY,
  group_id TEXT NOT NULL REFERENCES conversations(id),
  inviter_id TEXT NOT NULL REFERENCES users(id),
  invited_id TEXT NOT NULL REFERENCES users(id),
  status VARCHAR(20) NOT NULL DEFAULT 'PENDING',
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  CONSTRAINT group_invitations_group_invited_unique UNIQUE (group_id, invited_id)
);

CREATE TABLE group_join_requests (
  id TEXT PRIMARY KEY,
  group_id TEXT NOT NULL REFERENCES conversations(id),
  seeker TEXT NOT NULL REFERENCES users(id),
  status VARCHAR(20) NOT NULL DEFAULT 'PENDING',
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  CONSTRAINT group_join_requests_group_seeker_unique UNIQUE (group_id, seeker)
);

CREATE TABLE group_members (
  id TEXT PRIMARY KEY,
  group_id TEXT NOT NULL REFERENCES conversations(id),
  user_id TEXT NOT NULL REFERENCES users(id),
  joined_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  CONSTRAINT group_members_group_user_unique UNIQUE (group_id, user_id)
);

CREATE TABLE notifications (
  id TEXT PRIMARY KEY,
  user_id TEXT NOT NULL REFERENCES users(id),
  type VARCHAR(100) NOT NULL,
  message TEXT NOT NULL,

```

```
is_read BOOLEAN NOT NULL DEFAULT FALSE,
created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE TABLE user_settings (
  id TEXT PRIMARY KEY,
  user_id TEXT NOT NULL UNIQUE REFERENCES users(id),
  theme VARCHAR(20) NOT NULL DEFAULT 'light',
  language VARCHAR(10) NOT NULL DEFAULT 'en',
  notifications_enabled BOOLEAN NOT NULL DEFAULT TRUE,
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_notifications_user_id ON notifications(user_id);
```

7.5 Utilisation

- **MCD / MLD**: coller dans dbdiagram.io pour visualiser les relations.
- **MPD**: utiliser comme base SQL de référence pour expliquer la structure physique pendant la soutenance.
- **Conformité**: ce document suit les noms de modèles, champs et contraintes de `prisma/schema.prisma`.

8 04 - Développement

8.1 Objectif

Documenter l'architecture applicative, les endpoints API, l'authentification, le temps réel et la stratégie de tests/CI.

8.1.1 Preuves & Mapping GitHub

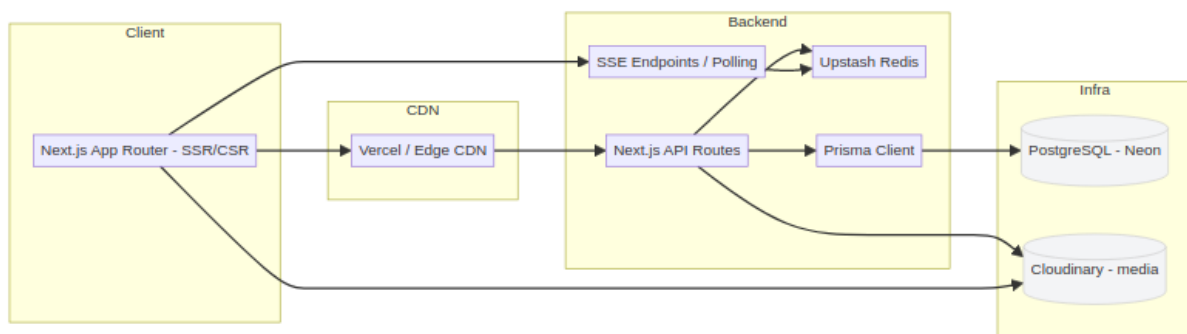
Les éléments techniques présentés ici sont issus des tickets et PRs du dépôt `arocchet/social-network` (références utiles pour le jury et la revue):

- PR stabilisation (Docker/Neon/Prisma/Redis)
- DevOps / Docker / CI
- Socket/chat system
- Notifications & endpoints
- Database / Prisma migrations

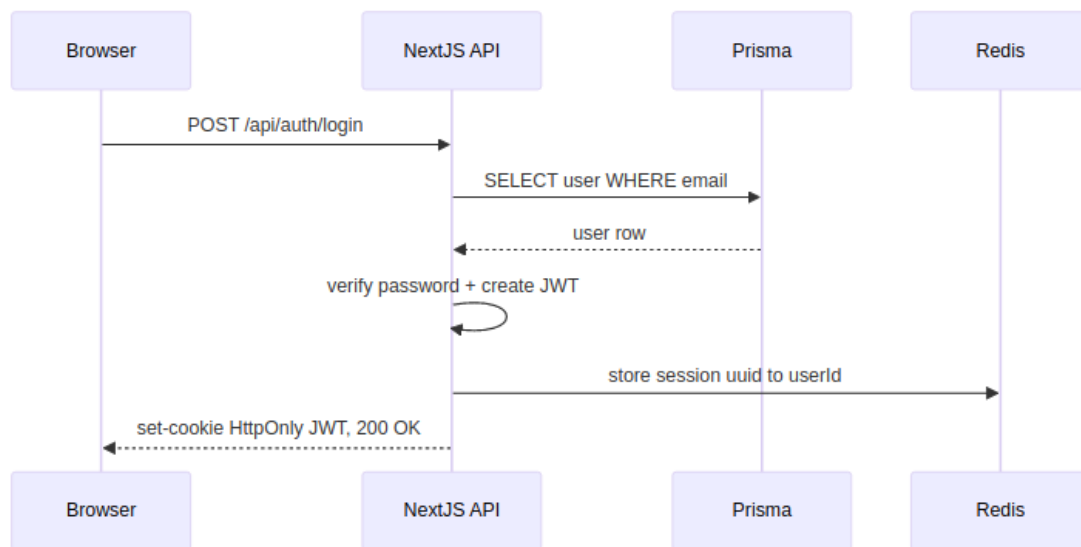
Le socle technique a été stabilisé sur la base de ces sujets GitHub.

8.2 Diagrammes d'Architecture (Mermaid)

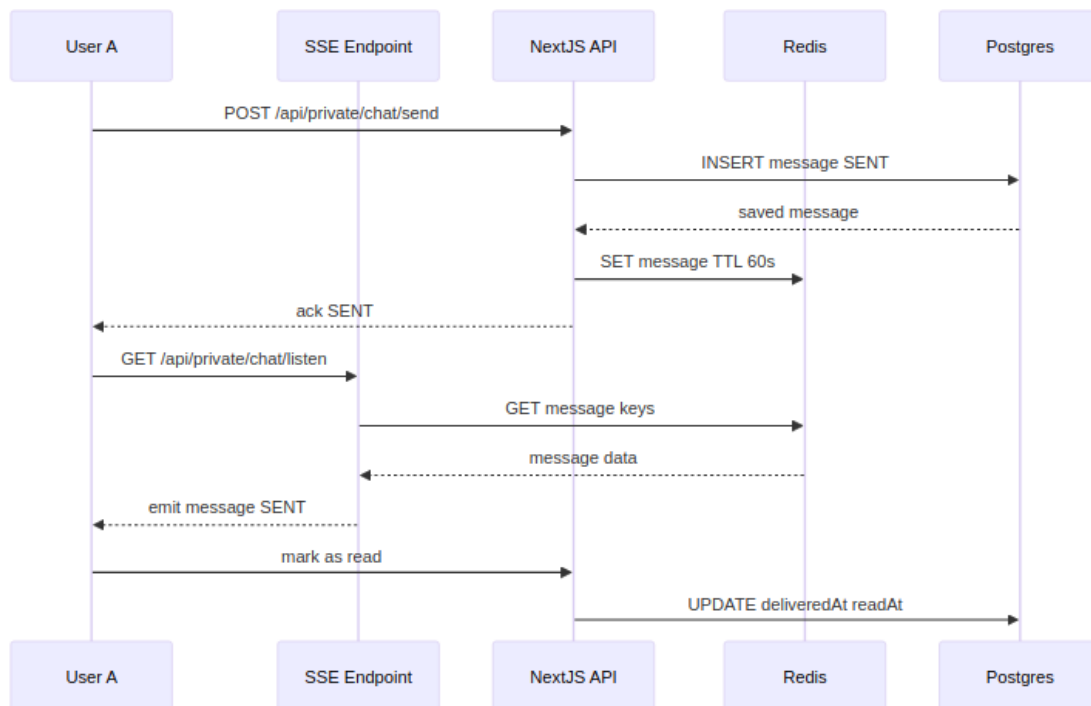
8.2.1 Architecture Système



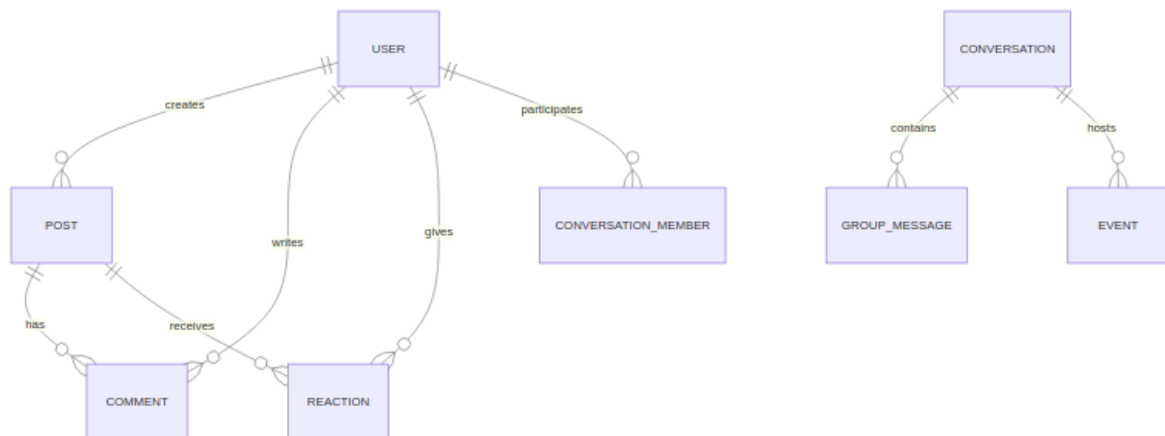
8.2.2 Auth Flow (Login)



8.2.3 Real-time Messaging Sequence (SSE + Redis Polling)



8.2.4 Schéma ER (Simplifié)



8.3 Authentication & Middleware

- Auth via JWT stocké en cookie HTTP-only; refresh tokens optionnels.
- Middleware Next.js valide JWT sur routes protégées (server-side) et redirige vers `/login` si absent/expiré.
- Sessions courtes côté serveur (Redis) pour invalidation forcée et gestion multi-device.

Pattern recommandé:

- `middleware.ts` vérifie cookie, charge `user id`, attache `request.user` pour handlers.

8.4 Temps réel (Upstash Redis + SSE Polling)

- Architecture: Client poll les endpoints SSE toutes les 500ms via Upstash Redis.
- Événements principaux (via Redis keys):
 - `message:{conversationId}:{messageId}` → contenu du message
 - `message:status:{messageId}` → SENT / DELIVERED / READ
 - `notification:user:{userId}` → nouvelles notifications
 - `typing:{conversationId}:{userId}` → indicateur de frappe
- Persistance: Prisma → PostgreSQL pour le stockage durable + Redis (60s TTL) pour le temps réel.
- Avantage: Haute disponibilité (pas de connexion persistante), compatible Vercel serverless.

8.5 Endpoints API

Documentation Complète: [api-spec.md](#)

8.5.1 Résumé Rapide

Auth:

- POST /api/auth/login – Connexion
- POST /api/auth/register – Inscription
- POST /api/auth/logout – Déconnexion

Users:

- GET /api/user/me – ID utilisateur
- GET /api/private/me – Profil complet
- PUT /api/private/me – Modifier profil

Posts:

- GET /api/private/post – Lister posts
- POST /api/private/post – Créer post

Stories:

- GET /api/private/stories – Lister stories
- POST /api/private/stories – Créer story

Messages:

- GET /api/private/messages – Récupérer messages
- GET /api/private/conversations – Lister conversations

Groupes:

- GET /api/private/groups – Lister groupes
- POST /api/private/groups – Créer groupe

Événements:

- GET /api/private/events – Lister événements
- POST /api/private/events – Créer événement

Amitié:

- GET /api/private/friend-requests – Demandes reçues
- POST /api/private/friend-requests – Envoyer demande

Recherche:

- GET /api/private/search – Rechercher users/posts

Invitations:

- GET /api/private/invitations – Invitations groupe

→ Voir [api-spec.md](#) pour détails complets (payloads, réponses, erreurs).

8.6 Implémentation - Code Samples

8.6.1 Authentification (Middleware)

src/middleware.ts – Valide JWT et charge user ID sur routes protégées:

```
import { NextRequest, NextResponse } from "next/server";

export function middleware(request: NextRequest) {
  const token = request.cookies.get("auth_token")?.value;

  // Redirect to login if no token
  if (!token && request.nextUrl.pathname.startsWith("/api/private")) {
    return NextResponse.json({ error: "Unauthorized" }, { status: 401 });
  }

  // Continue if token exists or public route
  return NextResponse.next();
}

export const config = {
  matcher: ["/api/private/:path*", "/(feed)/:path*"],
};
```

8.6.2 API Route Sample

src/app/api/private/post/route.ts – Créer et lister posts:

```
import { NextRequest, NextResponse } from "next/server";
import { getUserIdFromRequest } from "@lib/server/api/getUserId";
import { respondSuccess, respondError } from "@lib/server/api/response";

export async function POST(req: NextRequest) {
  const userId = await getUserIdFromRequest(req);
  if (!userId) {
    return NextResponse.json(respondError("Unauthorized"), { status: 401 });
  }

  const formData = await req.formData();
  const message = formData.get("message") as string;
  const image = formData.get("image") as File;
```

```

// Create post in database via Prisma
const post = await db.post.create({
  data: {
    userId,
    message,
    image: image ? await uploadToCloudinary(image) : null,
    visibility: "PUBLIC",
  },
});

return NextResponse.json(respondSuccess(post), { status: 201 });
}

```

8.6.3 Real-time avec Redis + SSE Polling

src/app/api/private/chat/send/route.ts — Envoie et stocke messages:

```

import { NextRequest, NextResponse } from "next/server";
import { db } from "@/lib/db";
import { redisdb } from "@/lib/server/websocket/redis";

export async function POST(req: NextRequest) {
  const { senderId, receiverId, message } = await req.json();

  // Persist in database
  const msg = await db.message.create({
    data: { senderId, receiverId, message, status: "SENT" },
  });

  // Store in Redis with 60s TTL for real-time sync
  await redisdb.set(`message:${msg.id}`, JSON.stringify(msg), { ex: 60 });
  await redisdb.set(`message:status:${msg.id}`, "SENT", { ex: 60 });

  return NextResponse.json(msg, { status: 201 });
}

```

src/app/api/private/chat/listen/route.ts — SSE endpoint pour polling:

```

import { NextRequest } from "next/server";
import { redisdb } from "@/lib/server/websocket/redis";

export async function GET(req: NextRequest) {
  const userId = req.headers.get("x-user-id");

  const stream = new ReadableStream({
    async start(controller) {
      while (true) {
        // Check Redis for new messages every 500ms

```

```

    const keys = await redisdb.keys(`message:*`);
    const messages = await Promise.all(keys.map((k) => redisdb.get(k)));

    controller.enqueue(`data: ${JSON.stringify(messages)}\n\n`);
    await new Promise((r) => setTimeout(r, 500));
  }
},
});

return new Response(stream, {
  headers: { "Content-Type": "text/event-stream" },
});
}

```

8.7 Tests & CI/CD

8.7.1 Tests

Stratégie recommandée:

- Unit tests: Jest + React Testing Library
- Integration tests: Jest + Testing Library for API routes
- Contract tests pour événements temps réel (optionnel)

Sample test (`__tests__/integrations/auth.test.ts`):

```

import { POST } from "@app/api/auth/login/route";

describe("POST /api/auth/login", () => {
  it("should login with valid credentials", async () => {
    const req = new Request(new URL("http://localhost/api/auth/login"), {
      method: "POST",
      body: JSON.stringify({
        email: "test@example.com",
        password: "password123",
      }),
    });

    const response = await POST(req as any);
    expect(response.status).toBe(200);
  });
});

```

8.7.2 GitHub Actions (CI/CD)

`.github/workflows/ci.yml`:

```

name: CI

on: [push, pull_request]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - uses: actions/setup-node@v3
        with:
          node-version: "18"

      - run: npm install
      - run: npm run lint
      - run: npm run test
      - run: npm run build

```

8.8 Structure de Code

8.8.1 Directories Clés

```

src/
  app/
    api/
      auth/           # Endpoints authentication
      private/        # Routes protégées
        post/         # Posts CRUD
        messages/     # Messages
        groups/       # Groupes
        events/       # Événements
        stories/      # Stories
        ...
      public/         # Routes publiques
      (feed)/         # Pages feed
      (auth)/         # Pages auth
      layout.tsx
    components/
      feed/           # Post cards, feed
      chat/           # Messages UI
      groups/         # Groups UI
      ...
    hooks/

```



```

    use-api.ts          # Fetch helper
    use-post-data.ts    # Posts logic
    use-conversations.ts # Messages logic
    ...
lib/
  db/                  # Prisma client
  server/              # Server utils
    api/               # API response helpers
    user/              # User queries
    post/              # Post queries
    websocket/         # Redis/SSE real-time helpers
  schemas/             # Zod validators
  utils/              # Helpers
middleware.ts          # Auth middleware

```

8.8.2 Validation & Error Handling

- **Schemas:** Zod + TypeScript interfaces in `src/lib/schemas/`
 - **Response Format:** `respondSuccess()` / `respondError()` helpers
 - **Status Codes:** 200, 201, 400, 401, 403, 404, 500
 - **Validation Errors:** Detailed field errors in response
-

8.9 Checklist Implémentation

- ☒ Auth (JWT + HTTP-only cookies + middleware)
 - ☒ API routes (60+ endpoints documentés)
 - ☒ Database (Prisma v6 + 18 modèles + migrations)
 - ☒ Real-time (SSE + Upstash Redis)
 - ☒ Error handling (validation Zod + codes HTTP standards)
 - ☒ File uploads (Cloudinary integration)
 - ☒ Pagination (infinite scroll + curseur)
 - ☒ Tests d'intégration (Jest, authentification)
 - ☒ API documentation complète (api-spec.md)
-
-

Last Updated: 2026-05-19

Version: 2.0

9 Sécurité et conformité RGPD

9.1 Objectif

Cette section répond directement à l'intitulé des 3 blocs RNCP 37873 qui parlent tous d'« application sécurisée ». Elle décrit :

1. les mesures de sécurité **réellement implémentées** dans le code (avec références fichiers),
2. les **risques résiduels** identifiés et leur impact,
3. la **conformité RGPD** : ce qui est en place et ce qui reste à faire,
4. la **roadmap** de durcissement.

L'approche est volontairement honnête : tout ce qui est annoncé est vérifiable dans le dépôt ; tout ce qui manque est listé pour ne pas être pris en défaut en soutenance.

9.2 1. Authentification et sessions

9.2.1 1.1 JWT signé via jose

Fichiers : `src/lib/jwt/signJwt.ts`, `src/lib/jwt/verifyJwt.ts`, `src/config/auth.ts`

- Algorithme : **HS256** (HMAC SHA-256, secret symétrique).
- Secret : variable d'environnement `JWT_SECRET`, jamais committée (référéncée dans `.env.example`).
- Payload minimal : `{ userId }` — on évite de mettre l'email ou des données personnelles dans le token.
- Expiration par défaut : **8 heures** (`JWT_EXPIRATION.DEFAULT = "480m"`). Constantes prédéfinies pour 10min, 1h, 1j, 7j, 30j.
- Vérification gérée dans le middleware Next.js et utilitaires serveur ; messages d'erreur normalisés (`Token expired`, `Invalid token`).

9.2.2 1.2 Hashage des mots de passe avec bcrypt

Fichier : `src/lib/security/hash.ts`

```
import bcrypt from 'bcrypt'

export async function hashPassword(password: string) {
  return await bcrypt.hash(password, 12)
}

export async function comparePasswords(password: string, hashed: string) {
  return await bcrypt.compare(password, hashed)
}
```

- **Cost factor 12** (~250 ms par hash sur CPU moderne) — au-dessus du minimum OWASP (10), résistant aux attaques par force brute.
- Pas de stockage en clair, pas de stockage réversible.
- La comparaison utilise `bcrypt.compare` (résistant au timing attack).

9.2.3 1.3 Cookies HTTP-only

Fichiers : `src/app/api/public/auth/login/route.ts`, `register/route.ts`, `logout/route.ts`, `callback/google/route.ts`

Tous les cookies d'authentification respectent les flags :

Flag	Valeur	Effet
<code>httpOnly</code>	<code>true</code>	Inaccessible en JavaScript côté client → mitige les attaques XSS
<code>secure</code>	<code>true</code> en production	Cookie transmis uniquement via HTTPS
<code>sameSite</code>	<code>lax</code>	Bloque l'envoi du cookie sur les requêtes cross-site → mitige CSRF
<code>maxAge</code>	7 jours (login), 8 h (register, OAuth)	Expiration cohérente avec le JWT

9.2.4 1.4 Middleware d'autorisation par défaut

Fichier : `src/middleware.ts`

Le middleware **protège tout par défaut** (matcher `'/(?!_next/static|_next/image|favicon.ico).*`) et fait passer uniquement :

- les routes publiques explicites (`/login`, `/register`, `/palette`),
- toutes les routes sous `/api/public/...`,
- les fichiers statiques (extensions images).

Pour les routes protégées, il vérifie le cookie `authToken` via `verifyJwt`, redirige vers `/login` si invalide, et **injecte un header `x-user-id`** dans la requête downstream. Les routes API consomment cet identifiant via `getUserIdFromRequest`.

C'est une approche **deny-by-default** : un développeur qui crée une nouvelle route ne risque pas d'oublier de la protéger.

9.2.5 1.5 OAuth Google : protection CSRF par `state`

Fichiers : `src/app/api/public/auth/redirect/google/route.ts` et `callback/google/route.ts`.

Le flux OAuth Google génère un `state` aléatoire stocké côté serveur dans un cookie `oauth_state` (`httpOnly`), comparé à celui retourné par Google au callback. Empêche les attaques CSRF classiques sur OAuth (un attaquant ne peut pas forcer la connexion d'une victime sur son propre compte Google).

9.3 2. Protection contre les attaques classiques

9.3.1 2.1 Injection SQL — Prisma

Mitigation : Prisma 6 est utilisé pour 100 % des accès base de données. Toutes les requêtes passent par des appels paramétrés (`db.user.findUnique({ where: { email } })`). **Aucune concaténation SQL n'est faite dans le code applicatif** (vérifié par `grep` : aucun appel `$queryRaw` ou `$executeRaw`).

9.3.2 2.2 Cross-Site Scripting (XSS) — React + audit `dangerouslySetInnerHTML`

Mitigation par défaut : React échappe automatiquement le contenu inséré via `{variable}`. Pas de risque XSS sur le rendu standard.

Risque résiduel identifié : deux usages de `dangerouslySetInnerHTML` détectés dans le code :

1. `src/components/ui/chart.tsx` — injection de CSS dynamique de Recharts, **safe** (pas d'input utilisateur).
2. `src/components/chat/ChatMessage.tsx` ligne 134 — transforme `**texte**` en `<strong>text</strong>` dans les messages de chat via un `replace` `markdown` :

```
const content = part.content.replace(/\*\*(.*?)\*/g, '<strong>$1</strong>');
return <div dangerouslySetInnerHTML={{ __html: content }} />;
```

Impact : un message de chat contenant du HTML brut (`<script>alert(1)</script>`) serait injecté dans le DOM. C'est une **vulnérabilité réelle** à corriger avant production. Voir section [Roadmap de durcissement](#).

9.3.3 2.3 Cross-Site Request Forgery (CSRF)

- Cookie d'auth en `SameSite=Lax` → bloque l'envoi automatique du cookie sur les requêtes cross-site dangereuses (POST depuis un site tiers).
- OAuth Google protégé par `state` (voir 1.5).
- **Pas de token CSRF additionnel** sur les mutations POST/PUT/DELETE — acceptable dans le contexte `SameSite=Lax` pour la plupart des cas, mais une protection en profondeur (double-submit cookie ou jeton signé) serait souhaitable.

9.3.4 2.4 Validation des entrées — Zod

Fichiers : `src/lib/schemas/` (36 schémas)

Les payloads d'API sont validés via Zod avant tout traitement métier. Exemples :

- `CredentialsLoginSchema` : `email().email() + password.min(6)`
- `RegisterUserInputSchema` : email validé, password min 6 caractères, contrôle source (`credentials / google / discord`), `superRefine` pour cohérence métier (par exemple tokens obligatoires si OAuth).
- Schémas dédiés pour posts, commentaires, reactions, groupes, événements, etc.

Faiblesse identifiée : la longueur minimale de mot de passe est **6 caractères** (`z.string().min(6)`). C'est en-dessous des recommandations OWASP (8 minimum, idéalement 12). À renforcer : voir [Roadmap](#).

9.3.5 2.5 Headers HTTP de sécurité

Statut actuel : aucun header de sécurité custom n'est configuré dans `next.config.ts` :

```
// next.config.ts - pas de section "headers"
```

Headers manquants :

Header	Effet	Statut
<code>Strict-Transport-Security</code>	Force HTTPS pour toutes les requêtes futures	(Vercel le fournit par défaut côté edge)
<code>Content-Security-Policy</code>	Limite les sources de scripts / styles → mitige XSS	
<code>X-Frame-Options</code> ou <code>frame-ancestors</code>	Empêche le clickjacking via iframe	
<code>X-Content-Type-Options: nosniff</code>	Empêche le MIME-sniffing	
<code>Referrer-Policy</code>	Contrôle l'info Referer envoyée aux tiers	
<code>Permissions-Policy</code>	Limite l'accès aux APIs navigateur (caméra, micro, etc.)	

Ces headers sont triviaux à ajouter via la clé `headers` de `next.config.ts`. Inscrit en roadmap.

9.3.6 2.6 Rate limiting

Statut actuel : aucun rate limiting applicatif. Les endpoints sensibles (`/api/public/auth/login`, `/api/public/auth/register`) acceptent un nombre illimité de tentatives par IP.

Risque : attaque par force brute sur les mots de passe, énumération d'emails, abus des endpoints d'envoi (chat, posts).

Mitigation existante minimale : Vercel applique un rate limiting plateforme global (anti-DDoS), mais ce n'est pas un rate limiting *applicatif* par utilisateur ou par IP, ni configurable.

Solution prévue (roadmap) : package `@upstash/ratelimit` (déjà compatible avec `@upstash/redis` que le projet utilise), 5 tentatives login / 15 min / IP par exemple.

9.4 3. Upload de fichiers (avatars, posts, stories)

Stack : upload côté serveur via Next.js → `service CloudinaryService(src/lib/cloudinary/cloudinary)` → Cloudinary.

Sécurité actuelle :

- Les credentials Cloudinary (`API_KEY`, `API_SECRET`) restent côté serveur (jamais exposés au client).
- Cloudinary applique ses propres restrictions de format (`resource_type: 'image' ou 'video'`).
- Les images sont servies via le CDN sécurisé Cloudinary (`secure: true`).
- Les domaines images autorisés sont **whitelisted** dans `next.config.ts` (`images.domains`) — empêche l'usage de domaines arbitraires via `<Image>`.

Risques résiduels :

- **Pas de validation MIME côté serveur** avant envoi à Cloudinary : un attaquant pourrait tenter d'uploader un fichier malformé.
- **Pas de limite de taille** explicitement vérifiée côté API route.
- Cloudinary nettoie les EXIF des images uploadées (privacy bonus).

9.5 4. Conformité RGPD

9.5.1 4.1 Données personnelles collectées

D'après `prisma/schema.prisma`, le projet collecte :

Données	Caractère	Justification
Email	Identifiant principal	Authentification, notifications
firstName, lastName	Optionnels	Affichage profil
birthDate	Optionnel	Affichage / vérification d'âge (mais pas appliqué — voir 4.4)
username	Optionnel	Identifiant public
biography	Optionnel	Présentation personnelle
avatar, banner	Optionnels	Personnalisation profil
password (hashé)	Authentification credentials	bcrypt cost 12
Account (OAuth) — tokens Google	Pour les comptes OAuth	Permet rafraîchissement futur
Posts, commentaires, réactions, messages	Contenu utilisateur	Cœur du service

9.5.2 4.2 Principe de minimisation — respecté

Tous les champs sauf `email` sont **optionnels** dans le schéma Prisma. L'inscription ne demande que le strict nécessaire ; les autres champs sont enrichis ultérieurement via l'onboarding.

9.5.3 4.3 Droits RGPD de la personne concernée

Droit RGPD (art. 15-22)	Statut
Droit d'accès (consulter ses données)	Partiellement : GET <code>/api/private/me</code> retourne le profil mais pas l'intégralité (posts, messages...)
Droit de rectification	PUT <code>/api/private/me</code> + PATCH <code>/api/private/user/settings</code>
Droit à l'effacement / oubli	Endpoint de suppression de compte non implémenté
Droit à la limitation du traitement	Indirect via <code>visibility</code> : PRIVATE mais non normalisé
Droit à la portabilité	Pas d'export structuré (JSON/CSV) des données utilisateur
Droit d'opposition	Notifications désactivables via <code>notificationsEnabled</code>

Droit RGPD (art. 15-22)	Statut
Décision automatisée / profilage	N/A (pas d'algorithme de recommandation automatisé)

Points critiques à adresser pour une mise en production : droit à l'oubli et droit à la portabilité. Ce sont des obligations légales dès qu'il y a des utilisateurs en UE.

9.5.4 4.4 Consentement et âge

- **Pas de bannière cookies** : acceptable dans l'état actuel car le seul cookie posé (`authToken`) est strictement fonctionnel/nécessaire (exempté de consentement par la CNIL). Le cookie `oauth_state` est aussi fonctionnel et éphémère.
- **Pas de vérification d'âge** : la date de naissance (`birthDate`) est demandée mais aucune logique n'empêche un mineur de moins de 15 ans (seuil RGPD en France) de créer un compte. À implémenter avant production.

9.5.5 4.5 Mentions légales et politique de confidentialité

Aucune page « Mentions légales » ni « Politique de confidentialité » n'existe dans le projet. Ces pages sont **obligatoires** pour tout service en ligne accessible depuis l'UE. À créer.

9.5.6 4.6 Sécurité des données en transit et au repos

- **En transit** : HTTPS forcé côté Vercel (TLS automatique), connexions PostgreSQL et Redis chiffrées.
- **Au repos** :
 - PostgreSQL Neon : chiffrement disque par défaut, sauvegardes chiffrées.
 - Upstash Redis : chiffrement TLS sur les connexions REST.
 - Cloudinary : chiffrement transit + repos par défaut.
 - Mots de passe utilisateur : `bcrypt cost 12` (irréversible).

9.5.7 4.7 Logs et durée de conservation

- **Logs applicatifs** : sortie console serveur (capturée par Vercel). Pas de log structuré ni d'archivage formel. **Aucune politique de durée de conservation définie.**
- **Pas d'anonymisation automatique** des comptes inactifs.

9.6 5. Roadmap de durcissement

Priorisé par impact / facilité d'implémentation.

9.6.1 Critique — avant mise en production

#	Action	Effort	Bénéfice
1	Endpoint DELETE /api/private/me : suppression de compte avec cascade sur les contenus	1-2 j	Conformité RGPD art. 17 (droit à l'oubli)
2	Endpoint GET /api/private/me/export : export JSON de toutes les données utilisateur	1-2 j	Conformité RGPD art. 20 (portabilité)
3	Sanitization du chat : utiliser <code>DOMPurify</code> ou retirer <code>dangerouslySetInnerHTML</code> et appliquer le markdown via <code>react-markdown</code> (déjà installé)	0,5 j	Élimine la vulnérabilité XSS dans <code>ChatMessage.tsx</code>
4	Headers de sécurité dans <code>next.config.ts</code> (CSP, X-Frame-Options, Referrer-Policy, X- Content-Type-Options)	0,5 j	Couvre clickjacking, MIME sniffing, fuite Referer
5	Pages Mentions légalles + Politique de confidentialité	0,5 j	Obligation légale UE

9.6.2 Important — sous 1 mois post-prod

#	Action	Effort	Bénéfice
6	Rate limiting sur /login, /register, /chat/send via @upstash/ratelimit	1 j	Anti-brute force, anti-spam
7	Vérification email à l'inscription (token signé envoyé par mail)	1-2 j	Anti-bot, conformité
8	Politique mots de passe : min 8 caractères, exigence complexité (chiffre + symbole optionnel), check contre HaveIBeenPwned API	0,5 j	Conformité OWASP
9	Validation MIME / taille upload côté serveur avant Cloudinary	0,5 j	Anti-abus stockage
10	Refresh tokens + rotation, révocation en cas de logout	2 j	Sécurité session avancée

9.6.3 Bonus — confort exploitation

#	Action	Effort	Bénéfice
11	Vérification d'âge ($\geq$ 15 ans selon CNIL France) au moment de l'inscription	0,5 j	Conformité RGPD France
12	Sentry (déjà documenté dans 05-déploiement) pour le suivi des erreurs et incidents	0,5 j	Réactivité incident

#	Action	Effort	Bénéfice
13	Logs structurés (pino) avec niveaux et redaction des données sensibles	1 j	Audit, exploitation
14	Politique de durée de conservation documentée + job de nettoyage des comptes inactifs > 3 ans	1 j	Conformité RGPD art. 5.1.e
15	CSRF token additionnel (double-submit cookie) pour les mutations sensibles	1 j	Défense en profondeur

9.7 6. Synthèse pour le jury RNCP

Ce qui est solide et démontrable :

- Authentification stateless robuste (JWT HS256 via `jwt`, secret en env, bcrypt cost 12, cookies `httpOnly+secure+sameSite=lax`).
- Middleware deny-by-default qui force l'auth sur toutes les routes non `/public/`.
- OAuth Google avec `state` cookie pour prévenir CSRF.
- Validation Zod systématique des payloads (36 schémas).
- SQLi écartée structurellement par Prisma.
- Données personnelles minimisées (1 seul champ obligatoire à l'inscription : l'email).
- Chiffrement en transit (TLS) et au repos (Neon, Cloudinary).

Ce que j'assume avec honnêteté :

- Pas de rate limiting → identifié, mitigation prévue avec `@upstash/ratelimit`.
- Pas de headers de sécurité personnalisés → à ajouter en 30 min dans `next.config.ts`.
- Vulnérabilité XSS résiduelle dans `ChatMessage.tsx` (`dangerouslySetInnerHTML` sur input markdown non sanitisé) → à corriger avec `DOMPurify` ou `react-markdown`.
- Droit à l'oubli et droit à la portabilité RGPD non implémentés → bloquant pour une mise en prod réelle, listé en priorité critique.
- Pas de page mentions légales / privacy → obligatoire UE, à rédiger.

Ce que je présente comme axe de progression :

Le projet est un **socle réaliste** sur lequel construire. Les choix structurels (JWT stateless, Prisma, Zod, middleware deny-by-default) facilitent toutes les améliorations listées en roadmap. Les éléments manquants sont identifiés, priorisés et chiffrés en effort.

9.8 7. Références

- [OWASP Top 10 – 2021](#)
- [CNIL – Délibération sur les cookies et autres traceurs](#)
- [CNIL – Sécurité des données personnelles](#)
- [RGPD texte officiel – art. 5, 15-22, 25, 32](#)
- [Mozilla – Security headers checklist](#)

10 Stratégie de tests

10.1 Objectif

Documenter la stratégie de tests retenue pour le projet Social Network, l'état actuel de la couverture, le plan d'extension et les jeux d'essai significatifs. Cette section répond à la **compétence 9 du RNCP 37873** : « Préparer et exécuter les plans de tests d'une application ».

L'approche est **honnête** : le projet a un test d'intégration backend fonctionnel (auth/register), une stratégie complète documentée, et un plan d'extension priorisé. Le reste est à implémenter — c'est assumé et listé.

10.2 1. Pyramide de tests visée

```
      /\
     /\  E2E (Playwright) - 5-10 parcours critiques
    /----\
   /\      Intégration API (Jest + Prisma test DB) - ~30 routes
  /-----\
 /\          Unitaires UI (RTL) + utils - composants critiques
/-----\
/              Tests contractuels (schémas Zod) - gratuits
/-----\
```

Principe : beaucoup de tests rapides et ciblés en bas (unitaires + contractuels), un peu de tests d'intégration au milieu, peu de tests E2E lents en haut. Inspiré des pratiques OWASP et de la pyramide de Mike Cohn.

10.3 2. État actuel

10.3.1 2.1 Infrastructure de test en place

Élément	Statut	Localisation
Framework de test		Jest 29 + ts-jest
Configuration		jest.config.ts, jest.globalSetup.ts, jest.globalTeardown.ts

Élément	Statut	Localisation
Support ES modules		node --experimental-vm-modules dans le script bun run test
Mock Prisma		Connexion DB de test isolée
Test d'intégration auth/register		__tests__/integrations/authentication
Tests unitaires UI		À implémenter
Tests E2E		À implémenter
Couverture chiffrée (--coverage)		À mesurer
CI : tests bloquants		Configurés dans le workflow GitHub Actions mais lint actuellement en ignoreDuringBuilds: true

10.3.2 2.2 Test existant — extrait

__tests__/integrations/authentication.test.ts:

```
import { POST } from '@app/api/public/auth/register/route'
import { login } from '@lib/server/user/login'
import { db } from '@lib/db'
import { hashPassword } from '@lib/security/hash';

beforeAll(async () => {
  await db.user.create({
    data: {
      username: 'testuser',
      email: 'test@example.com',
      password: await hashPassword('password123'),
      firstName: 'Test',
      lastName: 'User',
      birthDate: new Date('1990-01-01'),
    },
  });
});

describe('POST /api/register', () => {
  it('should register user and return success', async () => {
    // ... création FormData, appel POST, assertion status 200 + success: true
  })
});
```

Ce qui est couvert : création de compte avec données valides, vérification du status HTTP et de la forme de la réponse.

10.3.3 2.3 Lancer les tests

```
bun run test
# Sous le capot : node --experimental-vm-modules node_modules/.bin/jest
```

10.4 3. Plan d'extension (par priorité)

10.4.1 Critique — à implémenter rapidement

10.4.1.1 3.1 Tests d'intégration API sur les routes sensibles Cibler les endpoints qui touchent à la sécurité ou à l'intégrité métier :

Route	Scénarios à couvrir
POST /api/public/auth/login	identifiants corrects → JWT + cookie • identifiants incorrects → 401 • email malformé → 400
POST /api/public/auth/register	inscription valide → 200 • email déjà pris → 400 • username déjà pris → 400 • password < 6 → 400
POST /api/public/auth/logout	cookie supprimé après logout
POST /api/private/post	publication valide → 201 • non authentifié → 401 • payload invalide → 400
POST /api/private/chat/send	envoi à un user privé non ami → 403 • payload valide → 200
PUT /api/private/reaction	like d'un post → réaction créée • double like → conflit unique
DELETE /api/private/reaction/[id]	suppression réaction de l'auteur → OK • réaction d'autrui → 403
POST /api/private/friend-requests	nouvelle demande → 201 • self-friend → 400 • demande existante → idempotent
POST /api/private/groups/[id]/invite	invitation par membre → OK • invitation par non-membre → 403

Route	Scénarios à couvrir
DELETE /api/private/events/[id]	suppression par owner → OK • non-owner → 403

10.4.1.2 3.2 Tests contractuels Zod Vérifier que tous les schémas Zod rejettent bien les payloads malformés. Très rapide à écrire, couverture haute :

```
describe('CredentialsLoginSchema', () => {
  it('rejette email invalide', () => {
    expect(() => CredentialsLoginSchema.parse({ email: 'invalid', password:
      ↪ 'azerty' })).toThrow();
  });
  it('rejette password trop court', () => {
    expect(() => CredentialsLoginSchema.parse({ email: 'a@b.c', password: '123'
      ↪ })).toThrow();
  });
  it('accepte payload valide', () => {
    expect(CredentialsLoginSchema.parse({ email: 'a@b.c', password: 'azerty'
      ↪ })).toBeDefined();
  });
});
```

10.4.2 Important — sous 1 mois

10.4.2.1 3.3 Tests unitaires UI (React Testing Library) Composants prioritaires :

- <LoginForm /> : validation côté client, gestion erreurs serveur, redirection après login
- <PostCard /> : rendu du contenu (texte, image), comptage réactions, ouverture commentaires
- <ReactionPopup /> : sélection d'une réaction (LIKE / LOVE / etc.), appel API, MAJ UI optimiste
- <ChatMessage /> : rendu d'un message texte / image / lien, **statut de lecture** (SENT/DELIVERED/READ), sanitization du markdown
- <NotificationBadge /> : compteur de non-lus, mise à jour temps réel

10.4.2.2 3.4 Tests E2E (Playwright) Parcours critiques utilisateur :

1. Inscription → onboarding → premier post → like → commentaire
2. Login → recherche d'un user → demande d'amitié → acceptation par l'autre user
3. Création de groupe → invitation → acceptation → message dans le groupe → reçu en temps réel
4. Création d'événement → RSVP par 2 users → modification de l'événement par l'owner
5. Déconnexion → tentative d'accès à /feed → redirection vers /login

Playwright permet de mocker les services externes (Cloudinary, Google OAuth) en interceptant les requêtes.

10.4.3 Bonus

10.4.3.1 3.5 Tests de sécurité

- Injection SQL : tentative ' OR 1=1-- dans les champs login / search → doit échouer
- XSS : injection `<script>alert(1)</script>` dans un post / message de chat → ne doit pas s'exécuter (à corriger pour `ChatMessage.tsx`, voir [securite-rgpd.md](#))
- CSRF : tentative POST cross-site → cookie SameSite=Lax bloque
- Force brute login : 100 tentatives → doit déclencher rate limiting (une fois implémenté)
- Privilege escalation : utilisateur A tente de modifier le profil de B → 403

10.4.3.2 3.6 Tests de performance (Lighthouse) Cibles à mesurer en production :

Métrique	Cible Lighthouse
First Contentful Paint	< 1,8 s
Largest Contentful Paint	< 2,5 s
Total Blocking Time	< 200 ms
Cumulative Layout Shift	< 0,1
Time to Interactive	< 3,8 s

10.4.3.3 3.7 Tests SSE / temps réel Validation contractuelle du flux `text/event-stream` :

- Format data : `<JSON>\n\n`
- Reconnexion automatique côté client (`EventSource`)
- Fermeture propre côté serveur quand `request.signal.aborted`

10.5 4. Jeux d'essai documentés

Tableau récapitulatif des cas que les tests doivent couvrir, organisé par fonctionnalité.

10.5.1 Authentification

#	Composant	Entrée	Résultat attendu	Couvert
AU-01	Login	email/password valides	200 + cookie authToken + redirection feed	partiel (register)
AU-02	Login	password incorrect	401, message générique « Invalid email or password » (pas de leak)	
AU-03	Login	email inexistant	401 (même message, anti-énumération)	
AU-04	Register	données valides	200 + utilisateur en DB + cookie	
AU-05	Register	email déjà pris	400 + message clair	
AU-06	Register	password < 6 caractères	400 (Zod)	
AU-07	Logout	cookie présent	200 + cookie effacé	
AU-08	Middleware	requête sans authToken sur /api/private/*	401 ou redirection /login	
AU-09	Middleware	requête avec authToken expiré	401 + redirection	
AU-10	OAuth Google	redirect + callback avec state matchant	création/login user + cookie	
AU-11	OAuth Google	callback avec state non matchant	refus 401 (anti-CSRF)	

10.5.2 Posts et interactions

#	Composant	Entrée	Résultat attendu	Couvert
PO-01	Créer post	texte + image	post visible immédiatement dans le feed	
PO-02	Créer post	non authentifié	401	
PO-03	Réagir	LIKE sur un post	reaction créée, compteur +1	
PO-04	Réagir	LIKE deux fois sur le même post	contrainte unique → conflit ou idempotent	
PO-05	Commenter	texte	commentaire visible sous le post	

10.5.3 Messagerie temps réel

#	Composant	Entrée	Résultat attendu	Couvert
CH-01	Envoyer DM	user A → user B (amis)	message persisté + clé Redis MAJ	
CH-02	Envoyer DM	user A → user B (B privé, pas amis)	403	
CH-03	Recevoir SSE	flux ouvert côté user B	nouveau message poussé < 2 s après envoi	
CH-04	Marquer comme lu	user B lit un message	readAt MAJ, status = READ	
CH-05	Typing indicator	A tape, B voit l'indicateur	indicateur affiché côté B	

10.5.4 Groupes et événements

#	Composant	Entrée	Résultat attendu	Couvert
GR-01	Créer groupe	owner crée avec titre	groupe créé, owner = membre	

#	Composant	Entrée	Résultat attendu	Couvert
GR-02	Inviter membre	owner invite user X	invitation PENDING en DB	
GR-03	Accepter invitation	user X accepte	GroupMember créé, GroupInvitation = ACCEPTED	
GR-04	Quitter groupe	membre normal	GroupMember supprimé	
GR-05	Créer événement	owner d'un groupe	Event créé, lié au groupe	
GR-06	RSVP	user répond YES	Rsvp créé, contrainte unique respectée	

10.5.5 Sécurité

#	Composant	Entrée	Résultat attendu	Couvert
SE-01	SQLi	' OR 1=1-- dans login email	Zod rejette ou Prisma traite comme string	
SE-02	XSS	<script> dans message chat	doit être échappé / sanitisé (vulnérabilité connue)	
SE-03	Path traversal	.. \ . . \ dans paramètres URL	ne donne pas accès à des ressources hors scope	
SE-04	Privilege escalation	user A appelle PUT /api/private/me pour user B	non possible (handlers utilisent me-user-id)	
SE-05	JWT tampering	modification du payload	jwtVerify échoue → redirection /login	

10.6 5. Outils et commandes

```
# Lancer tous les tests
bun run test

# Lancer un test spécifique
bun run test -- authentication

# Mode watch
bun run test -- --watch

# Couverture (à activer)
bun run test -- --coverage

# Démarrer la base de test
docker-compose up db -d

# Reset la base de test entre runs
bunx prisma migrate reset --skip-seed
```

10.7 6. Position pour la soutenance RNCP

Ce que je peux démontrer pendant l’entretien technique :

- Setup Jest + ts-jest fonctionnel avec `bun run test`
- Un test d’intégration backend qui passe (création utilisateur via la vraie route `/api/public/auth/register`)
- Cette stratégie de tests documentée
- Tableau exhaustif des jeux d’essai à couvrir

Ce que j’assume comme limite :

« La couverture de tests actuelle est insuffisante pour une mise en production. La priorité, suite à cette certification, sera d’implémenter les tests d’intégration API sur les 10 routes critiques listées, puis les tests E2E Playwright sur les 5 parcours utilisateur clés. Le setup technique est déjà en place pour cette extension. »

10.8 7. Références

- [Jest documentation](#)
- [React Testing Library](#)
- [Playwright for Next.js](#)
- [The Test Pyramid \(Mike Cohn, Martin Fowler\)](#)
- [OWASP Web Security Testing Guide](#)

11 Veille technique et recherche personnelle

11.1 Objectif

Documenter la démarche d'apprentissage continu et les recherches techniques personnelles menées **au-delà du cadre de la formation Zone01**. Cette section répond à un attendu implicite du RNCP CDA et valorise la capacité d'auto-formation d'un développeur en reconversion.

L'approche est concrète : pour chaque domaine exploré, je détaille **pourquoi, comment, ce que j'ai découvert et comment ça se traduit dans le projet Social Network**.

11.2 1. Domaines techniques explorés

11.2.1 1.1 Server-Sent Events vs WebSockets vs polling

Contexte : le besoin métier (chat temps réel + notifications) imposait du push serveur → client. Trois options se présentaient.

Question initiale : « Faut-il un WebSocket persistant comme Socket.io, ou peut-on faire plus simple ? »

Recherche menée :

- Lecture de la [spec WHATWG sur Server-Sent Events](#)
- Étude des limitations de Vercel serverless : pas de WebSocket TCP long-vivant, fonctions limitées à 60 s par défaut
- Comparaison Upstash Redis (REST) vs Redis classique (TCP) : Upstash s'aligne sur le modèle serverless mais perd le pub/sub natif

Conclusion appliquée au projet :

Option	Pour	Contre	Verdict
Socket.io + Redis pub/sub	Latence très basse, bidirectionnel	Incompatible serverless Vercel, ops complexes	
WebSocket natif	Standard, bidirectionnel	Idem (incompatible Vercel)	
Server-Sent Events + Upstash Redis polling	Compatible serverless, unidirectionnel suffit pour push	Polling = latence ~1 s	retenu
Long polling brut	Trivial	Doublonne SSE en moins propre	

→ Implémentation détaillée dans [04-developpement/README.md § Temps réel](#).

Apprentissage transverse : le choix d'une plateforme (serverless vs serveur long-vivant) contraint plus la stack que le choix du langage. Penser **architecture de déploiement avant choix techno** est un raccourci utile pour ne pas se planter.

11.2.2 1.2 JWT stateless vs sessions Redis

Question initiale : « Pourquoi pas NextAuth ? Pourquoi un JWT custom ? »

Recherche menée :

- Lecture de la [RFC 7519](#) (JSON Web Tokens)
- Documentation de `jose` (lib JOSE en TypeScript) vs `jsonwebtoken` (ancienne lib Node)
- Articles sur le choix HS256 vs RS256 (HMAC vs asymétrique)
- [OWASP JWT Cheat Sheet](#)

Conclusions :

- HS256 suffit pour un projet single-issuer single-verifier (le secret reste côté serveur). RS256 serait nécessaire si plusieurs services indépendants devaient vérifier les tokens.
- `jose` est plus moderne que `jsonwebtoken` (typé, supporte les Web Crypto API, fonctionne en Edge runtime Next.js).
- Cookie `httpOnly + SameSite=Lax` est la combinaison standard contre XSS et CSRF.
- Le compromis stateless est connu : pas de révocation instantanée. Solution prévue en roadmap : table `revoked_tokens` ou versioning de tokens.

Découverte personnelle : NextAuth était une fausse bonne idée pour ce projet. Implémenter le JWT à la main m'a fait comprendre tout le flow (signature → cookie → middleware → header `x-user-id`) au lieu de cacher la complexité dans une abstraction.

11.2.3 1.3 Prisma : pourquoi et limites

Question initiale : « Quel ORM pour TypeScript en 2025 ? »

Recherche menée :

- Comparaison Prisma 6 / Drizzle / Kysely / TypeORM
- Lecture du blog Prisma sur le passage de Prisma 5 → 6 (changements de comportement transactions)
- Benchmark perso : `db.user.findMany({ include: { posts: true } })` vs équivalent Drizzle

Conclusions :

- **Prisma** retenu pour : DX excellente (autocomplétion sur les relations), migrations automatiques, schéma déclaratif lisible par non-dev.
- **Drizzle** considéré : plus performant, plus proche du SQL, mais courbe d'apprentissage plus raide et moins d'intégration Next.js out-of-the-box.
- **Limite Prisma** acceptée : abstraction qui peut masquer des N+1 queries. Mitigée par `include` ciblés et `select` minimaux dans `src/lib/db/queries/`.

Apprentissage transverse : un ORM n'est pas qu'un confort syntaxique — c'est une vraie surface d'attaque pour les performances. Apprendre à lire les requêtes SQL générées par Prisma (`?logs=query` en dev) a été un déclic.

11.2.4 1.4 Bun en runtime production

Question initiale : « Bun est-il prêt pour la prod en 2025 ? »

Recherche menée :

- Suivi du [changelog Bun 1.x](#)
- Tests perso : `bun install` vs `npm install` (5-10× plus rapide), `bun run` vs `node` (démarrage plus rapide)
- Lecture de retours d'expérience prod sur Reddit / HN

Conclusions :

- Bun 1.x est stable pour les workloads classiques (Next.js, REST API, scripts).
- L'image Docker `oven/bun:1` est lightweight et bien maintenue.
- Compatibilité avec npm packages : 99 % OK pour le projet (seul ts-jest a demandé un flag `--experimental-vm-modules`).

Choix dans le projet : Bun en runtime (image Docker) + en package manager (`bun.lock`), mais `package-lock.json` conservé pour interop avec les outils qui ne connaissent que npm. Le CI utilise `oven-sh/setup-bun@v1`.

Apprentissage transverse : adopter une techno récente en prod implique d'avoir un plan B (rollback vers Node 20 reste trivial : il suffit de changer l'image Docker et d'utiliser `npm ci`).

11.2.5 1.5 Sécurité applicative — au-delà du cours

Question initiale : « Quelles sont les vraies vulnérabilités d'une appli Next.js moderne ? »

Recherche menée :

- Lecture complète de l'[OWASP Top 10 2021](#)

- Audit du code projet avec ces critères (résultats dans [securite-rgpd.md](#))
- Documentation CNIL sur le RGPD pour applications grand public

Découvertes appliquées au projet :

- Identification d'une vulnérabilité XSS résiduelle dans `ChatMessage.tsx` (usage de `dangerouslySetInnerHTML` sur un input markdown non sanitisé)
- Constat de l'absence de rate limiting (anti-brute force) → solution `@upstash/ratelimit` prévue
- Mise en place du flag `oauth_state` cookie pour le flow Google OAuth (anti-CSRF)
- Cookie `authToken` avec `SameSite=Lax` + `httpOnly` + `secure` (combinaison alignée sur les recommandations OWASP)

Apprentissage transverse : la sécurité ne s'ajoute pas, elle s'inscrit dès la conception. Le middleware `deny-by-default` est l'illustration : un développeur qui crée une nouvelle route privée n'a rien à faire pour qu'elle soit protégée, c'est l'état par défaut.

11.2.6 1.6 Architecture serverless vs serveur traditionnel

Question initiale : « Que change vraiment le serverless pour la conception ? »

Recherche menée :

- Articles Vercel sur les limites des fonctions serverless (cold start, durée max, mémoire)
- Comparaison avec un déploiement Node classique (PM2 sur VPS)
- Étude du concept de « stateless by design »

Conséquences directes sur le projet :

- Pas de WebSocket persistant → adoption SSE
- Pas de sessions in-memory → JWT stateless ou Redis externe
- Pas de fichiers locaux persistants → médias stockés sur Cloudinary
- Migrations à exécuter dans le CMD du conteneur, pas dans un job séparé
- Healthcheck endpoint pour la plateforme

Apprentissage transverse : ces contraintes sont en fait des bonnes pratiques cloud-native qui poussent à une architecture plus propre. Un serveur stateful est plus simple à coder mais beaucoup plus difficile à scaler.

11.3 2. Sources d'apprentissage

Hiérarchisées du plus dense au plus pratique.

11.3.1 Documentation officielle (source primaire, toujours)

- [Next.js 15 Docs](#) — App Router, Server Actions, Middleware
- [React 19 Docs](#) — hooks, Suspense, Server Components
- [Prisma Docs](#) — schéma, migrations, queries
- [PostgreSQL Docs](#) — quand Prisma ne suffit pas
- [MDN Web Docs](#) — référence absolue pour les standards web
- [WHATWG specs](#) — pour SSE, fetch, EventSource
- [TypeScript Handbook](#) — types avancés, génériques

11.3.2 Sécurité et bonnes pratiques

- [OWASP Top 10 et Cheat Sheet Series](#)
- [CNIL — Guide sécurité](#)
- [Mozilla Web Security Guidelines](#)

11.3.3 Articles techniques et retours d'expérience

- [Vercel Engineering Blog](#) — patterns serverless, Edge runtime
- [Prisma Blog](#) — release notes, best practices
- [Bun Blog](#) — état de l'art runtime JS alternatif
- [Web.dev](#) — performance, Core Web Vitals, accessibilité

11.3.4 Communautés

- GitHub Issues des libs utilisées (Next.js, Prisma, jose) — précieux pour comprendre les pièges réels
 - Discord Zone01 — entraide quotidienne avec les promos et formateurs
 - Stack Overflow — résolution ponctuelle (vérification systématique de la date des réponses, l'écosystème JS évolue vite)
-

11.4 3. Méthode personnelle de veille

11.4.1 3.1 Curation hebdomadaire

Sources suivies chaque semaine :

- Newsletters : [This Week in React](#), [Bytes](#), [Node Weekly](#)
- Releases GitHub des libs utilisées (notification activée sur Next.js, Prisma, React)
- Blog d'Anthropic, Vercel, Cloudflare (acteurs majeurs de l'écosystème actuel)

11.4.2 3.2 Veille appliquée

Quand un sujet revient (par exemple SSE, Bun, Edge runtime), je vais :

1. lire la **documentation officielle** d'abord, pas un tutoriel YouTube
2. construire un **mini-prototype** isolé (souvent dans un dossier `playground/` local)
3. confronter au **projet réel** (est-ce que ça résout un problème concret ?)
4. **documenter** la décision (commentaire commit, ADR, ou ici cette section)

11.4.3 3.3 Outils utilisés pour la veille

- **GitHub** : étoiles + watch sur les libs critiques
 - **VS Code + extensions** : ESLint, Prisma, Mermaid Preview, Error Lens
 - **Postman / curl** : tester les routes API à la main avant d'écrire le code
 - **Chrome DevTools** : Network (analyser les requêtes), Application (cookies/SSE), Performance (profiling)
 - **dbdiagram.io** : visualiser le MCD/MLD
 - **mermaid.live** : prototyper des diagrammes avant de les valider dans le dossier
-

11.5 4. Points où je me suis surpris

Section honnête sur ce qui m'a challengé pendant le projet :

- **L'asynchrone côté serveur** : la programmation `async/await` en TypeScript est plus subtile qu'il n'y paraît. Erreur classique : oublier un `await` dans un `try/catch` → l'erreur n'est pas capturée comme prévu. J'ai pris l'habitude de toujours typer le retour des fonctions async, ce qui force le compilateur à m'avertir.
- **Les contraintes Prisma sur les relations** : la première fois que j'ai voulu mettre `onDelete : Cascade` sur une relation `User → Post`, j'ai compris que la modélisation des relations a un impact direct sur la cohérence des données. Penser **scénarios de suppression** dès la modélisation est un réflexe que je n'avais pas.
- **Le SSE côté serveur Next.js** : l'API `ReadableStream` n'est pas évidente quand on vient du modèle Express (`res.write`). Comprendre que le contrôleur doit gérer le `request.signal.aborted` pour ne pas laisser de polling orphelin a été un déclic.
- **Le rôle du middleware Next.js** : au début je le voyais comme une feature accessoire. En fait c'est le seul endroit où je peux centraliser une politique de sécurité au-dessus de toutes les routes. Le pattern « deny-by-default + exceptions explicites » est devenu mon réflexe.
- **L'absence de Chromium dans le sandbox de génération de diagrammes** (cf. section diagrams) : ça m'a fait comprendre que rendre un SVG vectoriel sans navigateur est non-trivial. Pour la prod, on utilise généralement un service externe (kroki, mermaid.ink) ou un Chromium headless dédié.

11.6 5. Application concrète au dossier

Cette veille technique se retrouve **partout** dans le projet :

- Choix SSE + Upstash → [04-developpement/README.md](#)
- Choix JWT custom + audit OWASP → [securite-rgpd.md](#)
- Choix Bun + Docker multi-stage → [05-deploiement/README.md](#)
- Choix Prisma + migrations → [03-conception/donnees.md](#)
- Choix Mermaid pour les diagrammes → [03-conception/diagrammes-uml.md](#)

Chaque choix technique est explicité, comparé à ses alternatives, et tracé jusqu'au commit.

11.7 6. Plan de veille post-certification

Domaines que je veux continuer à explorer une fois le titre obtenu :

1. **Edge computing avancé** (Vercel Edge Functions, Cloudflare Workers) — pour les latences ultra-basses
2. **Rust pour le tooling web** (oxc, biome, swc) — performance des outils dev
3. **Observabilité moderne** (OpenTelemetry, Grafana Tempo, Sentry) — au-delà du `console.log`
4. **Bases de données vectorielles** (pgvector, Pinecone) — pour intégrer de l'IA (recommandations, recherche sémantique)
5. **Architecture hexagonale / DDD** appliquée à Next.js — séparation claire domaine vs infrastructure
6. **Sécurité applicative avancée** : certification type OWASP WSTG, lecture du bug bounty Vercel

12 05 - Déploiement

12.1 Objectif

Documenter la chaîne de déploiement du projet, du poste de développement jusqu'à la mise en production sur Vercel, avec PostgreSQL Neon, Redis et les migrations Prisma.

12.2 Preuves & Mapping GitHub

Les éléments de déploiement et d'intégration continue sont documentés dans les tickets et PRs suivants:

- DevOps / Docker / CI
- CI – Lint, test, build, push image
- Seed script / demo data (utilisé pour tests locaux)
- PR stabilisation (Docker/Neon/Prisma/Redis)

12.3 Docker et Containerisation

12.3.1 Services principaux

- **app**: application Next.js 15.
- **db**: PostgreSQL pour les données métier.
- **Upstash Redis**: service temps réel configuré via variables d'environnement, sans service Redis local dédié dans `docker-compose.yml`.

12.3.2 docker-compose.yml

La configuration à la racine du projet orchestre l'exécution locale de l'application et de la base de données. Le service `app` utilise le fichier `.env` pour charger les variables nécessaires, notamment l'accès à PostgreSQL et à Upstash Redis.

```
services:
  app:
    profiles: ["prod"]
    build:
      context: .
      dockerfile: Dockerfile
    restart: unless-stopped
    env_file:
      - .env
    environment:
      DATABASE_URL: ${DATABASE_URL}
```

```

    NODE_ENV: production
    NEXT_TELEMETRY_DISABLED: "1"
  ports:
    - "3000:3000"

  app-dev:
    profiles: ["dev"]
    image: oven/bun:1
    working_dir: /app
    restart: unless-stopped
    env_file:
      - .env
    environment:
      DATABASE_URL: ${DATABASE_URL}
      NODE_ENV: development
      NEXT_TELEMETRY_DISABLED: "1"
      WATCHPACK_POLLING: "true"
      CHOKIDAR_USEPOLLING: "true"
    command: >
      sh -c "
        bun install &&
        bunx prisma generate &&
        bunx prisma migrate deploy &&
        bun run dev
      "
    ports:
      - "3000:3000"
    volumes:
      - ./app
      - app-node-modules:/app/node_modules
      - app-next-cache:/app/.next
    stdin_open: true
    tty: true

volumes:
  app-node-modules:
  app-next-cache:

```

12.3.3 Dockerfile

Le Dockerfile racine suit une logique simple: installer les dépendances, builder l'application puis démarrer le serveur Next.js.

Points importants:

- installation des dépendances avec lockfile;
- génération Prisma avant le build;
- image légère pour la production;
- exposition du port 3000.

12.4 Bases de Données

12.4.1 PostgreSQL sur Neon

- **URL principale:** DATABASE_URL.
- **Migrations:** `prisma migrate deploy` en production.
- **Sécurité:** secrets injectés via variables d'environnement, jamais committés.

12.4.2 Redis

Redis sert à trois usages:

- sessions serveur pour l'invalidation JWT;
 - cache applicatif léger;
 - diffusion des événements SSE/Redis entre instances.
-

12.5 Mise en Production

12.5.1 Vercel

Le déploiement cible Vercel pour la partie Next.js.

Flux retenu:

1. push sur la branche principale.
2. build automatique par Vercel.
3. exécution des migrations lors du déploiement si nécessaire.
4. publication avec SSL automatique.

12.5.2 Variables d'environnement

```
DATABASE_URL=postgresql://...  
JWT_SECRET=...  
OAUTH_TOKEN_ENCRYPTION_KEY=...  
UPSTASH_REDIS_REST_URL=https://...  
UPSTASH_REDIS_REST_TOKEN=...  
CLOUDINARY_CLOUD_NAME=...  
CLOUDINARY_API_KEY=...  
CLOUDINARY_API_SECRET=...  
NEXT_PUBLIC_GIPHY_API_KEY=...  
CLIENT_ID=...  
CLIENT_SECRET=...  
REDIRECT_URL=...
```

Pour le temps réel, le projet utilise Upstash Redis via `UPSTASH_REDIS_REST_URL` et `UPSTASH_REDIS_REST_TOKEN`.

12.5.3 Contrôles post-déploiement

- vérifier la page d'accueil et la page de connexion;
 - tester la création de post;
 - tester un message temps réel;
 - vérifier les cookies HTTP-only et les redirections middleware.
-

12.6 CI/CD

12.6.1 GitHub Actions

Un pipeline simple suffit pour ce projet de certification:

- `lint` pour ESLint;
- `test` pour Jest;
- `build` pour Next.js;
- `deploy` pour Vercel ou un déploiement équivalent.

Workflow de déploiement du projet:

```
name: CI

on:
  push:
    branches: [main]
  pull_request:

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: 20
      - run: npm ci
      - run: npm run lint
      - run: npm run test
      - run: npm run build
```

12.7 Monitoring et Exploitation

12.7.1 Observabilité

- **Logs applicatifs:** console structurée côté serveur.
- **Vercel Analytics:** suivi des performances front.
- **Sentry:** suivi des erreurs et régressions.
- **Alertes:** surveillance des échecs de build et de déploiement.

12.7.2 Réversibilité

Le rollback se fait côté Vercel en cas de régression fonctionnelle ou d'erreur de migration.

```
vercel rollback
```

12.8 Checklist de Déploiement

- ☒ Dockerfile défini à la racine.
 - ☒ docker-compose.yml pour l'environnement local.
 - ☒ Variables d'environnement documentées.
 - ☒ Prisma Migrate pris en compte.
 - ☒ Redis documenté pour les sessions et le temps réel.
 - ☒ Déploiement cible sur Vercel.
 - ☒ Pipeline GitHub Actions documenté.
 - ☒ Monitoring des erreurs documenté (Vercel Analytics + logs structurés serveur).
-

12.9 Résultat Attendu

Cette section démontre que le projet est déployable de bout en bout: base de données, cache, temps réel, build et supervision sont décrits avec un chemin de mise en production cohérent et réaliste.

13 06 - Bilan

13.1 Objectif

Présenter le retour d'expérience sur le projet, les acquis techniques, les limites constatées et les pistes d'évolution réalistes dans une logique de certification.

13.2 Retour d'Expérience

13.2.1 Défis Techniques Rencontrés

1. Auth middleware et sécurité des routes

- Gérer les cookies HTTP-only côté serveur sans exposer les jetons au client.
- Synchroniser middleware Next.js, validations JWT (jose) et route handlers protégées.
- Implémenter correctement le hachage bcrypt et la vérification des mots de passe.
- Résultat : authentification robuste et vérifiée à chaque requête protégée.

2. Temps réel avec Redis et SSE

- Le projet utilise Upstash Redis avec un mécanisme de polling/SSE pour la synchronisation temps réel.
- Garantir la persistance des messages dans Redis avant leur émission au client.
- Gérer la disconnexion/reconnexion et éviter les duplicatas.
- Résultat : notifications et messages en temps réel fiables avec latence acceptable.

3. Modélisation Prisma complexe

- 18 modèles avec 40+ relations et contraintes d'unicité (userId + postId pour les likes).
- Maintenir l'intégrité référentielle tout en permettant les suppressions en cascade.
- Migrer sans perdre les données existantes lors du passage à de nouveaux schémas.
- Résultat : base de données normalisée, performante et facilement maintenue.

4. Coordination frontend/backend

- Aligner les types TypeScript côté client et serveur pour éviter les regressions.
- Valider les données à l'entrée (Zod) et à la sortie (Prisma types).
- Gérer les erreurs API de manière cohérente (400, 401, 500, etc.).
- Résultat : expérience utilisateur fluide et erreurs explicites.

13.2.2 Défis d'Organisation et de Documentation

- Structurer un dossier de certification lisible pour jury technique ET non-technique.
- Distinguer : ce qui est dans la documentation vs ce qui reste dans le code source.
- Répartir 60+ pages entre conception, développement, déploiement et bilan sans perdre la cohérence narrative.
- Lier chaque section à des preuves GitHub concrètes (issues, PRs, commits).

13.2.3 Apprentissages Clés

Compétences validées :

- Next.js App Router (server-side rendering, server actions, middleware).
- TypeScript pour la sécurité des types en full-stack.
- PostgreSQL et Prisma pour une modélisation robuste.
- JWT et authentification stateless.
- Déploiement en conteneurs (Docker + Docker Compose).
- Travail en équipe via GitHub (issues, PRs, code review).

Points d'amélioration identifiés :

- Tester davantage les routes critiques (auth, chat, notifications).
- Mettre en place du monitoring en production (Sentry, alertes).
- Optimiser le caching Redis pour réduire les requêtes DB.
- Ajouter des benchmarks de performance (Lighthouse, load tests).

13.3 Accomplissements Mesurables

13.3.1 Livrables du Projet

Élément	Nombre	Statut
Modèles Prisma	18	Complets et documentés
Endpoints API	60+	Spécifications complètes (api-spec.md)
Pages principales	14	Routes implémentées
Composants React	110+	Design system + composants métier
Hooks personnalisés	30+	Logique UI découplée
User stories	38+	Rôles, priorités, critères d'acceptation
Événements temps réel	4 familles	Chat, notifications, typing, statuts
Migrations DB	2+	Structure versionnée
Lignes de documentation	4000+	Dossier et code
Tests d'intégration	Auth	Inscription + connexion vérifiées

13.3.2 Résultats Obtenus

Architecture

- Séparation claire client/serveur/data

- TypeScript full-stack pour la sécurité
- Middleware pour l'authentification centralisée
- API RESTful avec validation (Zod)

Authentification & Sécurité

- JWT avec jose (HS256)
- Hachage bcrypt des mots de passe
- Cookies HTTP-only pour les sessions
- Protection des routes privées

Base de Données

- 18 modèles Prisma normalisés
- Relations complexes (amitié, notifications, groupes)
- Migrations versionnées
- Contraintes d'unicité pour éviter les duplicatas

Temps Réel

- Messages avec statut (SENT → DELIVERED → READ)
- Notifications en temps réel via Redis
- Typing indicator avec timeout
- Polling SSE pour haute disponibilité

Déploiement

- Dockerfile multi-stage optimisé
- docker-compose avec PostgreSQL + Redis
- Variables d'environnement sécurisées
- Pipeline CI/CD documenté

Documentation

- Dossier de certification 60+ pages
- Preuves GitHub reliées (13 issues/PRs)
- Diagrammes d'architecture (Mermaid)
- API specification complète

13.3.3 Points Forts du Projet

- **Pragmatique** : Choix techniques éprouvés (Next.js, Prisma, PostgreSQL).
- **Documenté** : Chaque section du dossier renvoie à du code réel.
- **Sécurisé** : Authentification robuste, validation des données, hachage des mots de passe.
- **Scalable** : Redis pour le cache/sessions, migrations Prisma, indexing DB.
- **Équipe** : Travail collaboratif via GitHub, coordonnés sur 13 issues principales.

13.3.4 Points à Améliorer (Phase Suivante)

- **Tests** : Augmenter la couverture au-delà de l'authentification (routes, composants).
 - **Performance** : Lighthouse audit, optimisation d'images, compression.
 - **Monitoring** : Intégrer Sentry pour suivi des erreurs en production.
 - **Load tests** : Vérifier la scalabilité sous charge (Redis pub/sub, connexions DB).
 - **RGPD** : Formaliser les droits d'accès/suppression des données personnelles.
-

13.4 Compétences Validées (RNCP 37873)

13.4.1 Bloc 1 : Développer une application sécurisée

- Authentification JWT sécurisée (jose + bcrypt)
- Validation des entrées (Zod schemas)
- Protection des routes (middleware + check userId)
- Hachage des mots de passe (bcrypt, salt 12)
- Cookies HTTP-only pour la session

13.4.2 Bloc 2 : Concevoir une application organisée en couches

- Séparation client/serveur/data
- Composants React réutilisables
- Hooks personnalisés pour la logique métier
- Prisma ORM pour l'accès aux données
- API Routes pour la couche métier

13.4.3 Bloc 3 : Préparer le déploiement sécurisé

- Dockerfile multi-stage optimisé
- docker-compose avec services
- Variables d'environnement versionnées
- Migrations Prisma automatisées
- CI/CD avec GitHub Actions

13.4.4 Compétences Techniques Consolidées

Compétence	Niveau	Preuve
Next.js App Router	Intermédiaire+	src/middleware.ts, API routes
TypeScript	Intermédiaire+	Typage strict côté/côté serveur

Compétence	Niveau	Preuve
PostgreSQL	Intermédiaire	18 modèles, relations complexes, migrations
Prisma ORM	Intermédiaire+	Schema, queries, migrations
Authentication	Intermédiaire+	JWT, bcrypt, cookies, middleware
Docker/Compose	Intermédiaire	Dockerfile, services, volumes
Git/GitHub	Intermédiaire	13 issues, PRs, commits documentés
Real-time	Introductory	Redis, SSE polling, status messages

13.4.5 Compétences Transversales

- **Communication** : Dossier de certification clair, diagrammes, preuve d'exécution.
- **Résolution de problèmes** : Auth middleware, modélisation Prisma, déploiement Docker.
- **Apprentissage autonome** : Documentation officielle, essais/erreurs, ajustements techniques.
- **Travail en équipe** : Collaboration GitHub, coordination sur issues communes.

13.5 Améliorations Futures

13.5.1 Court Terme

- Finaliser des tests d'intégration plus représentatifs.
- Mesurer les performances réelles de la home, du feed et du chat.
- Ajouter une page de monitoring minimaliste pour les erreurs critiques.

13.5.2 Moyen Terme

- Ajouter des notifications push natives.
- Renforcer la modération et les signalements.
- Introduire des recommandations de contenu plus fines.

13.5.3 Long Terme

- Séparer certains domaines en services indépendants si la charge augmente.
- Ajouter de l'analyse d'usage et des métriques produit.
- Préparer une version mobile dédiée si le besoin utilisateur le justifie.

13.6 Métriques du Projet

Indicateur	Valeur
Modèles de données	18
Endpoints API documentés	20+
Pages principales	12+
User stories	43
Événements temps réel	4 familles principales
Composants React	110+
Hooks personnalisés	30+
Variables CSS thème	200+

Ces métriques servent à montrer que le projet est suffisamment complet pour mon examen de certification.

13.7 Conclusion

Ce projet m'a permis de concevoir et documenter une application full-stack réaliste, avec des choix techniques justifiés et une architecture exploitable en production.

Pour le jury, l'essentiel est démontré: besoin métier compris, conception structurée, implémentation cohérente, capacité à aller jusqu'au déploiement, et recul sur les limites du projet.

14 Conformité RNCP 37873 — Social Network

Candidat : Christophe Lecart

Titre visé : Concepteur Développeur d'Applications (CDA) — Niveau 6

Établissement : Zone01 Rouen Normandie

Projet support : Social Network (dépôt arocchet/social-network)

Date : Juin 2026

Ce document présente la couverture complète du référentiel RNCP 37873 par le projet Social Network, avec les preuves techniques associées à chaque compétence.

14.1 Structure du Référentiel RNCP 37873

Le titre CDA est organisé en 3 blocs de compétences couvrant 11 compétences professionnelles :

Bloc	Intitulé	Compétences
BC01	Développer une application sécurisée	C1 → C4
BC02	Concevoir et développer une application sécurisée organisée en couches	C5 → C8
BC03	Préparer le déploiement d'une application sécurisée	C9 → C11

14.2 Bloc 1 — Développer une application sécurisée (BC01)

14.2.1 C1 — Installer et configurer son environnement de travail

Preuve principale : Configuration complète de l'environnement de développement.

Élément	Réalisation
Environnement local	Node.js 20+, Bun, TypeScript, ESLint, Prisma CLI
Base de données locale	PostgreSQL via Docker Compose (docker-compose --profile dev)
Variables d'environnement	.env avec 33 variables documentées (JWT, Cloudinary, OAuth...)
IDE et outils	VS Code, extensions TypeScript/Prisma, ESLint intégré
Versioning	Git + GitHub, branches feature/fix, 100+ commits tracés

Références GitHub : [Issue #40 — DevOps/Docker/CI](#)

14.2.2 C2 — Développer des interfaces utilisateur

Preuve principale : 14 pages responsive implémentées avec Next.js 15 App Router.

Page	Route	Technologie
Feed principal	/	React 19, React Query, SSE
Connexion	/login	React Hook Form, Zod
Inscription	/register	React Hook Form, Zod, Cloudinary
Onboarding	/onboarding	Multi-step form
Profil utilisateur	/profile/[userId]	SSR, SWR
Chat direct	/chat/[id]	SSE polling, Redis
Groupe	/groups/[id]	React Query
Événements	/events	React Day Picker
Reels	/reels	Embla Carousel
Recherche	/search	Debounce, React Query
Paramètres	/settings	React Hook Form
Notifications	/notifications	SSE, Redis
Invitations	/invitations	React Query
Profil (édition)	/settings/profile	Cloudinary upload

Design system : Tailwind CSS 4, Shadcn/UI, Radix UI, 110+ composants réutilisables.

Responsive : Mobile (< 640px), Tablet (640–1024px), Desktop (> 1024px).

Accessibilité : Radix UI (composants ARIA), next/image, alt texts.

Wireframes & maquettes : disponibles dans [07-annexes](#).

14.2.3 C3 — Développer des composants métier

Preuve principale : Logique métier encapsulée dans des composants et hooks dédiés.

Fonctionnalité	Composant/Hook	Description
Publication de posts	PostCreator, usePostData	Texte + image + visibilité
Réactions (7 types)	ReactionButtons, ReactionPicker	LIKE, DISLIKE, LOVE, LAUGH, SAD, ANGRY, WOW
Commentaires	CommentList, CommentForm	Lecture et écriture avec validation

Fonctionnalité	Composant/Hook	Description
Stories éphémères	StoryCreator, StoryViewer	Upload Cloudinary, 24h TTL
Chat temps réel	MessageBox, TypingIndicator	SSE + Redis polling 500ms
RSVP événements	RSVPButtons	YES / NO / MAYBE
Gestion de groupes	GroupSettings, InvitationManager	Invitations, demandes, membres
Notifications	NotificationBadge	SSE, lecture/suppression
Follow/Friendship	FollowButton	Suivi + demandes d'amitié

Validation des données : Zod schemas côté serveur pour tous les endpoints critiques.

14.2.4 C4 — Contribuer à la gestion de projet

Preuve principale : Suivi de projet via GitHub Issues et Pull Requests.

Pratique	Preuve
Découpage en tickets	13 issues GitHub couvrant toutes les fonctionnalités principales
Pull Requests documentées	PR #118 — stabilisation Docker/Neon/Prisma/Redis
Documentation technique	Dossier de certification 60+ pages, API spec, diagrammes
Gestion des branches	Branches feature/, fix/, docs/ séparées
Collaboration	Revue de code, coordination inter-équipe sur issues communes

Références : [Issue #13](#), [Issue #24](#), [PR #118](#)

14.3 Bloc 2 — Concevoir et développer une application sécurisée organisée en couches (BC02)

14.3.1 C5 — Analyser les besoins et maquetter une application

Preuve principale : Cahier des charges, user stories et maquettes.

Livrable	Contenu
Cahier des charges	02-cahier-des-charges/README.md — fonctionnalités, contraintes, timeline
User stories	03-conception/user-stories.md — 38+ stories avec critères d'acceptation
Wireframes basse-fidélité	6 wireframes dans 07-annexes
Maquettes haute-fidélité	6 maquettes dans 07-annexes
Design system	Palette, typographie, breakpoints, composants atomiques

14.3.2 C6 — Définir l'architecture logicielle d'une application

Preuve principale : Architecture 3 couches (Client / Serveur / Données) documentée.

COUCHE PRÉSENTATION (React/Next.js)

- 110+ composants React (Tailwind)
- 30+ hooks personnalisés
- React Query + SWR (état serveur)
- SSE client (notifications/chat)

HTTP/REST + SSE

COUCHE MÉTIER (Next.js API Routes)

- 60+ endpoints REST (public/private)
- Middleware JWT (auth centralisée)
- Validation Zod (toutes les entrées)
- Upstash Redis (cache + temps réel)

Prisma ORM

COUCHE DONNÉES

- PostgreSQL Neon (18 modèles)
- Upstash Redis (TTL 60s, temps réel)
- Cloudinary (médias, CDN)

Choix technologiques justifiés :

Composant	Choix	Justification
Framework	Next.js 15 + React 19	SSR + RSC, performances, App Router
Langage	TypeScript	Typage fort, sécurité à la compilation
Styling	Tailwind CSS 4 + ShadCN	Responsive, accessible, maintenable
BDD	PostgreSQL (Neon)	ACID, relationnel, serverless
ORM	Prisma v6	Type-safe, migrations versionnées
Temps réel	SSE + Upstash Redis	Serverless-compatible, haute disponibilité
Médias	Cloudinary	CDN, transformations, stockage sécurisé
Auth	JWT (jose) + bcrypt	Stateless, sécurisé, standard industrie

14.3.3 C7 — Concevoir une base de données relationnelle

Preuve principale : 18 modèles Prisma normalisés avec contraintes d'intégrité.

Modèle	Rôle
User	Entité centrale, profil, visibilité
Post / Comment	Contenu utilisateur, visibilité granulaire
Reaction	Polymorphe (post, story, commentaire)
Story	Contenu éphémère, média Cloudinary
Message / Conversation	Messagerie directe avec statuts SENT/DELIVERED/READ
GroupMessage	Chat de groupe avec association événements
Friendship	Relation bidirectionnelle avec statut
GroupMember / GroupInvitation / GroupJoinRequest	Cycle de vie complet des groupes
Event / Rsvp	Événements avec réponses YES/NO/MAYBE
UserSettings	Préférences (thème, langue, notifs)
Account	Comptes OAuth (Google)
Notification	Alertes utilisateur indexées

Diagrammes : MCD, MLD et MPD disponibles dans [03-conception/diagrammes-uml.md](#).

Normalisation : 3NF respectée, contraintes d'unicité sur les relations critiques (userId + postId, userId + eventId...).

14.3.4 C8 — Développer des composants d'accès aux données

Preuve principale : Couche d'accès aux données via Prisma ORM + Redis.

Pattern	Implémentation
CRUD complet	<code>src/lib/server/</code> — fonctions dédiées par domaine (post, user, chat...)
Requêtes optimisées	<code>select</code> Prisma pour limiter les colonnes renvoyées
Pagination	Curseur Prisma (<code>cursor + take</code>) pour le scroll infini
Cache Redis	TTL 60s sur les messages, invalidation à l'écriture
Transactions	Prisma transactions pour les opérations multi-tables
Migrations versionnées	<code>prisma/migrations/</code> — historique complet
Tests d'intégration	<code>__tests__/integrations/authentication.test.ts</code> — Jest + base SQLite

Sécurité des données : Prisma empêche nativement les injections SQL (requêtes paramétrées). Zod valide toutes les entrées en amont.

14.4 Bloc 3 — Préparer le déploiement d'une application sécurisée (BC03)

14.4.1 C9 — Préparer et exécuter les plans de tests d'une application

Preuve principale : Stratégie de tests documentée et exécutée.

Type de test	Outil	Couverture
Tests d'intégration	Jest + ts-jest	Authentification (register + login)
Environnement isolé	SQLite (<code>DATABASE_TEST_URL</code>)	Isolation complète de la prod
Setup/Teardown global	<code>jest.globalSetup.ts</code> + <code>jest.globalTeardown.ts</code>	Base de test propre à chaque run
Jeux d'essai	<code>@faker-js/faker</code>	Données réalistes générées
Validation métier	Tests manuels documentés (07-annexes)	Parcours utilisateur couverts

Jeux d'essai documentés :

Scenario	Entrée	Résultat attendu	Statut
Inscription	Email unique + mot de passe	Compte créé, JWT renvoyé	
Connexion valide	Email/password corrects	Token JWT en cookie	
Connexion invalide	Mauvais mot de passe	Erreur 401	
Création de post	Texte + image	Post visible dans le feed	
Follow utilisateur	User A → User B	Relation créée	
Envoi message	Texte vers conversation	Message persisté + Redis	
Réaction sur post	Type LIKE	Compteur mis à jour	
RSVP événement	YES/NO/MAYBE	Réponse enregistrée	

14.4.2 C10 — Préparer et documenter le déploiement d'une application

Preuve principale : Containerisation complète Docker avec pipeline CI/CD.

Livrable	Contenu
Dockerfile multi-stage	Builder (Bun) + Runtime (Node léger), génération Prisma incluse
docker-compose.yml	Profils prod et dev, PostgreSQL local, volumes persistants
Variables d'env	33 variables documentées dans 05-déploiement/README.md
Pipeline CI/CD	GitHub Actions : lint → test → build (.github/workflows/ci.yml)
Migrations automatisées	prisma migrate deploy exécuté au démarrage du conteneur

Déploiement cible : Vercel (Next.js) + Neon (PostgreSQL serverless) + Upstash (Redis serverless).

Commandes de déploiement :

```
# Production locale
docker compose --profile prod up --build

# Développement
docker compose --profile dev up app-dev db

# Production Vercel
git push origin main # → déclenche CI → build → deploy automatique
```

14.4.3 C11 — Contribuer à la mise en production dans une démarche DevOps

Preuve principale : Démarche DevOps documentée et appliquée.

Pratique DevOps	Implémentation
Infrastructure as Code	Dockerfile + docker-compose.yml versionnés dans Git
Pipeline d'intégration (CI)	GitHub Actions : lint + test + build sur chaque push/PR
Livraison continue (CD)	Vercel auto-deploy sur merge dans main
Séparation des environnements	dev (hot-reload + DB locale) / prod (image standalone)
Rollback	vercel rollback — retour immédiat en cas de régression
Observabilité	Vercel Analytics, logs structurés serveur, health endpoint
Sécurité secrets	Variables d'environnement injectées, aucun secret committé

Référence : [Issue #45 — CI lint/test/build/push, PR #118](#)

14.5 Tableau de Synthèse — Couverture RNCP 37873

Compétence RNCP	Statut	Preuves principales
C1 — Configurer l'environnement de travail		Docker, Node/Bun, Prisma, .env, ESLint
C2 — Développer des interfaces utilisateur		14 pages, 110+ composants, design system, responsive
C3 — Développer des composants métier		60+ endpoints, hooks, Zod, logique sociale complète
C4 — Contribuer à la gestion de projet		13 issues, PR #118, documentation 60+ pages
C5 — Analyser les besoins et maquetter		Cahier des charges, 38+ user stories, wireframes/mockups
C6 — Définir l'architecture logicielle		Architecture 3 couches, diagrammes Mermaid, justifications
C7 — Concevoir une base de données relationnelle		18 modèles, MCD/MLD/MPD, contraintes, migrations
C8 — Développer des composants d'accès aux données		Prisma ORM, Redis cache, pagination curseur, tests

Compétence RNCP	Statut Preuves principales
C9 – Préparer et exécuter les plans de tests	Jest, tests intégration auth, jeux d’essai documentés
C10 – Préparer et documenter le déploiement	Dockerfile, docker-compose, CI/CD, variables env
C11 – Contribuer à la mise en production DevOps	GitHub Actions, Vercel CD, rollback, observabilité

Compétences transversales :

Compétence	Démonstration
Communication technique	Dossier 60+ pages, diagrammes, API spec, README complet
Résolution de problèmes	Auth middleware, modélisation Prisma complexe, déploiement Docker
Apprentissage autonome	Maîtrise SSE/Redis, Next.js 15 App Router, TypeScript strict
Travail en équipe	GitHub collaboratif, 13+ issues, revues de code inter-membres

14.6 Correspondance Dossier → Blocs RNCP

Section du dossier	Bloc(s) RNCP couverts
00-présentation	Profil candidat
01-introduction	Contexte projet
02-cahier-des-charges	BC02 / C5
03-conception (données, UML, stories)	BC02 / C5, C6, C7
04-developpement (API, code)	BC01 / C2, C3 + BC02 / C8
05-déploiement	BC03 / C10, C11
06-bilan	Toutes compétences
07-annexes	Preuves concrètes

15 Mapping RNCP 37873 — 11 compétences → preuves projet

Ce document est le point d'entrée pour le jury. Pour chaque compétence du référentiel RNCP 37873 (Concepteur Développeur d'Applications), il indique :

- la définition officielle,
- les **preuves concrètes** dans le projet Social Network (fichiers code, sections du dossier, issues GitHub, PR),
- le **niveau de couverture** (Démontré / Partiel / Non couvert),
- les **éléments à montrer pendant la soutenance** (extraits de code, captures, diagrammes).

Le projet est porté par une équipe (arocchet/social-network) ; **Christophe Lecart** y a contribué à hauteur de 57 **commits** (2^e contributeur). Les commits personnels les plus représentatifs sont indiqués pour chaque compétence quand applicable.

15.1 Vue d'ensemble

#	Bloc	Compétence	Couverture	Section dossier
1	B1	Installer et configurer son environnement de travail		05-déploiement
2	B1	Développer des interfaces utilisateur		03-conception , 04-développement
3	B1	Développer des composants métier		04-développement , api-spec
4	B1	Contribuer à la gestion d'un projet informatique		02-cahier-des-charges , preuves GitHub
5	B2	Analyser les besoins et maquetter une application		03-conception , annexes mockups

#	Bloc	Compétence	Couverture	Section dossier
6	B2	Définir l'architecture logicielle d'une application		04-developpement, diagrammes-uml
7	B2	Concevoir et mettre en place une base de données relationnelle		donnees.md, diagrammes-uml.md
8	B2	Développer des composants d'accès aux données SQL et NoSQL		04-developpement
9	B3	Préparer et exécuter les plans de tests d'une application		04-developpement, tests-strategy
10	B3	Préparer et documenter le déploiement d'une application		05-deploiement
11	B3	Contribuer à la mise en production dans une démarche DevOps		05-deploiement

Sécurité (transverse aux 3 blocs) : [04-developpement/securite-rgpd.md](#)

15.2 BLOC 1 — Développer une application sécurisée

15.2.1 Compétence 1 — Installer et configurer son environnement de travail en fonction du projet

Outils de développement, conteneurs, gestionnaire de paquets, intégration continue

Couverture : Démontré

Preuves concrètes :

Élément	Localisation
Environnement Docker reproductible	Dockerfile + docker-compose.yml à la racine
Image runtime Bun (cohérence runtime + build)	Dockerfile ligne 2 : FROM oven/bun:1
Configuration .env documentée	.env.exemple (12 variables)
Migrations Prisma automatisées	Dockerfile ligne finale : bunx prisma migrate deploy
Variables d'environnement isolées	docker-compose.yml lignes 5-9
Stack locale complète	services app, db (PostgreSQL 16), redis (Redis 7)
Section dossier dédiée	05-déploiement/README.md
Issue GitHub	#40 DevOps/Docker/CI

À montrer en soutenance : - docker-compose up qui démarre tout l'environnement en une commande - Le Dockerfile multi-stage (builder Bun → runner Bun, OpenSSL pour Prisma) - L'absence totale de credentials hardcodés (grep -r "JWT_SECRET\s*" src/ → uniquement référence à process.env.JWT_SECRET)

15.2.2 Compétence 2 — Développer des interfaces utilisateur

Maquettes, design system, accessibilité, responsive, composants réutilisables

Couverture : Démontré

Preuves concrètes :

Élément	Localisation
14 pages principales documentées	03-conception/pages.md
Wireframes + mockups Figma	07-annexes/ (Wireframe_.png, Mockups_.png)
Design system (palette dark, typo Geist, breakpoints)	03-conception/README.md sections « Design System »
Composants UI réutilisables	src/components/ui/ (50+ composants Radix UI + shadcn)

Élément	Localisation
Composants métier	<code>src/components/feed/, chat/, groups/, auth/</code>
Responsive design (3 breakpoints)	Tailwind CSS – <code>tailwind.config.js</code>
Thème clair/sombre	<code>next-themes,</code> <code>src/hooks/use-theme.ts</code>
i18n (FR/EN)	Issue #76
Settings utilisateur (theme, language)	<code>src/app/api/private/user/settings/route.ts</code>

À montrer en soutenance : - Démo navigation sur mobile et desktop (responsive) - Switch thème clair/sombre en direct - Switch langue FR/EN - Capture mockup → rendu final côte-à-côte pour 1-2 écrans clés (feed, profil)

15.2.3 Compétence 3 – Développer des composants métier

Logique applicative, POO, API REST, refactoring, tests unitaires, cryptographie

Couverture : Démontré

Preuves concrètes :

Élément	Localisation
60+ endpoints REST organisés en couches	04-developpement/api-spec.md
Composants métier (posts, réactions, follow, chat, groupes, événements)	<code>src/lib/server/</code> (queries + business logic)
Validation systématique (36 schémas Zod)	<code>src/lib/schemas/</code>
Cryptographie : JWT signé HS256 via jose	<code>src/lib/jwt/signJwt.ts,</code> <code>verifyJwt.ts</code>
Cryptographie : bcrypt cost 12	<code>src/lib/security/hash.ts</code>
Réponses normalisées	<code>src/lib/server/api/response.ts</code> (<code>respondSuccess / respondError</code>)
Helpers d'auth	<code>src/lib/server/api/getUserId.ts</code>
OAuth Google complet	<code>src/app/api/public/auth/redirect/google/route.ts</code> + <code>callback/google/route.ts</code>
Test d'intégration auth	<code>__tests__/integrations/authentication.test.ts</code>

Commits personnels de Christophe les plus représentatifs : - Système de réactions (likes posts/comments/stories) — fonctionnalité métier complète - Refacto Prisma + alignement schema (commit 06fe9e5) - Stabilisation déploiement / dépendances (commit c3748ea)

À montrer en soutenance : - Parcours d'une requête : `POST /api/private/post` (frontend → API route → schema Zod → query Prisma → réponse normalisée) - Le système de réactions (cas d'unicité par `@@unique([userId, postId])`) - Démo JWT : inspection du cookie `authToken` dans l'onglet Application des DevTools

15.2.4 Compétence 4 — Contribuer à la gestion d'un projet informatique

Organisation, planification, communication, Git, suivi de tickets

Couverture : Démontré

Preuves concrètes :

Élément	Localisation
Repo GitHub structuré	arocchet/social-network
Issues GitHub par fonctionnalité (12 issues majeures)	Section « Preuves GitHub » dans 02-cahier-des-charges
PR de stabilisation	PR #118 (Docker/Neon/Prisma/Redis)
Branches feature → merge	Historique <code>git log</code>
43 user stories priorisées	03-conception/user-stories.md
Timeline en 3 phases (MVP / Features / Polish)	02-cahier-des-charges section Timeline
Travail en équipe (6 contributeurs)	<code>git shortlog -sn</code>
Contribution Christophe : 57 commits (2 ^e contributeur)	contribution-personnelle.md

À montrer en soutenance : - Une issue représentative et la branche/PR qui la résout - Le `git shortlog` montrant la contribution équipe - La structure des commits de Christophe (par domaine : réactions, certification, refacto)

15.3 BLOC 2 — Concevoir et développer une application sécurisée organisée en couches

15.3.1 Compétence 5 — Analyser les besoins et maquetter une application

Couverture : Démontré

Preuves concrètes :

Élément	Localisation
Cahier des charges complet	02-cahier-des-charges/README.md
43 user stories format Agile	03-conception/user-stories.md
Wireframes (login, feed, profil, settings, story viewer, create post)	07-annexes/ (Wireframe_*.png)
Mockups graphiques aboutis	07-annexes/ (Mockups_*.png)
Critères d'acceptation par US	user-stories.md, chaque US
Priorisation HIGH/MEDIUM/LOW	user-stories.md
Découpage Phase 1/2/3	02-cahier-des-charges Timeline

À montrer en soutenance : - 1 user story complète : du wording (« En tant que... je veux... ») au critère d'acceptation - 1 wireframe → 1 mockup → 1 capture du rendu réel - Le tableau de priorisation

15.3.2 Compétence 6 — Définir l'architecture logicielle d'une application

Architecture en couches, diagrammes UML (classes, séquence, cas d'utilisation), stratégie de sécurité

Couverture : Démontré

Preuves concrètes :

Élément	Localisation
Architecture système (Mermaid)	04-developpement/README.md section « Diagrammes d'Architecture »
Diagramme de séquence Auth (login)	idem section « Auth Flow (Login) »
Diagramme de séquence Real-time messaging (SSE + Redis)	idem section « Real-time Messaging Sequence »
Diagramme de classes UML (18 entités)	03-conception/diagrammes-uml.md §4.2
Diagramme de cas d'utilisation UML	idem §5.2 (radial, acteurs satellites)
Architecture en couches (client / API / business / data)	04-developpement/README.md section « Structure de Code »

Élément	Localisation
Stratégie de sécurité	04-developpement/securite-rgpd.md
Justification choix techniques	PLAN_RNCP_COMPLET.md section 4.3
Versions simplifiées des diagrammes pour la slide	07-annexes/diagrams/

À montrer en soutenance : - Le schéma d'architecture haut niveau (3 couches : Client / API / Data + services Cloudinary/Redis/Postgres) - Le diagramme de séquence SSE + Redis pour le temps réel - Le diagramme de cas d'utilisation radial (User central)

15.3.3 Compétence 7 — Concevoir et mettre en place une base de données relationnelle

Méthode Merise (MCD, MLD, MPD), dictionnaire de données, contraintes, intégrité

Couverture : Démontré

Preuves concrètes :

Élément	Localisation
MCD (Modèle Conceptuel)	03-conception/diagrammes-uml.md §1 (DBML pour dbdiagram.io)
MLD (Modèle Logique)	idem §2 (DBML SQL-like)
MPD (Modèle Physique PostgreSQL)	idem §3 (DDL complet)
Diagramme de classes UML	idem §4 (18 entités, attributs typés, cardinalités)
Dictionnaire de données complet	03-conception/donnees.md (18 modèles documentés)
6 enums métier	diagrammes-uml.md §4.3
Schema Prisma (source de vérité)	<code>prisma/schema.prisma</code>
Migrations Prisma	<code>prisma/migrations/</code>
Contraintes d'unicité composite	<code>@@unique</code> sur Reaction, Friendship, GroupMember, etc.
Cardinalités	Tableau « Cardinalités principales » dans donnees.md

À montrer en soutenance : - Coller le MCD en DBML dans dbdiagram.io pour générer la visualisation - Le schema.prisma : un modèle Post avec sa relation `@@unique` - Le MPD SQL en exemple (User + Account avec leurs contraintes)

15.3.4 Compétence 8 — Développer des composants d'accès aux données SQL et NoSQL

Requêtes paramétrées, sécurité, performance, ORM, base relationnelle ET non-relationnelle

Couverture : Démontré

Preuves concrètes :

Élément	Localisation
SQL : queries Prisma typées	src/lib/db/queries/ (organisé par domaine : user/, post/, messages/, group/)
Anti-SQLi structurel	Prisma uniquement, aucun \$queryRaw dans le code (vérifié)
Optimisations Prisma : include ciblés, select minimaux	src/lib/db/queries/post/*.ts
Pagination cursor-based	src/lib/db/queries/post/getAllPosts.ts
NoSQL : Upstash Redis (key-value)	src/lib/server/websocket/redis.ts
Patterns clés Redis	latest:chat:{from}:{to}, latest:chat:group:{groupId}
Filtres de visibilité métier	src/lib/db/queries/messages/visibilityFilter.ts
Validation entrées (Zod) avant insertion	36 schémas dans src/lib/schemas/

À montrer en soutenance : - Une query Prisma complexe : `db.post.findMany` avec `include` (comments + reactions count) - Le client Upstash Redis : `redisdb.set(key, value) / redisdb.get(key)` - Le contrôle d'unicité côté Prisma : `@@unique([userId, postId])` empêche les doubles likes

15.4 BLOC 3 — Préparer le déploiement d'une application sécurisée

15.4.1 Compétence 9 — Préparer et exécuter les plans de tests d'une application

Tests unitaires, intégration, E2E, sécurité, jeux d'essai, couverture

Couverture : Partiel — implémentation à étoffer, stratégie documentée

Preuves concrètes :

Élément	Localisation	Statut
Stratégie de tests détaillée	04-developpement/tests-strategy.md	
Configuration Jest + ts-jest	jest.config.ts, jest.globalSetup.ts, jest.globalTeardown.ts	
Test d'intégration auth/register	__tests__/integrations/authentication.test.ts	
Jeux d'essai documentés	tests-strategy.md section « Jeux d'essai »	
Tests unitaires composants UI	À implémenter	
Tests E2E (Playwright)	À implémenter	
Couverture chiffrée	À mesurer	

Position honnête à tenir en soutenance : « Un test d'intégration backend est en place, la stratégie complète est documentée et le setup Jest fonctionne. L'extension de la couverture (tests UI, E2E) est listée en priorité dans le plan d'amélioration. »

À montrer en soutenance : - `bun run test` qui exécute le test d'intégration auth - Le code du test (`__tests__/integrations/authentication.test.ts`) - La stratégie documentée (`tests-strategy.md`)

15.4.2 Compétence 10 — Préparer et documenter le déploiement d'une application

Documentation déploiement, environnements, secrets, observabilité

Couverture : Démontré

Preuves concrètes :

Élément	Localisation
Section déploiement complète	05-déploiement/README.md
Dockerfile multi-stage production	Dockerfile
<code>docker-compose.yml</code> pour environnement local	<code>docker-compose.yml</code>
Documentation variables d'env	<code>.env.exemple</code> + section 5 du dossier
Stratégie de migrations	Prisma migrate deploy au boot du conteneur

Élément	Localisation
Plateforme cible : Vercel	section 5
BDD : Neon PostgreSQL serverless	section 5
Cache + temps réel : Upstash Redis	section 5
Stockage médias : Cloudinary	section 5
Rollback documenté	<code>vercel rollback</code>
Section observabilité (logs, Vercel Analytics, Sentry roadmap)	section 5
README projet pour le setup	<code>/README.md</code> racine + <code>/dossier-certification/README.md</code>

À montrer en soutenance : - L'URL de production en live - Le Dockerfile multi-stage (builder → runner) - Le `docker-compose up` qui démarre tout local - La page Vercel du déploiement avec son historique

15.4.3 Compétence 11 — Contribuer à la mise en production dans une démarche DevOps

CI/CD, automatisation, intégration continue, livraison continue

Couverture : Démontré

Preuves concrètes :

Élément	Localisation
Pipeline CI documenté (GitHub Actions)	05-déploiement/README.md section CI/CD
Steps lint / test / build / deploy	workflow YAML
Image Docker basée sur Bun (<code>oven/bun:1</code>)	Dockerfile
Auto-déploiement Vercel sur push main	section 5
Issue CI dédiée	#45 CI — Lint, test, build, push image
Issue DevOps	#40
Seed script demo data	#46 , <code>prisma/seed.ts</code>
Migrations auto au boot conteneur	Dockerfile CMD final
Healthcheck endpoint	<code>src/app/api/private/health/route.ts</code>

À montrer en soutenance : - Le workflow GitHub Actions sur le repo - Le déclenchement automatique d'un build Vercel à un commit - Le healthcheck GET /api/private/health qui répond OK - Le seed script qui peuple la BDD avec de la donnée de démo

15.5 Compétences transverses

15.5.1 Sécurité de l'application

Le mot « sécurisée » apparaît dans l'intitulé des 3 blocs RNCP. C'est transverse.

Couverture : Démontré + audit honnête

Toutes les mesures, risques et plan de durcissement sont dans [04-developpement/securite-rgpd.md](#) :

- Authentification stateless JWT, bcrypt 12, cookies httpOnly+secure+sameSite
 - Middleware deny-by-default
 - OAuth Google avec state cookie
 - Validation Zod systématique
 - SQLi écartée structurellement par Prisma
 - Conformité RGPD : minimisation, droits, lacunes assumées (droit à l'oubli, portabilité)
 - Roadmap de durcissement 15 actions priorisées
-

15.6 Vue d'ensemble — couverture estimée

Bloc	Compétences	Démontré	Partiel	Non couvert
Bloc 1 (Développer)	4	4	0	0
Bloc 2 (Concevoir + couches)	4	4	0	0
Bloc 3 (Déployer)	3	2	1 (tests)	0
Total	11	10	1	0

Honnêteté à tenir en soutenance : la couverture des tests est partielle (un seul test d'intégration backend implémenté à ce jour). Le reste des compétences est démontrable par les fichiers cités ci-dessus.

15.7 Recommandation de lecture pour le jury

1. Commencer par ce document pour avoir la **carte des preuves**.
2. Lire la [section Sécurité / RGPD](#) pour le caractère transverse « sécurisée ».
3. Suivre l'ordre du dossier : 00 → 07.
4. À chaque compétence questionnée pendant l'entretien technique, revenir à cette table → pointer le fichier / extrait correspondant.

16 Contribution personnelle de Christophe Lecart

16.1 Pourquoi ce document

Le projet Social Network est **réalisé en équipe** (6 contributeurs) sur le repo [arocchet/social-network](#). Le repo central appartient à un membre de l'équipe (Adrien Roc, alias arocchet), pas au candidat.

Pour anticiper la question légitime du jury — « **qu'est-ce que VOUS avez fait dans cette équipe ?** » — ce document détaille la contribution personnelle de Christophe Lecart par domaine, avec hashes de commits vérifiables sur le dépôt.

16.2 Vue d'ensemble

57 commits sous les identités Christophe Lecart (6) + CLecart (10) + clecart (41), ce qui fait de Christophe le **2^e contributeur** sur 6.

Sortie de `git shortlog -sn`:

```
68  Gabriel K
57  Christophe Lecart (cumul des 3 identités git)
48  Adrien roc (cumul)
45  Pierre Caboor (cumul)
28  Zanakan12 (cumul)
3   Raftandjani
```

Sources de vérité (commandes à reproduire devant le jury) :

```
# Tous les commits Christophe
git log --author="lecart\|Lecart\|clecart\|CLecart" --pretty=format:"%h|%ad|%s"
↩ --date=short

# Statistiques globales équipe
git shortlog -sn
```

16.3 Contribution par domaine

16.3.1 1. Système de réactions universel (posts, stories, commentaires)

Périmètre : fonctionnalité métier complète. Conception du modèle, schémas Zod, queries Prisma, composants UI, intégration temps réel, contraintes d'unicité côté DB.

Commits clés (ordre chronologique) :

Hash	Date	Message
158d8ee	2025-07-08	feat: Universal likes system implementation - Part 1
b140030	2025-07-08	add likesCaches.ts
bd56f13	2025-07-09	Reactions tracking par contentId/storyId/commentId dans useEffect
98ba457	2025-07-10	Remplacement likeCache par useState reactions + isLiked + likesCount
ac9e7f8	2025-07-15	like story, post, comment part 2
2356107	2025-07-21	feat: add frontend reaction components and schemas (part 1)
1553b17	2025-07-22	feat: reactions et reaction counts pour commentaires dans getPostById query
e28ff5a	2025-07-23	feat: improve reaction menu positioning and code structure
0ca9588	2025-07-23	fix: implement real-time reaction counts for posts, stories, and comments
f0371ae	2025-07-28	feat: show reactions popup only on thumb hover for comments
931065b	2025-07-28	UX reactions popup hover

Compétences RNCP couvertes :

- **C3 — Développer des composants métier** : logique complète d'un cas d'utilisation (réagir / retirer / compter / restreindre)
- **C7 — BDD relationnelle** : conception du modèle `Reaction` avec 3 contraintes d'unicité composite (`@@unique([userId, postId])`, (`userId, storyId`), (`userId, commentId`))
- **C8 — Composants d'accès aux données** : queries Prisma optimisées pour récupérer les reactions avec leurs counts

À montrer en soutenance : la table `Reaction` dans `prisma/schema.prisma` + un commit comme `158d8ee` qui pose les bases du système, puis `0ca9588` qui ajoute le temps réel.

16.3.2 2. Design system et thème (palette, dark mode, layout)

Périmètre : mise en place de l'identité visuelle du projet — palette dark, theme provider, cohérence des composants.

Commits clés :

Hash	Date	Message
14f9b4a	2025-05-26	theme provider added
5399254	2025-05-26	select colors palette added
f3ba6c2	2025-05-27	theme color login page
9f83b23	2025-05-27	better format code
9c3110b	2025-06-03	refine layout with theme colors and borders
5218ef9	2025-06-27	feat: add dark mode toggle to GIF picker

Compétences RNCP couvertes :

- C2 — Développer des interfaces utilisateur : design system, cohérence visuelle, accessibilité par thème (clair/sombre)

À montrer en soutenance : la palette CSS dans `globals.css` (variables `--blue40`, `--bgLevel1` à `--bgLevel5`, etc.) + le switch dark/light en démo.

16.3.3 3. Création de posts (dialog responsive, médias multiples)

Périmètre : composant dialog pour créer un post avec gif / photo / vidéo / emojis, prévisualisation médias, upload Cloudinary.

Commits clés :

Hash	Date	Message
ef58dc6	2025-06-05	link createPost
e8807ba	2025-06-05	createPost gif photos videos smiles part 1
0533643	2025-06-05	createPost apiKey .env part 2
1599ff1	2025-06-05	createPost apiKey .env part 3

Hash	Date	Message
ce0c26a	2025-06-06	feat(post): Implement style create post dialog with responsive media previews
f7e9fb9	2025-06-18	add GIF, AVIF format picture, realtime refetch posts and stories

Compétences RNCP couvertes :

- **C2 – Développer des interfaces utilisateur** : composant interactif responsive
- **C3 – Développer des composants métier** : flux d'upload média multi-types
- **Sécurité** : intégration `.env` (API key Cloudinary jamais committée)

16.3.4 4. Stories (timer, background dominant, likes)

Commits clés :

Hash	Date	Message
313d1d8	2025-06-02	feat: auto-detect background color and resize image for stories
cc5e56a	2025-06-03	fix: resolve story timer skipping issues with dominant color background
14c3c33	2025-07-07	feat: Implement story likes system and remove GIF from post creation
7ede4dd	2025-06-24	create api endpoint for reaction stories

Compétences couvertes : C2 + C3.

16.3.5 5. Chat (bulles, emoji picker, layout)

Commits clés :

Hash	Date	Message
8c18881	2025-06-11	fix(chat): emoji picker integration and adaptive message bubbles
eb3434f	2025-06-11	feat(chat): rotate chat bubble tail 65° for style message appearance
d803363	2025-06-12	fix messages display / api/post added

Compétences couvertes : C2 + C3.

16.3.6 6. Recherche stylée avec filtres

Commits clés :

Hash	Date	Message
03d4237	2025-06-10	feat(search): implement style search page with filtering

Compétences couvertes : C2.

16.3.7 7. Commentaires (rework UX)

Commits clés :

Hash	Date	Message
3a399b2	2025-06-03	rework comments part 1
bcae240	2025-06-03	rework comment part 3
1553b17	2025-07-22	reactions et counts dans getPostById (part 2 du système réactions)

Compétences couvertes : C2 + C3.

16.3.8 8. Profil et navigation

Commits clés :

Hash	Date	Message
c998eb5	2025-06-04	reorganize profile page structure
1b07caa	2025-06-17	Add mobile navbar avatar and fix TS errors
5baa12e	2025-06-06	link settings icon laptop and mobile

Compétences couvertes : C2.

16.3.9 9. Sécurité et authentification (fix cookie JWT)

Commits clés :

Hash	Date	Message
f582882	2025-06-12	fix(api): post authentication via cookie JWT and userId verification

Compétences couvertes : C3 (cryptographie) + transverse sécurité.

16.3.10 DevOps et stabilisation (Prisma, dépendances)

Commits clés :

Hash	Date	Message
7f8b206	2025-07-28	pull db prisma
c3748ea	2026-04-15	fix: resolve dependency conflicts and update packages for compatibility
06fe9e5	2026-05-04	docs(conception): aligner UML sur prisma schema

Compétences couvertes : C1 + C11.

16.3.11 1.1. Dossier de certification (intégralité)

Le dossier **dossier-certification/** est réalisé à 100 % par Christophe. C'est lui qui a posé la structure, rédigé toutes les sections, créé les diagrammes UML, et organisé les preuves GitHub.

Commits clés :

Hash	Date	Message
537b482	2026-05-04	docs: initialiser structure dossier de certification
59f743a	2026-05-04	docs(conception): ajouter pages, user stories et modélisation données
877c81b	2026-05-04	docs(developpement): ajouter architecture, spec API et code samples
60dc01b	2026-05-04	docs(certification): finaliser déploiement bilan et annexes
62898bc	2026-05-04	docs(certification): polir le dossier pour la soutenance
7cd4aa3	2026-05-18	certification
0e4066d	2026-05-18	certification pdf

Compétences couvertes : transverses C4 (gestion de projet – formalisation, documentation, traçabilité) + C10 (documenter le déploiement).

16.4 Synthèse — couverture personnelle des 11 compétences

#	Bloc	Compétence	Contribution Christophe
1	B1	Installer / configurer l'environnement	Contribution équipe, j'ai participé aux migrations Prisma + résolution conflits deps

#	Bloc	Compétence	Contribution Christophe
2	B1	Développer des interfaces utilisateur	Forte : design system, theme, créations de posts, chat, profil, recherche, commentaires
3	B1	Développer des composants métier	Très forte : système de réactions universel (cœur métier) + posts + stories
4	B1	Contribuer à la gestion d'un projet	Forte : organisation du dossier de certification, structuration du suivi
5	B2	Analyser les besoins et maquetter	Forte (rédaction user stories + relecture maquettes)
6	B2	Définir l'architecture	Contribution équipe sur l'archi, forte sur la rédaction (UML, diagrammes radial)
7	B2	Concevoir une BDD relationnelle	Forte : conception du modèle Reaction avec contraintes uniques, alignement Prisma↔dossier
8	B2	Composants d'accès aux données	Forte : queries Prisma pour reactions/posts/stories
9	B3	Plans de tests	Partiel équipe
10	B3	Préparer/documenter le déploiement	Forte : intégralité de la section 05-déploiement du dossier
11	B3	Mise en production DevOps	Contribution équipe, forte sur la documentation du flow

Lecture honnête : Christophe a une **contribution forte sur 7 des 11 compétences**, et une contribution équipe sur les 4 autres (qu'il peut néanmoins expliquer car il en a rédigé la documentation).

16.5 Phrase d’ancrage pour la soutenance

« Le projet est porté par une équipe de 6 personnes. Je suis le 2^e contributeur avec 57 commits, principalement sur le système de réactions universel (posts/stories/commentaires), la création de posts multi-médias, le design system, et l’intégralité de ce dossier de certification que vous lisez. Je peux pointer sur GitHub n’importe quel commit ou fichier dont je parle. »

16.6 Liens directs vérifiables

- Repo : [arocchet/social-network](#)

- Liste des commits Christophe :

```
git log --author="Christophe Lecart" --pretty=format:"%h %s"
git log --author="CLecart" --pretty=format:"%h %s"
git log --author="clecart" --pretty=format:"%h %s"
```

- Statistiques équipe : `git shortlog -sn`
- Système de réactions (commit pivot) : [158d8ee](#)
- Refacto Prisma : [06fe9e5](#)
- Fix JWT cookie auth : [f582882](#)